

UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Manresa School of Engineering

Design of a Mesh Communication System Between Mobile Phones for Catastrophic Situations

Manresa, July 3, 2026

Capstone Project (24 ECTS) submitted by

Aarón Alcalá Alcaina

in partial fulfillment of the requirements for the

Bachelor's Degree in ICT Systems Engineering

Advisor: Aleix Llusà Serra

Topics: Mobile Networks, Mesh Networking, Disaster Communication, Wireless Communication Protocols

This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 License. To view a copy of this license, visit `https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode` or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

This manuscript has been typeset using LaTeX and the class `tfgitic` version 8.2

Declaration of Responsibility

I, Aarón Alcalá Alcaina, author of this Capstone Project,

DECLARE

That this project and the accompanying report are original and solely the result of my work. Furthermore, all sources consulted have been included in the bibliography.

By submitting this report for evaluation, I consider this declaration signed for the purposes outlined in the agreement CG/2019/05/10, dated October 8, 2019, of the Governing Council of the Universitat Politècnica de Catalunya, which approves the procedure for plagiarism prevention.

Manresa, July 3, 2026

Declaration on the Use of Generative AI Tools

I, Aarón Alcalá Alcaina, author of this Capstone Project,

DECLARE

That various generative artificial intelligence tools were utilized to support both the technical development and the writing of this document, specifically ChatGPT (OpenAI, GPT-5.5) and Claude Code (Anthropic).

These tools were primarily applied to the following aspects of the project:

- Assisting with the phrasing, clarity, and structural organization of several sections throughout the document (ChatGPT).
- Developing components of the mobile application (Claude Code).
- Generating and refining C++ code fragments (Claude Code).

All content generated with the assistance of AI tools has been thoroughly reviewed, verified, and edited by the author prior to its inclusion in the final version of this document.

Consequently, the final content of this work accurately represents the technical and intellectual contributions made by the author.

The author declares under their sole responsibility that:

- The content of this work is truthful, complete, and accurate.
- Generative artificial intelligence tools have been used transparently and ethically.
- Full responsibility for the ultimate authorship and content of this work is strictly assumed.

Manresa, July 3, 2026

Acknowledgments

I would like to express my deepest gratitude to my family and friends for their unconditional support throughout this project. Special thanks go to those who helped me test the system, patiently trying out the application in different scenarios and devices to make sure everything worked as expected. Their feedback was invaluable in identifying issues and improving the final result.

I am also profoundly grateful for their constant encouragement and understanding during the more demanding stages of this work, which helped me stay motivated and see this project through to completion.

Abstract

Modern messaging systems depend almost entirely on cellular networks and Internet infrastructure, a dependency that becomes critical during disasters, blackouts, or in areas without coverage—precisely when reliable communication matters most. This thesis presents *A3MESH*, a decentralized communication platform based on LoRa and BLE that allows smartphones to exchange messages through a multi-hop mesh network without relying on any pre-existing infrastructure.

The project integrates a compact, low-power PCB (ESP32, SX1276 LoRa transceiver, custom antenna), a layered, modular firmware, and an Android application that hides the network's complexity behind an experience similar to conventional messaging. Its central contribution is *Smart Gossip*, an adaptive forwarding algorithm that adjusts each node's relay probability based on link quality (RSSI), duplicate messages, and neighbor density, overcoming the congestion and energy limitations of traditional flooding. Its parameters were tuned through evolutionary optimization on a simulator built upon Meshtasticator, with results showing reduced delay and congestion versus conventional flooding and a comparable delivery ratio. The work is completed with a portable enclosure and experimental validation in laboratory and real-world conditions.

Resum

La dependència actual dels sistemes de missatgeria respecte a les xarxes cel · lulars i la infraestructura d'Internet els fa vulnerables davant de catàstrofes, apagades o zones sense cobertura, justament quan la comunicació és més crítica. Aquest treball presenta *A3MESH*, una plataforma de comunicació descentralitzada basada en LoRa i Bluetooth Low Energy (BLE) que permet a telèfons mòbils intercanviar missatges mitjançant una xarxa mallada multi-salt, sense necessitat de cap infraestructura prèvia.

El projecte integra una PCB compacta de baix consum (ESP32, transceptor LoRa SX1276 i antena pròpia), un firmware modular i estratificat, i una aplicació Android que abstrau la complexitat de la xarxa amb una experiència similar a la missatgeria convencional. La principal aportació és l'*Smart Gossip*, un algorisme adaptatiu d'encaminament que ajusta la probabilitat de retransmissió segons la qualitat de l'enllaç (RSSI), els missatges duplicats, i la densitat de veïns, superant les limitacions de congestió i consum de la difusió tradicional. Els seus paràmetres s'han optimitzat evolutivament sobre un simulador basat en Meshtasticator, amb resultats que mostren menys retard i congestió que la difusió clàssica i una taxa de lliurament comparable. El treball es completa amb una carcassa mecànica i la validació experimental en laboratori i en condicions reals.

Contents

Abstract iv

Resum iv

I. Thesis Body 1

1. Introduction 2
1.1. Motivation . . . 2
1.2. Objectives . . . 3
1.3. Scope and Limitations . . . 3
1.4. Contributions . . . 4
1.5. Thesis Structure . . . 4

2. Background 6
2.1. Mesh Communication Systems . . . 6
2.2. LoRa and LPWAN Technologies . . . 7
2.2.1. Introduction to LPWAN . . . 7
2.2.2. LPWAN versus LoRa: Clarifying the Terminology . . . 7
2.2.3. Fundamentals of LoRa . . . 7
2.2.4. Fundamental Trade-offs . . . 9
2.2.5. Practical Limitations of LoRa . . . 10
2.2.6. Rationale for LoRa Selection . . . 10
2.3. Bluetooth Low Energy . . . 11
2.3.1. Architecture . . . 11
2.3.2. The Generic Attribute Profile (GATT) . . . 12
2.3.3. Low-Power Operation . . . 12
2.3.4. BLE as the Smartphone Bridge . . . 12
2.4. Existing Solutions . . . 12
2.4.1. Meshtastic . . . 13
2.4.2. LocalMesh and MeshCore . . . 13
2.5. Gossip-Based Routing Protocols . . . 14
2.5.1. The Flooding Problem . . . 14
2.5.2. Probabilistic Forwarding . . . 14
2.5.3. Controlled Gossip . . . 15
2.5.4. Adaptive Gossip and the Research Gap . . . 15

3. Main Content 16
3.1. System Architecture . . . 16
3.1.1. General Architecture . . . 16
3.1.2. Node Interaction Model . . . 17

3.1.3. Communication Flow . . . 17
3.1.4. Design Requirements . . . 19
3.2. Development Methodology and Project Management . . . 20
3.2.1. Development Workflow . . . 20
3.2.2. Agile Methodology . . . 20
3.2.3. Sprint Planning with Jira . . . 21
3.2.4. Version Control with Git . . . 21
3.2.5. Repository Organisation . . . 22
3.3. Network Simulation and Routing Algorithm Design . . . 22
3.3.1. Purpose of the Simulator . . . 22
3.3.2. Simulation Environment . . . 23
3.3.3. Evaluated Routing Strategies . . . 24
3.3.4. Smart Gossip Algorithm . . . 25
3.3.5. Evolutionary Optimization Process . . . 26
3.3.6. Performance Metrics . . . 28
3.3.7. Simulation Results and Analysis . . . 29
3.3.8. Final Parameter Selection . . . 33
3.4. Hardware Design . . . 34
3.4.1. Hardware Requirements . . . 34
3.4.2. System Block Diagram . . . 35
3.4.3. Power Subsystem . . . 35
3.4.4. Processing Subsystem . . . 37
3.4.5. RF Subsystem . . . 39
3.4.6. USB Programming Interface . . . 40
3.4.7. PCB Design . . . 41
3.4.8. Prototype Manufacturing . . . 42
3.4.9. Mechanical Integration . . . 42
3.5. Firmware Architecture and Implementation . . . 43
3.5.1. Firmware Design Philosophy . . . 43
3.5.2. Layered Architecture . . . 45
3.5.3. Physical Communication Layer . . . 46
3.5.4. Mesh Logic Layer . . . 49
3.5.5. System Services Layer . . . 52
3.5.6. Application and Interface Layer . . . 53
3.5.7. Security Architecture . . . 55
3.6. Mobile Application Development . . . 56
3.6.1. Application Objectives . . . 56
3.6.2. User Experience Design . . . 57
3.6.3. Android Communication Architecture . . . 57
3.6.4. BLE Integration . . . 58
3.6.5. Messaging Interface . . . 59
3.6.6. Contact and Node Management . . . 60
3.6.7. Offline Interaction Model . . . 60
3.6.8. Development Strategy . . . 60
3.7. Experimental Results and Evaluation . . . 61
3.7.1. Test Methodology . . . 61
3.7.2. Range Evaluation . . . 61

3.7.3. Mesh Relay Demonstration . . . 61
3.7.4. Power Consumption Analysis . . . 63

4. Economic Study **64**
4.1. Labour Cost . . . 64
4.2. Materials Cost per Unit . . . 64
4.3. Total Project Cost . . . 65

5. Conclusions **67**
5.1. Achieved Objectives . . . 67
5.2. Main Contributions . . . 67
5.3. Lessons Learned . . . 68

6. Future Work **69**

Bibliography **71**

II. Appendices **77**

A. AT Command Reference **78**
A.1. Basic Command . . . 78
A.1.1. `AT` — Connectivity check . . . 78
A.2. Messaging Commands . . . 79
A.2.1. `AT+SEND` — Transmit plain (unencrypted) message . . . 79
A.2.2. `AT+SENDE2E` — Transmit end-to-end encrypted message . . . 79
A.2.3. `AT+SENDGRP` — Transmit group-encrypted broadcast . . . 79
A.2.4. `AT+SENDNET` — Transmit network-layer encrypted message . . . 79
A.2.5. `AT+SENDKX` — Transmit key-exchange packet . . . 80
A.3. Query Commands . . . 81
A.3.1. `AT+NODEID?` — Query local node identifier . . . 81
A.3.2. `AT+NEIGHBORS?` — Query neighbor table . . . 81
A.3.3. `AT+INFO?` — Query system diagnostics . . . 81
A.3.4. `AT+LORA?` — Query LoRa radio configuration . . . 82
A.4. Radio Configuration Commands . . . 82
A.4.1. `AT+LORA` — Set LoRa radio parameters . . . 82
A.5. Key Management Commands . . . 82
A.5.1. `AT+SETNETKEY` — Provision shared network key . . . 82
A.5.2. `AT+SETPEERKEY` — Provision node-to-node peer key . . . 82
A.5.3. `AT+SETGRPKEY` — Provision group key . . . 83
A.5.4. `AT+CLRKEYS` — Clear all provisioned keys . . . 83
A.6. Unsolicited Notifications . . . 84

B. PCB Schematics **85**

C. Bill of Materials **89**

D. Source Code Repositories **91**
D.1. Node Firmware 91
D.2. PCB Hardware Design 92
D.3. Android Companion Application 93
D.4. Network Simulator 94
D.5. Mechanical Enclosure 94

Part I.

Thesis Body

1. Introduction

Modern smartphones excel at communication as long as an operator's cellular network or a working internet connection is available. When either fails—during a natural disaster, a large-scale power outage, or simply out in the countryside where there is no coverage—the device becomes essentially mute for messaging purposes, precisely when reaching family, friends, or emergency services matters most. This thesis addresses that gap by combining three familiar technologies: *mesh networking*, in which every device forwards traffic for its neighbours instead of relying on a central tower; *offline operation*, meaning that the system works without any pre-existing infrastructure or internet access; and two low-power radios, *LoRa* for the multi-kilometre links between nodes and *Bluetooth Low Energy* (BLE) for the short link between each smartphone and its companion node.

The concrete outcome of this work is a platform called *A3MESH*. It consists of a pocket-sized custom electronic board that can be adhered to the back of a phone case and paired to the phone over BLE, an Android application that behaves like a conventional messaging app, and a novel forwarding algorithm—*Smart Gossip*—that lets these nodes cooperate as a self-organising mesh with no configuration on the part of the user. Together they enable two smartphones to exchange encrypted text messages across several kilometres of urban or rural terrain, hopping through intermediate nodes when necessary, without any cellular network, Wi-Fi, or internet connection being involved.

1.1. Motivation

The dependence of modern messaging on cellular networks and cloud services turns every centralized infrastructure into a Single Point of Failure (SPOF). The Spain-wide blackout of April 2025 made this concrete: for several hours a large fraction of the population lost the ability to reach family, friends, or emergency services precisely when reaching them mattered most [Ála+22]. Similar failure modes recur during fires, floods, earthquakes, and prolonged power outages, and they extend permanently to remote areas where no coverage exists in the first place.

The response from both the research community and the telecommunications industry has been to explore infrastructure-independent alternatives. Open-source projects such as Meshtastic have demonstrated that low-cost LoRa hardware can sustain useful messaging without any operator involvement [Mes26], and, at the 2026 Mobile World Congress, established vendors such as Viasat and Qualcomm presented satellite and hybrid connectivity solutions targeting the same class of scenarios [Reu26]. In both cases the technical viability is no longer in question; the remaining barrier is usability. Existing infrastructure-independent platforms require non-trivial technical configuration and therefore exclude the very users who most benefit from resilient communication.

This thesis therefore treats usability as a first-class design requirement alongside range, autonomy, and reliability. Smartphone integration, a self-configuring mesh, and a familiar user

interface are pursued as explicit goals, so that the resulting platform can be adopted without any radio-networking knowledge.

1.2. Objectives

The overarching goal of this thesis is to design and validate a decentralized low-power mesh communication platform that remains usable by non-technical users. This goal decomposes into the following specific objectives:

- Design a portable hardware platform based on LoRa and BLE that supports multi-hop mesh communication.
- Develop a modular firmware architecture covering packet forwarding, neighbor discovery, power management, and the BLE–LoRa communication pipeline.
- Investigate the scalability limits of flooding-based mesh dissemination and evaluate alternative routing approaches through simulation.
- Design and implement a routing algorithm that adjusts its forwarding behavior according to locally observed network conditions.
- Provide a user experience comparable to conventional messaging applications by hiding all networking complexity behind a smartphone client.
- Design a compact enclosure that integrates naturally with the user's existing smartphone.
- Validate the platform through simulation, laboratory measurements, and real-world communication experiments.

1.3. Scope and Limitations

The work covers the complete end-to-end system, from the custom hardware and embedded firmware, through the routing simulator, to the Android application that acts as the user interface. Its scope therefore spans low-power wireless communication, mesh routing, embedded systems, mobile software, and mechanical integration.

Several deliberate limitations frame the project. First, the platform is a research prototype: industrial certification, mass production, and environmental qualification lie outside its scope. Second, the mobile application targets Android only, since a cross-platform port would have displaced attention from the networking work at the core of the thesis. Third, the deliberate absence of Internet gateways, which is precisely what makes the system resilient, also excludes interoperability with conventional messaging services. Fourth, the scalability results reported in Chapter 3.7 rest on simulation and on small-scale field deployments; validating them at the scale of hundreds of physical nodes was not feasible within a bachelor thesis. Finally, the cryptographic layer is designed to protect practical privacy in a low-power decentralized setting rather than to provide certified security guarantees.

1.4. Contributions

The main contributions of this thesis are:

- *Smart Gossip*, an adaptive forwarding algorithm that computes each node's relay probability from locally observable signals (RSSI, neighbor count, and duplicate copies overheard), together with an evolutionary tuning procedure [ES03] that optimizes its parameters on a simulator built for this purpose.
- A custom LoRa mesh simulator, extending Meshtasticator with a pluggable routing interface and a batch evaluation harness that produces reproducible, statistically comparable results across routing strategies and network densities.
- A complete hardware and firmware platform: a compact ESP32-based board with a PCB antenna adapted from Texas Instruments Design Note DN024 [Tex08], and a layered firmware that implements the BLE–LoRa pipeline, neighbor discovery, adaptive power management, and end-to-end encryption.
- An Android application that exposes the platform through an interface indistinguishable from a conventional messaging application, keeping the mesh entirely invisible to the end user.
- Experimental validation of the platform through simulation, laboratory measurements, and field tests.

Against a Managed Flooding baseline, Smart Gossip reduces air transmissions by 27–32 %, propagation delay by 58–86 %, and packet collisions by 18–33 % across networks of 15–50 nodes, at the cost of a delivery-ratio penalty of 4.5–12.4 percentage points. A composite performance score favors Smart Gossip in three out of four evaluated network sizes.

1.5. Thesis Structure

The remainder of this document is organized as follows. Chapter 1 introduces the motivation, objectives, scope, and main contributions of the project. Chapter 2 then reviews the technological and theoretical background: mesh communication as an architectural concept, LoRa and the wider LPWAN family, Bluetooth Low Energy, and the existing off-grid mesh platforms and gossip-based routing protocols on which this work builds.

The core technical development of the thesis follows. Section 3.1 presents the overall system architecture and communication model. Section 3.2 summarizes the engineering workflow and project management strategy adopted throughout the project. Section 3.3 describes the development of the network simulator, the proposed Smart Gossip routing algorithm, and the evolutionary optimization process used for parameter tuning. Section 3.4 details the custom low-power communication board, including its mechanical integration. Section 3.5 explains the layered embedded architecture implementing mesh networking functionality, communication pipelines, and power management mechanisms. Section 3.6 describes the Android-based user interface and smartphone integration approach. Finally, Section 3.7 analyzes the performance of the proposed system through simulation and real-world testing.

Chapter 4 presents the economic study of the project, breaking down engineering effort, materials cost, and total project cost. Chapter 5 summarizes the main conclusions obtained

during the development of the project, discusses the achieved objectives and contributions, and Chapter 6 proposes possible future improvements and research directions.

The document also includes several appendices containing complementary technical information: the complete AT-command reference (Appendix A), the electrical schematic and copper layers of the printed circuit board (Appendix B), the full bill of materials (Appendix C), and the source-code repository organization (Appendix D).

2. Background

This chapter provides the technical background required to place A3MESH within the wider landscape of decentralized communication systems. Section 2.1 reviews mesh communication as an architectural concept and identifies the design tensions inherent to it. Section 2.2 introduces LoRa and the LPWAN family, focusing on the modulation scheme, its configurable parameters, and the regulatory limits that constrain any real deployment. Section 2.3 covers Bluetooth Low Energy in the aspects relevant to its role as the smartphone-to-node link. Sections 2.4 and 2.5 then survey the mesh platforms most closely related to this work—Meshtastic, MeshCore, and LocalMesh—and review the gossip-based routing protocols on which the Smart Gossip algorithm of Chapter 3.3 is built.

2.1. Mesh Communication Systems

In a mesh network, the participating devices—*nodes*—cooperate as peers to exchange information without a fixed centralized infrastructure [AWW05; MM04]. Messages that cannot reach their destination in a single radio hop are relayed through intermediate nodes; new nodes can join at any time to extend coverage, and paths reconfigure themselves as the topology evolves. Figure 2.1 contrasts this arrangement with the star topology on which cellular and Wi-Fi networks rely, and with the degenerate point-to-point link at the opposite extreme.

Depending on the routing discipline, packet relaying may rely on deterministic routing tables, probabilistic forwarding, flooding, or adaptive strategies. Each has its own trade-off, but the fundamental tension is the same throughout the mesh literature: increasing the number of nodes should improve connectivity, yet every additional relay competes for the same shared channel. Redundant retransmissions therefore inflate congestion, collisions, and energy consumption, and this effect is particularly pronounced in low-power wireless systems whose battery and bandwidth budgets are tight [MM04]. Additional difficulties include topology instability under mobility and the greater coordination overhead of decentralized systems.

Centralized architectures avoid these issues at the price of a single point of failure and a permanent dependency on operator infrastructure. Mesh networks trade some of that simplicity and global optimality for resilience and infrastructure independence.

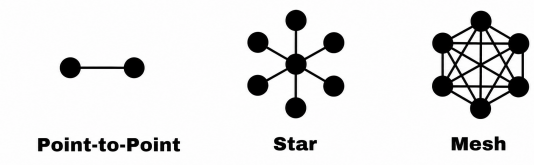


Figure 2.1.: Centralized star topology, decentralized mesh topology, and point-to-point link.

2.2. LoRa and LPWAN Technologies

2.2.1. Introduction to LPWAN

The Internet of Things (IOT) has created a class of communication requirements that traditional wireless technologies were never designed to satisfy: thousands of inexpensive, battery-powered devices, scattered over wide geographical areas, each transmitting only a few bytes of data sporadically. Low-Power Wide-Area Networks (LPWANs) emerged precisely to fill this gap, offering kilometre-scale range and multi-year battery lifetimes at the cost of very low data rates [RKS17]. Short-range technologies such as Wi-Fi, Bluetooth and Zigbee provide high throughput but only over tens of metres, while conventional cellular networks offer coverage at the expense of complex protocols, subscription costs and energy budgets incompatible with *coin-cell operation* [Mek+19]. LPWAN deliberately accepts the opposite trade-off: it sacrifices throughput in exchange for range and energy efficiency. This makes it the natural choice for smart cities, precision agriculture, distributed sensing, remote monitoring and resilient communications in infrastructure-degraded scenarios. It is important to stress that LPWAN is not a single technology but a *category* of technologies—including LoRa, Sigfox, and NB-IOT—that share these optimization goals [RKS17].

2.2.2. LPWAN versus LoRa: Clarifying the Terminology

Within this landscape, LoRa (Long Range) is one specific LPWAN technology: a proprietary physical-layer (PHY) modulation developed by Semtech [Sem15]. A frequent source of confusion is the distinction between LoRa and LoRaWAN. LoRa defines only the radio modulation, whereas LoRaWAN is an open MAC and network-layer specification, maintained by the LoRa Alliance, that builds a star-of-stars architecture with gateways and network servers on top of the LoRa PHY [LoR17]. Crucially, LoRa can be used *without* LoRaWAN: the raw PHY is accessible to any system designer, enabling custom protocols above it. This is exactly the approach adopted in A3MESH, which operates directly over the LoRa physical layer to implement a decentralized mesh topology rather than the infrastructure-centric model imposed by LoRaWAN. LoRa should therefore be understood as a flexible physical layer on which non-conventional architectures can be built.

2.2.3. Fundamentals of LoRa

LoRa is based on Chirp Spread Spectrum (CSS) modulation, in which each symbol is encoded as a *chirp*: a signal whose instantaneous frequency sweeps linearly across the entire channel bandwidth [Van17]. Unlike narrowband schemes, where all the signal energy is concentrated in a small portion of the spectrum, CSS spreads each symbol over the full bandwidth. This spreading yields a substantial processing gain at the receiver, which correlates the incoming signal against known chirp patterns and effectively integrates the symbol energy over time. The practical consequences are remarkable: strong robustness against thermal noise and narrowband interference, high tolerance to multipath fading and Doppler effects, and—most strikingly—the ability to demodulate signals received *below* the noise floor, with sensitivities down to approximately -137 dBm [Sem15; Aug+16]. The long range of LoRa therefore does not come from transmitting at high power, which is in any case limited by regulation, but from extreme receiver sensitivity achieved through spreading and temporal energy integration.

Symbol Representation in CSS. At the physical layer, a LoRa symbol is not conveyed by selecting a sinusoidal tone, as in conventional FSK schemes, but by choosing one out of 2^{SF} possible cyclic shifts of a reference chirp. In other words, the information content of the symbol is mapped onto the time-frequency position of the chirp within the symbol interval. From a practical perspective, this can also be interpreted as a discrete frequency offset, where each symbol corresponds to a different starting point along the same linear frequency sweep. As a result, all LoRa symbols occupy the full channel bandwidth and have the same duration and average power; what changes from one symbol to another is only the relative shift of the chirp pattern.

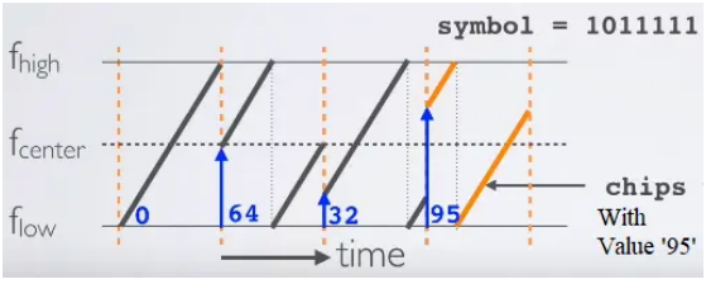


Figure 2.2.: LoRa symbol-to-chirp mapping [RF 24].

At the receiver, demodulation is performed by correlating the incoming waveform with a locally generated reference chirp, a process commonly referred to as *dechirping*. This operation converts the cyclic shift of the received chirp into a tone whose frequency is proportional to the transmitted symbol index. The subsequent FFT stage then identifies the spectral bin associated with that tone, allowing the receiver to recover the symbol value efficiently. This is one of the key reasons why LoRa achieves high sensitivity and robust reception under low signal-to-noise ratios [Wir24].

Implications of the Configurable Parameters

The LoRa PHY therefore exposes a small but highly influential set of parameters that directly shape the link budget, throughput, and robustness of the system. Figure 2.3 illustrates the effect of the main LoRa parameters on a time-frequency waterfall plot.

Spreading Factor (SF). The spreading factor, ranging from SF7 to SF12, determines the number of distinct *cyclic shifts* that can be applied to a chirp, yielding up to 2^{SF} unique symbols. By definition, the symbol duration is $T_s = 2^{SF}/BW$ [Sem15]. Each unit increase in SF roughly doubles the airtime, so its effect on time-on-air, energy consumption and latency is exponential, while sensitivity improves. Higher SFs thus extend range at the cost of drastically reduced bitrate. Different SFs are quasi-orthogonal, allowing partially concurrent transmissions, although this orthogonality is imperfect in practice [Cro+18]. Besides, in a mesh context this trade-off is critical: high SFs improve connectivity between distant nodes but inflate channel occupancy, increasing congestion and the probability of collisions as the network grows [Bor+16].

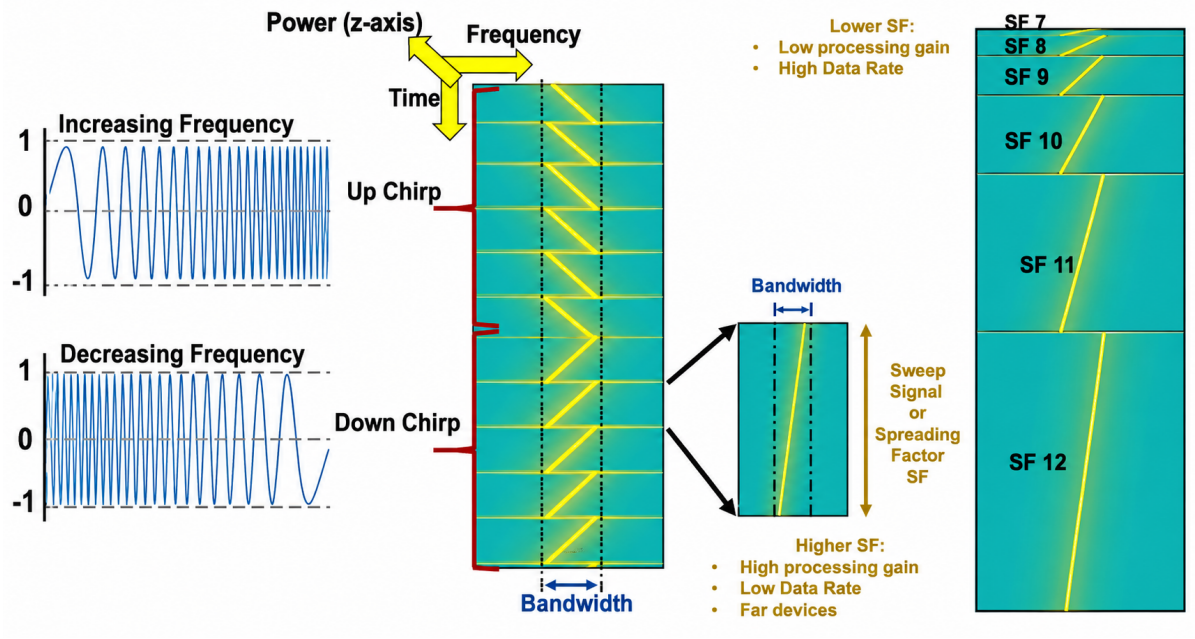


Figure 2.3.: LoRa Chirp Spread Spectrum illustration [Wir24].

Bandwidth (BW). LoRa typically operates with bandwidths of 125, 250, or 500 kHz. Narrower bandwidths capture less thermal noise and therefore improve sensitivity, but they proportionally lengthen symbols and reduce data rate [Aug+16]. As with SF, the designer trades speed for robustness.

Coding Rate (CR). LoRa incorporates Forward Error Correction with coding rates of 4/5, 4/6, 4/7, and 4/8. Higher redundancy improves the receiver's ability to recover corrupted symbols in noisy or interference-prone environments, at the cost of lower effective throughput and longer airtime, i.e. reduced spectral efficiency [Sem15].

Other parameters. Transmission power (typically up to +20 dBm in the European 868 MHz ISM band[1]) directly shapes the link budget, range, and energy consumption. The preamble length controls how reliably receivers detect and synchronize to incoming frames, an important lever for duty-cycled listeners. An optional CRC protects payload integrity, and the sync word provides logical separation between co-located networks sharing the same channel [Sem15].

2.2.4. Fundamental Trade-offs

No LoRa configuration simultaneously maximizes range, throughput, latency, and spectral efficiency. The two extremes of the configuration space illustrate this clearly: SF12 with 125 kHz bandwidth and CR 4/8 yields maximum range and robustness, with airtimes of over a second for modest payloads, whereas SF7 with 500 kHz and CR 4/5 maximizes throughput and

[1]The 27 dBm limit referenced in simulation configurations applies exclusively to the 869.4–869.65 MHz sub-band under ETSI EN 300 220, which permits a 10% duty cycle; the general 868 MHz ISM band is limited to +14 dBm with a 1% duty cycle.

Table 2.1.: Comparison of wireless technologies for IOT and off-grid deployments. LoRa uniquely combines kilometre-scale range, low power, and operator-independent operation [Mek+19; RKS17].

Technology	*Range*	*Data rate*	*Power*	*Infrastructure*
LoRa	1–15 km	0.3–27 kbps	Very low	None (license-free)
Sigfox	3–50 km	100 bps	Very low	Operator network
NB-IOT	1–10 km	up to 250 kbps	Low	Cellular operator
Wi-Fi	<100 m	>100 Mbps	High	Access point
Zigbee	<100 m	250 kbps	Low	Coordinator node
BLE	<100 m	1–2 Mbps	Very low	None

minimizes latency but sharply reduces link margin. Airtime is the variable that couples these choices to system-level behavior: longer frames raise collision probability, degrade scalability, and consume the limited duty-cycle budget (typically 1% in the EU 868 MHz sub-bands under ETSI regulations) [Ade+17; Bor+16].

2.2.5. Practical Limitations of LoRa

A mature assessment must acknowledge LoRa's limitations. Bitrates range only from roughly 0.3–27 kbps, and transceivers are half-duplex, unable to receive while transmitting. High airtimes at large SFs amplify the classical hidden-node problem, since carrier sensing is unreliable over kilometre-scale links, and collisions grow rapidly with network density [Bor+16; Ade+17]. Regulatory duty-cycle restrictions in ISM bands further cap achievable throughput and limit how often nodes may relay traffic. Finally, LoRa was conceived for star topologies, not mesh networking: it provides no native routing, scheduling or neighbour-discovery mechanisms. These shortcomings do not invalidate the technology, but they imply that an efficient mesh network over LoRa requires additional protocol mechanisms at the MAC and routing layers.

2.2.6. Rationale for LoRa Selection

After evaluating alternative technologies, each alternative fails at least one key requirement: Wi-Fi and BLE lack the necessary range; Zigbee meshes well but only over short links; NB-IOT depends entirely on operator-owned cellular infrastructure, an unacceptable single point of failure in disaster scenarios; and Sigfox relies on a proprietary, operator-managed network with severely constrained uplink and almost no downlink capability [Mek+19]. LoRa, in contrast, operates in license-free ISM bands without any operator, offers kilometre-scale links at milliwatt power levels, and exposes a highly configurable PHY that permits experimentation with custom mesh protocols. For these reasons—autonomy, range, energy efficiency, regulatory accessibility, and PHY flexibility—LoRa was selected as the physical layer of this project, with the understanding that the mesh-specific challenges identified above must be addressed by the protocol design presented in the following chapters.

2.3. Bluetooth Low Energy

BLE was introduced in version 4.0 of the Bluetooth Core Specification, is a short-range wireless technology designed from the ground up for ultra-low-power operation rather than as a mere optimization of classic Bluetooth [Blu23; GOP12]. Although it shares the brand and the 2.4 GHz ISM band with its predecessor, BLE is a distinct protocol stack: it uses 40 channels of 2 MHz with GFSK modulation, three of which are reserved for advertising, and it replaces the connection-oriented complexity of classic Bluetooth with a lightweight, asynchronous design oriented toward the sporadic exchange of small amounts of data.

2.3.1. Architecture

The BLE stack is organized into three blocks. The *controller* comprises the physical layer and the link layer, which manages device states (advertising, scanning, initiating, and connected), adaptive frequency hopping, and connection events. The *host* contains the Logical Link Control and Adaptation Protocol (L2CAP), the Attribute Protocol (ATT), the Generic Attribute Profile (GATT), the Security Manager, and the Generic Access Profile (GAP), which defines discovery and connection roles such as *central* and *peripheral*. Both blocks communicate through the Host Controller Interface (HCI) [Blu23; GOP12].

Communication follows two complementary patterns: connectionless broadcasting over the advertising channels, and point-to-point connections in which the central device polls the peripheral at a negotiated *connection interval*, allowing the radio to remain dormant between events.

Advertising and Scanning. To establish a connection, one device must advertise while a second device scans. The advertiser periodically transmits advertising packets on the three dedicated channels; the scanner listens and raises a scan-report event each time such a packet is received [Sil24]. Two scanning modes are available: in *passive* scanning the scanner only receives advertising packets, whereas in *active* scanning it additionally transmits a scan-request frame containing its own Bluetooth address, allowing the advertiser to respond with supplementary data. BLE implements a time-division duplex scheme, so all timing is governed by three tuneable parameters—advertising interval, scanning interval, and scanning window—whose interaction determines discovery latency and energy consumption [Blu23].

Connection Establishment. Once the scanner has collected sufficient information from the advertiser, it may open a connection, at which point it becomes the *central* device and the former advertiser becomes the *peripheral* [Sil24]. A connection-initiation failure can occur in two ways: the central explicitly cancels the pending request, or the procedure times out because the peripheral does not respond within the prescribed interval. The BLE standard defines a hard limit of six consecutive connection intervals during which either host may remain silent; if this threshold is exceeded, the link layer drops the connection [Blu23; Sil24].

Connection Parameter Negotiation. After a connection has been established, both hosts may request adjustments to the link parameters. The connection-parameter-update procedure can be triggered either by the central, the peripheral, or autonomously by the link layer during connection initiation. In all cases the central retains the final authority over parameter selection [Sil24]. A related sub-procedure, the *GATT MTU exchange*, is also performed at this

stage: the ATT protocol defines a default Maximum Transmission Unit (MTU) of 23 bytes, but if both devices support a larger value the MTU exchange procedure negotiates a higher limit, executed exactly once per connection [Blu23].

2.3.2. The Generic Attribute Profile (GATT)

Data exchange in connected mode is structured by GATT, a client–server model built on top of ATT. A GATT server exposes its state as a hierarchy of *services*, each grouping one or more *characteristics*: typed attributes identified by UUIDs that a client can read, write, or subscribe to through *notifications* and *indications* [Blu23]. This self-describing structure permits a client to discover at runtime what data a previously unknown device offers, without any out-of-band agreement. In practice, GATT acts as a universal application-layer contract: any BLE-capable device, including every modern smartphone, can interact with a custom peripheral by knowing only its service and characteristic UUIDs.

2.3.3. Low-Power Operation

BLE achieves multi-year operation on coin-cell batteries through aggressive duty cycling rather than reduced transmit power alone. Connection events and advertising bursts last only a few milliseconds, after which the radio returns to deep sleep; protocol overhead is minimal, and device discovery and connection establishment complete in milliseconds rather than seconds [GOP12; Tos+17]. Parameters such as the connection interval, slave latency, and advertising interval allow the designer to trade latency for energy. Reported peak currents in the range of tens of milliamperes, combined with sleep currents of a few microamperes, yield average power consumptions that are orders of magnitude below those of Wi-Fi or classic Bluetooth for intermittent traffic [Sie+12].

2.3.4. BLE as the Smartphone Bridge

The decisive argument for BLE in this project is ubiquity: native BLE support has been mandatory hardware in essentially every smartphone shipped in the last decade, and both Android and iOS expose mature GATT APIs to applications without any need for pairing infrastructure, routers, or licensed spectrum [GOP12]. Combined with sub-second connection times and negligible radio overhead on the handset, this makes BLE the natural short link between each node and its user: the node acts as a GATT peripheral exposing message characteristics, while the smartphone application, acting as central, becomes the human interface of the mesh without requiring any custom radio hardware on the user's side.

2.4. Existing Solutions

Building multi-hop networks over LoRa is an active area of both academic research and community-driven development. Surveys of the field show that, although LoRaWAN's star-of-stars topology remains dominant, a growing ecosystem of projects explores mesh architectures over the raw LoRa PHY [Pue+21; Won+24]. Among them, two open-source systems are particularly relevant to this thesis because they target the same use case and because they embody two opposite points in the routing design space: Meshtastic, based on network-wide flooding,

and MeshCore (deployed at scale by initiatives such as LocalMesh), based on structured routing through dedicated repeaters.

2.4.1. Meshtastic

Meshtastic is an open-source firmware and application ecosystem that turns inexpensive ESP32-based LoRa boards into an off-grid messaging platform [Mes24a]. Its architecture is deliberately egalitarian: every node runs the same stack, pairs with a smartphone over BLE for the user interface, and—by default—participates in packet forwarding. There is no central infrastructure, no network server and no special node required for the network to function, which makes deployment trivially simple.

Routing in Meshtastic follows a *managed flooding* strategy. A node that originates a message broadcasts it; every node that receives it rebroadcasts it once, until a hop-limit counter (three by default, seven at most) is exhausted [Mes24b]. The flooding is "managed" through a handful of heuristics: duplicate packets are recognized and suppressed via a per-packet identifier cache; rebroadcasts are delayed by a contention window weighted by the received SNR, so that distant nodes—which extend coverage the most—tend to retransmit first; and a node that overhears another copy of the packet during its back-off cancels its own retransmission. Since version 2.6, unicast messages additionally learn a next-hop after the first flooded delivery, although broadcast traffic, which dominates typical usage, continues to flood [Mes24b].

The strengths of this design are clear. Flooding requires no routing state, no topology knowledge and no control traffic; it is maximally robust to mobility and node failure, since any available path is implicitly used; and it guarantees that broadcasts reach every reachable node. These properties explain Meshtastic's popularity for hiking groups, events and emergency-preparedness communities. The weaknesses, however, are equally structural. Every message triggers up to one transmission per node within the hop horizon, so channel occupancy grows with node density rather than with traffic demand—a particularly serious problem over LoRa, where airtime is long and duty-cycle budgets are tight. Experimental and community evidence consistently reports degraded delivery rates and channel saturation as dense meshes grow toward the order of a hundred nodes, aggravated by periodic telemetry and position broadcasts that the protocol itself generates [Won+24; Mes24b]. In other words, Meshtastic inherits the classical *broadcast storm* pathology discussed in Section 2.5, mitigated but not solved by its heuristics.

A3MESH takes Meshtastic's architectural premises—full decentralisation, role-free nodes, BLE smartphone integration—as a baseline, but replaces blind rebroadcasting with a probabilistic, adaptive forwarding discipline. The goal is to retain flooding's robustness and statelessness while decoupling channel load from node density, so that the network degrades gracefully precisely in the dense, high-stress scenarios where flooding performs worst.

2.4.2. LocalMesh and MeshCore

LocalMesh is a community initiative, originating in the Netherlands and the United Kingdom, that deploys volunteer-operated LoRa repeaters to provide free, subscription-less emergency messaging when cellular and Internet infrastructure fail [Loc26]. Technically, LocalMesh networks run MeshCore, an open-source mesh protocol released in 2025 as a direct response to the scalability limits of flood-based systems [Mes25].

MeshCore abandons the egalitarian model in favour of explicit device roles. Ordinary user

devices (*companions*) never forward traffic; forwarding is reserved for dedicated *repeaters*, intentionally placed at elevated locations, while *room servers* provide store-and-forward group messaging. Routing is hybrid: the first message toward an unknown destination is flooded through the repeater backbone for path discovery; the acknowledgement returns the ordered sequence of repeaters traversed, and subsequent messages embed this source-routed path in their headers, so that only the designated next-hop repeater forwards each packet. If a cached path fails, the sender falls back to flooding to discover a fresh one [Mes25]. The result is a much quieter channel: after path discovery, most traffic is directed unicast rather than network-wide broadcast, which is why MeshCore networks scale comfortably beyond the practical limits of Meshtastic.

These gains, however, are bought with assumptions that cannot be made in this project. MeshCore's efficiency depends on a *planned, static backbone* of well-placed, always-on repeaters—volunteer infrastructure that takes months to build and that constitutes, in aggregate, a dependency not unlike the one resilient networks try to escape. Cached source routes also presume topological stability: under mobility or node loss, the protocol repeatedly falls back to flooding, eroding its advantage exactly when conditions are most adverse. A3MESH targets the complementary scenario: spontaneous, fully ad-hoc deployments of homogeneous, possibly mobile nodes, with no pre-positioned infrastructure and no role assignment. The research question is therefore whether the channel-load discipline that MeshCore achieves through *structure* can instead be achieved through *statistics*—that is, through gossip-based forwarding.

2.5. Gossip-Based Routing Protocols

2.5.1. The Flooding Problem

Flooding is the simplest possible dissemination primitive: every node retransmits every new packet exactly once. Its appeal in ad-hoc networks is that it requires no routes, no neighbour tables and no coordination, and it finds every reachable node by construction. Its cost was characterized in the seminal analysis of the *broadcast storm problem* by Ni et al., who showed that naive rebroadcasting produces severe redundancy, contention, and collisions: in dense topologies, each additional rebroadcast covers only a marginal sliver of new area while every transmission competes for the shared channel [Ni+99]. The problem is dramatically amplified over LoRa, where frames occupy the channel for hundreds of milliseconds at high spreading factors, transceivers are half-duplex, carrier sensing is unreliable over kilometre-scale links (the hidden-node problem), and regulatory duty cycles cap each node's airtime budget [Ade+17; Bor+16]. A LoRa mesh that floods therefore saturates at modest node counts, as the Meshtastic experience illustrates.

2.5.2. Probabilistic Forwarding

Gossip protocols attack this redundancy at its root by randomizing the forwarding decision: upon first reception of a packet, a node rebroadcasts it with probability p and stays silent with probability $1 - p$. The foundational result of Haas, Halpern, and Li is that this simple mechanism exhibits a *bistable* (percolation-like) behavior: below a critical probability the dissemination dies out quickly, while above it the message reaches almost all nodes almost as reliably as full flooding. In typical random topologies the threshold lies around $p \approx 0.6$–

0.8, meaning that 25–35% of transmissions can simply be removed with negligible loss of coverage [HHL06]. For LoRa meshes this is a directly monetizable saving: fewer transmissions translate into proportionally less airtime, fewer collisions, lower energy consumption, and slower duty-cycle exhaustion.

2.5.3. Controlled Gossip

Pure probabilistic forwarding has a known weakness: dissemination can die prematurely in sparse regions, where a node may be the *only* bridge between two parts of the network yet still decline to forward. A family of *controlled* gossip variants addresses this by combining the coin flip with deterministic safeguards [HHL06; WC02]. Representative mechanisms include forwarding with probability 1 for the first k hops to give the message critical mass (GOSSIP1(p, k)); forcing retransmission when a node has fewer neighbours than a threshold, so that articulation points never stay silent; and counter-based recovery, in which a node that has not overheard sufficient copies of a packet within a timeout concludes that the gossip is dying in its vicinity and retransmits regardless of its earlier decision. These hybrids retain most of the savings of pure gossip while restoring flooding-like reliability in irregular topologies—an essential property for A3MESH, whose deployments are expected to mix dense clusters with sparse long-range bridges.

2.5.4. Adaptive Gossip and the Research Gap

The protocols above still share a limitation: p and its companion thresholds are fixed at design time, while the optimal operating point depends on local density, traffic load and link quality, all of which vary across the network and over time. *Adaptive* gossip closes this loop by letting each node tune its forwarding probability from locally observable signals. Kyasanur et al.'s Smart Gossip, for instance, infers parent–child dependencies from overheard traffic and assigns each node a forwarding probability proportional to how much its neighbours actually depend on it, converging automatically to high probabilities in sparse regions and low ones in dense regions, with no manual configuration [KCG06]. Related adaptive schemes modulate p using neighbour counts, overheard duplicate counters or received signal strength [WC02].

This body of work, however, was developed and evaluated almost exclusively for IEEE 802.11 ad-hoc networks and classical wireless sensor networks—radios with millisecond airtimes, effective carrier sensing and no duty-cycle constraints. Its translation to LoRa is far from mechanical: long airtimes magnify the cost of every redundant transmission, half-duplex operation means a forwarding node is deaf while it helps others, SNR is cheaply observable and unusually informative thanks to CSS, and the duty cycle imposes a hard budget. Meanwhile, the deployed LoRa mesh systems reviewed in Section 2.4 occupy the two extremes of the spectrum—managed flooding without probabilistic suppression (Meshtastic) and structured routing requiring planned infrastructure (MeshCore)—leaving the middle ground essentially unexplored. This is the research gap addressed by this thesis: A3MESH brings *smart, adaptive gossip* to a fully decentralized LoRa mesh, using locally available information to modulate each node's forwarding probability, with the aim of approaching the channel efficiency of structured routing while preserving the zero-infrastructure, zero-configuration resilience of flooding.

3. Main Content

3.1. System Architecture

3.1.1. General Architecture

The architecture of the platform comprises four principal components: the mobile application, the BLE communication interface, the ESP32-based gateway node, and the LoRa mesh network as illustrated in Figure 3.1.

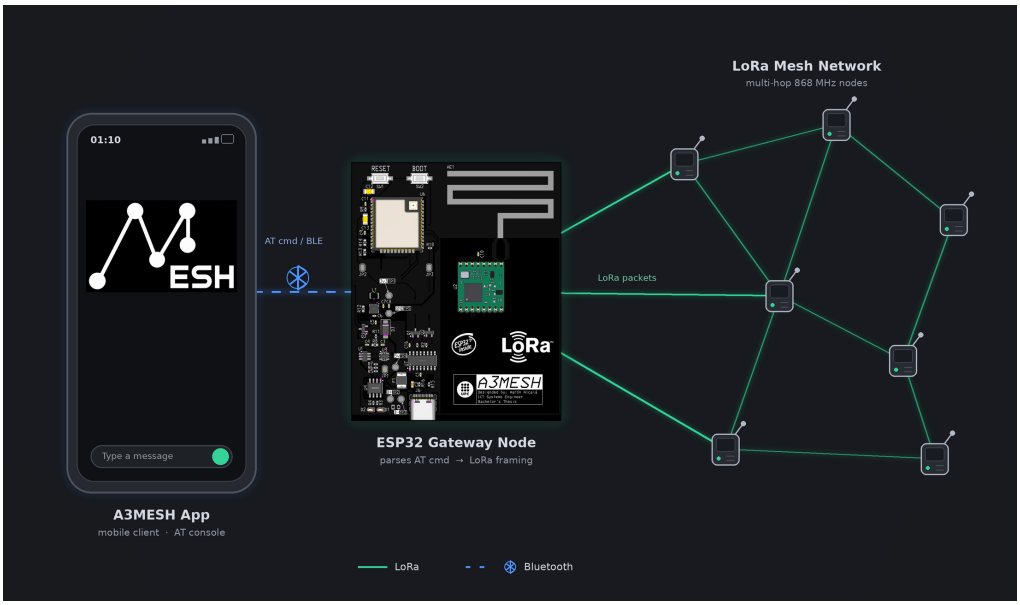


Figure 3.1.: High-level architecture of the A3MESH platform, showing the four main components and their interactions.

From the end user's perspective, the mobile application constitutes a user-friendly front end designed to mediate all interaction with the mesh network. Through it, users can send and receive messages, manage contacts, and configure network parameters as advanced settings. This component provides the accessibility and usability layer of the system, enabling users without any technical background to participate in the mesh network in a simple and intuitive manner.

Since smartphones lack native LoRa radio capabilities, an intermediary channel between the handset and the mesh network is required. This role is fulfilled by BLE.

To structure this communication, an AT-command-based interface has been implemented, through which the mobile application issues instructions to the node firmware. The ESP32 microcontroller, which provides native BLE support, is responsible for parsing these commands and translating them into concrete actions, either within the mesh network or on the node

itself, including packet processing, routing logic, encryption mechanisms, and the management of the LoRa radio.

Finally, the outermost layer of the architecture is the LoRa mesh network, which provides long-range wireless communication between nodes. This network constitutes the backbone of the system, allowing nodes to communicate with one another.

3.1.2. Node Interaction Model

The mesh layer is built on a deliberately egalitarian premise: all nodes are functionally identical peers. There are no coordinators, no dedicated repeaters, and no infrastructure roles of any kind; every node is simultaneously an endpoint for its own user and a potential relay for the traffic of others. This design choice follows directly from the resilience objectives of the system and stands in explicit contrast to the role-based architecture of MeshCore discussed in Section 2.4.2.

Neighbor awareness. Each node maintains a local notion of its *neighborhood*: the set of nodes whose transmissions it can hear directly. Crucially, this awareness is acquired *passively*. Since every LoRa transmission is inherently a broadcast, a node learns of the existence, identity, and link quality of its neighbors simply by overhearing the traffic that already flows through the channel.

Message propagation. Communication beyond direct radio range is achieved through multi-hop forwarding. The logical level distinguishes between a direct message and a network-wide announcement, encoded in the packet's addressing fields. With regard to the latter, a node receiving a packet that is not addressed to it does not discard it, but evaluates whether to act as a relay.

Forwarding decision. The defining characteristic of A3MESH's interaction model is that this relaying decision is *probabilistic and adaptive* rather than deterministic. Instead of rebroadcasting every new packet unconditionally—the flooding approach whose pathologies were analyzed in Section 2.5—each node forwards with a probability modulated by locally observable signals, such as the quality of the link over which the packet arrived and the level of redundancy it observes in its vicinity. Nodes in dense regions, where many candidate relays exist, suppress most of their retransmissions, whereas nodes in sparse regions, where they may constitute the only bridge between network segments, forward with near certainty. As a result, the network as a whole self-regulates its channel occupancy without any global coordination. The design of this forwarding policy, its mathematical formulation, and its optimization constitute the central contribution of this thesis and are developed in Chapter 3.3, together with its realization in Chapter 3.5.

3.1.3. Communication Flow

This section traces the complete lifecycle of a user message through the system, from the sender's smartphone to the recipient's, tying together the four architectural components introduced in Section 3.1.1. Figure 3.2 presents the corresponding sequence diagram.

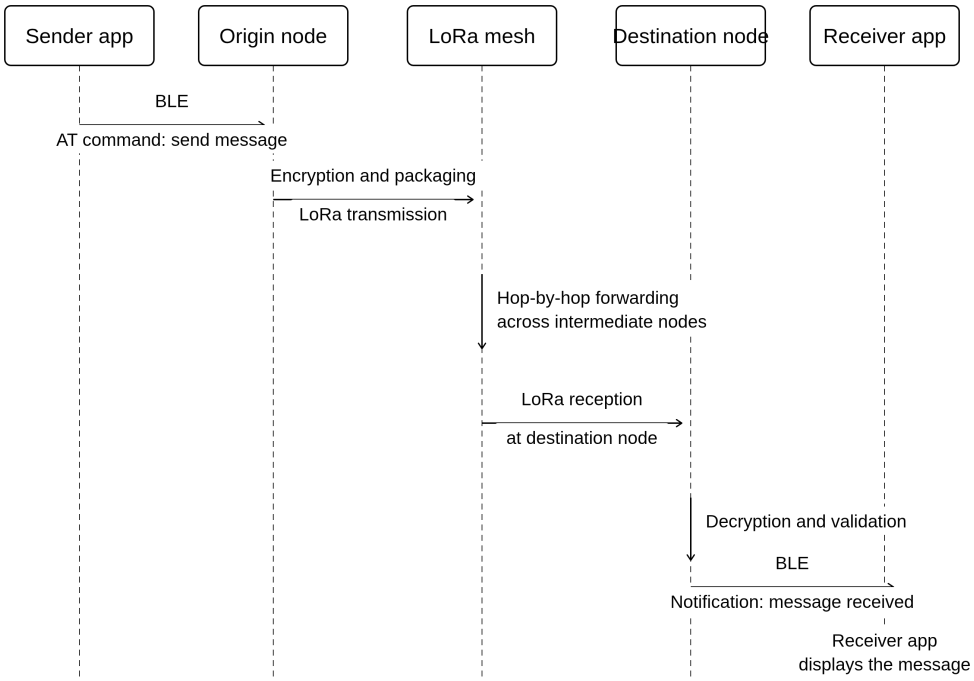


Figure 3.2.: End-to-end sequence diagram of a message exchanged between two users, traversing the BLE link, the originating node, the LoRa mesh, the destination node, and the final BLE link to the recipient's smartphone.

Origination. The lifecycle begins in the mobile application, where the user composes a message addressed to a contact. The application encodes the request as an AT command and writes it to the corresponding GATT characteristic of its paired node over BLE. From the user's perspective the interaction ends here: the experience is intentionally indistinguishable from that of a conventional messaging application.

Ingestion at the originating node. The node's firmware parses the AT command, validates it, and constructs a mesh packet: the payload is encrypted, the addressing fields, sequence number, and time-to-live (TTL) counter are populated, and the resulting frame is serialized for transmission. The node then transmits the packet over the LoRa radio as a broadcast, subject to the channel-access and duty-cycle constraints enforced by the firmware.

Mesh traversal. Every node within radio range receives the frame. Each receiver first checks the packet against its duplicate cache; previously seen packets are silently discarded, which guarantees that no packet is relayed more than once by the same node. For a new packet, the node performs two independent actions. First, it checks whether the packet is addressed to itself, in which case it is passed up to the local delivery path. Second—and regardless of addressing—it evaluates the adaptive forwarding decision described in Section 3.1.2: with the computed probability, it schedules a rebroadcast after a randomized back-off that desynchro-

nizes concurrent relays; otherwise it remains silent. Each rebroadcast decrements the TTL, which bounds the propagation horizon and prevents indefinite circulation. The packet thus expands through the network in stochastic waves, hop by hop, until it reaches the destination, the TTL expires, or the gossip extinguishes naturally once coverage is achieved.

Delivery. When the packet reaches a node whose identity matches the destination address, the firmware decrypts and validates the payload, stores the message, and notifies the paired smartphone through a BLE notification on the corresponding GATT characteristic. The recipient's application then decodes the notification and renders the message in the conversation view.

It must be emphasized that this description captures the system-level journey of a message; the internal mechanics of each stage—the queueing and task structure of the BLE-to-LoRa pipelines, the channel-access procedure, and the encryption scheme—are implementation concerns addressed in Chapter 3.5.

3.1.4. Design Requirements

The architecture described above is shaped by four system-level requirements. They are stated here together with their rationale and target figures; the subsequent chapters derive from them the concrete engineering requirements of each subsystem (Chapter 3.4 for the hardware platform, Chapter 3.5 for the firmware, and Chapter 3.6 for the mobile application), and Chapter 3.7 evaluates the extent to which they are met.

Low Power Consumption

A3MESH nodes are battery-powered devices intended for scenarios—emergencies, remote deployments, prolonged infrastructure outages—in which recharging opportunities cannot be assumed. Energy is therefore the most binding resource of the entire design, and the requirement is twofold. At the device level, a node must sustain multiple days of autonomous operation on a single battery charge, which demands aggressive duty cycling of both radios, exploitation of the ESP32 sleep modes, and a power subsystem with high conversion efficiency. At the network level, the forwarding protocol itself must be energy-aware: every retransmission suppressed by the gossip policy is energy saved not by one node but by the network as a whole. This dual perspective—energy as a hardware constraint and as a protocol design criterion—recurs throughout this work.

Scalability

The system must remain functional as node density grows, precisely the regime in which flood-based LoRa meshes degrade. The requirement is therefore formulated in terms of *graceful degradation*: channel load must grow with offered traffic rather than with the number of deployed nodes, and the addition of nodes to a region should improve, not erode, its connectivity. This requirement is the principal motivation for abandoning deterministic flooding in favor of the adaptive gossip policy, and it is evaluated quantitatively through the congestion metrics of Chapter 3.3.

Reliability

A messaging system intended for emergency use must deliver messages with high probability under adverse conditions: lossy links, node failures, mobility, and network partitions. Probabilistic forwarding introduces an inherent tension here, since every suppressed retransmission is also a lost redundancy opportunity. The requirement is therefore expressed as a target operating point: the forwarding policy must achieve a packet delivery ratio statistically indistinguishable from that of pure flooding—in particular in sparse topologies, where premature gossip die-out is the dominant failure mode—while consuming a substantially smaller airtime budget. Reliability is thus not pursued through end-to-end acknowledgements and retransmissions, which would multiply airtime consumption, but through the controlled redundancy that the gossip mechanism itself provides. The delivery-ratio metric defined in Chapter 3.3 quantifies compliance with this requirement.

User Accessibility

Finally, the system must be operable by users with no technical background whatsoever. In the intended scenarios, the people who most need resilient communication are not radio amateurs or engineers, and any configuration step—flashing firmware, selecting spreading factors, managing identifiers—constitutes a barrier that excludes them. This requirement drives the upper layers of the architecture: the smartphone, a device users already own and understand, is the sole human interface; the BLE link and pairing process require no networking knowledge; the mesh is self-configuring, with discovery, forwarding, and adaptation occurring autonomously; and the application deliberately reproduces the interaction patterns of mainstream messaging applications. Accessibility also constrains the physical design, as reflected in the ergonomic and enclosure decisions of Section 3.4.9.

3.2. Development Methodology and Project Management

3.2.1. Development Workflow

Development was structured around three interlocking tools: Scrum sprints for iteration cadence and supervisor alignment, Jira for task tracking and sprint planning, and Git for version control with branch names tied directly to Jira ticket identifiers.

3.2.2. Agile Methodology

A lightweight variant of the Scrum framework [SS20] was adopted for the duration of the project. Sprints of two weeks each provided the iteration unit. At the opening of each sprint, a planning meeting with the project supervisor was held to review the work completed in the preceding iteration, identify any deviations from the original plan, and select the tasks to be addressed in the forthcoming two weeks.

The two-week cadence balanced feedback frequency against iteration overhead: shorter cycles would have imposed disproportionate planning cost given tasks such as PCB fabrication, while longer cycles risked accumulating divergence from the supervisor's expectations before a correction could be applied. When a sprint's scope proved larger than anticipated, the subsequent sprint was adjusted to compensate rather than deferring indefinitely.

3.2.3. Sprint Planning with Jira

Each sprint's work items were recorded in a Jira project board maintained throughout the development period. Every discrete deliverable—a subsystem design, a firmware module, a simulation experiment, and a thesis chapter—was represented as a Jira issue assigned a unique ticket identifier of the form `TFG-`N, where N is a monotonically increasing integer.

Tickets were grouped under six epics—*Documentation*, *Debugging and Testing*, *FW Architecture*, *HW Architecture*, *System Architecture*, and *Mobile SW*—one per major subsystem, allowing progress to be assessed at a subsystem granularity during planning meetings.

This ticket-centric representation served two purposes beyond simple task tracking. First, it made the scope of each sprint explicit and negotiable at the planning meeting, preventing scope creep from being introduced informally between meetings. Second, and more importantly for day-to-day engineering, the ticket identifier was embedded directly in the version-control branch name, as described in the following section, creating a direct and auditable link between a Jira ticket and all the commits that contributed to resolving it.

Figure 3.3 shows a snapshot of the Jira timeline view, illustrating the distribution of hours spent in each epic over the course of the project.

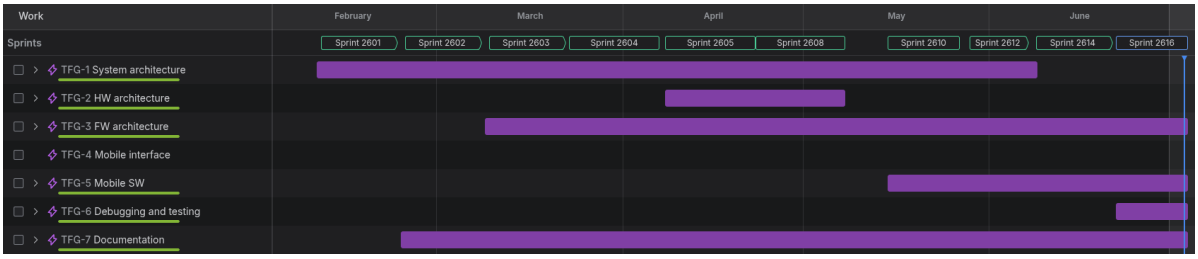


Figure 3.3.: Jira timeline view of the project, showing the distribution of hours spent in each epic over the course of the project.

3.2.4. Version Control with Git

Git [CS14] was used as the version control system, structured around a central invariant: the `main` branch must always represent a compilable, functional state of the project. To preserve this, no commits were made directly to `main`; instead, all development was carried out on dedicated feature branches, named following the convention `feature_TFG-`N`_`*summary*, where N is the Jira ticket number and *summary* is a concise, PascalCase description of the task. Including the ticket number in the branch name allows any line of the `git log` to be traced back to its originating task without consulting external documentation. The integration workflow follows a deliberate sequence: a feature branch is created from the current tip of `main`, all development commits are made on that branch, and before integrating the work, the latest state of `main` is pulled into the feature branch so that any conflicts are resolved there, ensuring `main` itself is never the site of conflict resolution.

Figure 3.4 illustrates this sequence as a branch diagram, showing the moment at which `main` is pulled into the feature branch before the final merge, thereby ensuring compatibility with any concurrent work that has already been integrated.

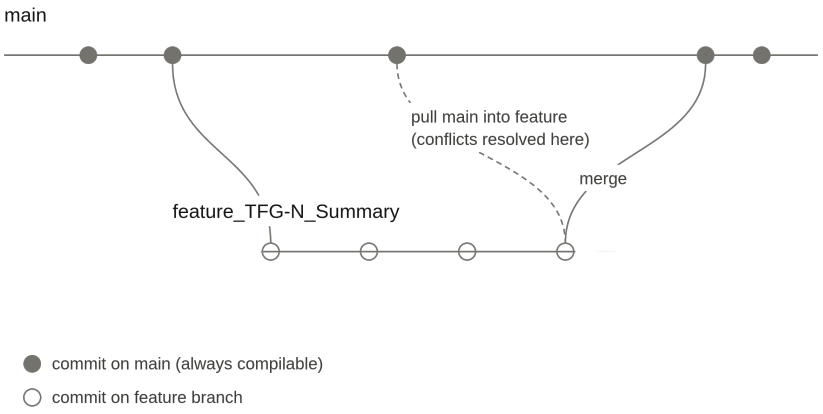


Figure 3.4.: Git branching workflow.

3.2.5. Repository Organisation

The entire project is maintained within a single repository whose directory structure enforces the orthogonality of its constituent subsystems. The root of the repository contains the LaTeX source of this thesis alongside five subdirectories, each of which corresponds to one self-contained technical component, as detailed in Appendix D.

This monorepo organisation was chosen deliberately so that a complete snapshot of the project at any point in history—including the thesis text, the firmware, the hardware design, and the simulation code—is recoverable from a single commit hash. The orthogonality of the subdirectories ensures that changes to one subsystem do not generate noise in the diffs of another.

3.3. Network Simulation and Routing Algorithm Design

3.3.1. Purpose of the Simulator

A network simulator was necessary for three complementary reasons. First, the adaptive forwarding function developed here is governed by five continuous parameters, each evaluation of which requires observing the network for a statistically meaningful period; a coarse exploration on real hardware would take thousands of multi-hour field deployments, whereas the same exploration in simulation completes in minutes. Second, the phenomena of interest—broadcast storms, channel congestion, gossip die-out—only emerge at network sizes of tens of nodes spread over kilometres, far beyond the prototype fleet that can be manufactured and deployed within a bachelor thesis. Third, a discrete-event simulator provides perfectly repeatable conditions in which two algorithms can be compared on *identical* topologies, traffic patterns, and channel realizations, something physical experiments—subject to interference, weather, and placement variability—cannot achieve.

3.3.2. Simulation Environment

Simulator Architecture

The experimental platform is built upon *Meshtasticator*, the open-source discrete-event simulator of the Meshtastic project [Mes24c]. Meshtasticator models a LoRa mesh as a set of node processes coordinated by the SimPy discrete-event engine [Tea24]: nodes exchange packets through a broadcast pipe that emulates the shared radio medium, and every transmission is propagated to all other nodes, where it may be received, lost due to insufficient signal strength, or destroyed by collision. The simulator implements the relevant PHY and MAC behavior of real Meshtastic devices as well as LoRa time-on-air computation, capture-effect collision modeling, listen-before-talk channel access with random back-off, and the official *managed flood* routing algorithm of the Meshtastic firmware, which provides a faithful baseline for comparison.

On top of that, the simulator was extended for this thesis in two main directions. First, the routing layer was refactored to support pluggable forwarding policies, and the candidate strategies described in Section 3.3.3 were implemented as selectable router types, culminating in the Smart Gossip router. Second, the configuration system was instrumented so that the parameters of the Smart Gossip decision function are exposed as runtime variables, a prerequisite for the automated optimization process of Section 3.3.5. Finally, a batch-execution harness and an optimization driver were developed on top of the simulator to run large experiment campaigns and collect the metrics defined in Section 3.3.6.

Node, Packet and Channel Models

Each simulated node generates application messages, maintains a duplicate cache, executes the configured forwarding policy, and respects the half-duplex constraint of the radio. Nodes are placed uniformly at random over a 15×15 km area, subject to a minimum-separation constraint, and approximately 30% of the nodes are mobile, exercising the protocols under topological change.

Apart from this, the radio configuration corresponds to the `LONG_FAST` preset on the EU 868 band: SF11, 250 kHz bandwidth and coding rate 4/5, yielding a time-on-air of approximately 550 ms per packet; transmit power is 27 dBm and receiver sensitivity −131.5 dBm. Path loss follows the 3GPP Suburban Macro model (Model 5), which produces realistic link ranges of roughly 1.8 km in this configuration. Two refinements increase realism: log-normal shadowing and per-link asymmetric gain offsets (normally distributed, $\sigma = 3$ dB), so that link quality differs between directions, as observed in real LoRa deployments [CBR17]. The hop limit is fixed at 3, matching the Meshtastic default.

Randomness and Reproducibility

Every stochastic element of the simulation—node placement, traffic generation, link asymmetries, mobility, and the gossip coin flips themselves—derives from explicitly seeded pseudo-random generators. Each repetition of a scenario uses a distinct, documented seed (0–29), and node positions are pre-generated and cached per seed, so that all routing algorithms are evaluated on *exactly* the same sequence of topologies. What is more, this pairing eliminates topology variance from inter-algorithm comparisons and makes every result in this chapter exactly reproducible from the configuration files listed in Appendix D.

3.3.3. Evaluated Routing Strategies

The design of the final algorithm proceeded incrementally: a sequence of forwarding strategies of increasing sophistication was implemented and evaluated, each addressing a weakness identified in its predecessor. This section describes the progression conceptually; quantitative results are deferred to Section 3.3.7.

In addition to this, in the `images/` folder of the repository, a set of several graphs is provided, which illustrate the performance of each routing strategy mentioned in this section compared among them in different combinations.

Pure Flooding

Pure flooding is the natural baseline: every node retransmits every packet it has not seen before, until the hop limit is exhausted. Its virtues are simplicity and maximal coverage—if any path exists within the hop horizon, flooding finds it. Its pathology, anticipated analytically in Section 2.5, was confirmed immediately in simulation: the number of transmissions grows linearly with node count regardless of traffic demand, and with LoRa airtimes of half a second per packet the channel saturates rapidly, manifesting as collisions, queueing delays, and duty-cycle exhaustion. Meshtastic's managed flood, which adds SNR-weighted contention windows and rebroadcast cancellation, mitigates the worst simultaneity but, as the results of Section 3.3.7 show, does not alter the fundamental scaling behavior. Managed flood, being the algorithm deployed in the most widely used comparable system, serves as the reference baseline throughout this chapter.

Probabilistic Gossip

The first refinement replaces the deterministic rebroadcast with a Bernoulli trial: each node forwards a new packet with fixed probability p and stays silent otherwise, following the classical gossip scheme of Haas et al. [HHL06]. After simulation, several values of p were tested, and the scheme's weakness is equally clear: no single value of p suits all regions of a heterogeneous network. Therefore, a probability low enough to relieve congestion in dense clusters causes premature die-out in sparse areas, where a silent node may be the only bridge between partitions; a probability high enough to protect sparse areas forfeits most of the savings in dense ones. Consequently, the optimal p is, in short, a *local* quantity—which motivates making the decision adaptive.

This routing logic is defined as *GOSSIP0N* in the simulator, where N value is represented between 2 and 8, related to the values of p tested.

RSSI-Aware Gossip

The second refinement modulates the forwarding probability with the received signal strength of the incoming packet. The intuition exploits a geometric fact of broadcast propagation: a packet received weakly arrives from far away, meaning the receiver lies near the edge of the transmitter's coverage area and is precisely the kind of node that extends the packet's reach furthest if it relays; conversely, a packet received strongly comes from a nearby transmitter whose coverage area largely overlaps the receiver's, making a relay largely redundant. RSSI thus acts as a zero-cost proxy for geographic progress, an idea well established in WSN and LPWAN routing [SYA11; CBR17]. This strategy concentrates retransmissions at the periphery

of each broadcast wave, improving the usefulness of each transmission, but it remains blind to how many *other* candidates surround the receiver.

This routing logic is defined as *RSSI_BASED* in the simulator.

Density-Aware Gossip

The third refinement addresses exactly that blindness: the forwarding probability is made inversely dependent on the number of neighbors a node currently observes, following the neighbor-knowledge principle of Tseng et al. [Tse+02]. A node surrounded by many peers can afford to stay silent, since the expected number of other relays is high; a node with few neighbors must forward, since it may be an articulation point of the topology. This mechanism attacks the broadcast storm at its root—it decouples the aggregate retransmission count from node density—and proved in simulation to be the single most effective congestion-mitigation ingredient. In isolation, however, it ignores both link quality and the redundancy already observed on the channel.

Each of the three signals—link quality, local density, observed redundancy—thus captures a complementary aspect of the forwarding decision. The natural conclusion is to combine them, which leads to the algorithm at the core of this thesis.

This routing logic is defined as *ADAPTIVE_GOSSIP* in the simulator.

3.3.4. Smart Gossip Algorithm

Decision Function

Smart Gossip computes, for every newly received packet, a retransmission probability that combines the three signals above as independent multiplicative factors, bounded from below by a survival floor:

$$p \;=\; \max(p_{\text{floor}},\;\; f_{\text{rssi}} \cdot f_{\text{density}} \cdot f_{\text{dupe}}) \tag{3.1}$$

with

$$f_{\text{rssi}} = \frac{1}{1 + e^{\,\alpha\,(\text{RSSI}-\text{RSSI}_{\text{ref}})}} \qquad \text{(logistic sigmoid)} \tag{3.2}$$

$$f_{\text{density}} = \frac{1}{1 + \beta\, n} \qquad \text{(rational decay, } n = \text{current neighbor count)} \tag{3.3}$$

$$f_{\text{dupe}} = e^{-\gamma\,(c-1)} \qquad \text{(exponential decay, } c = \text{copies of the packet received)} \tag{3.4}$$

The node then draws a uniform random number and retransmits if it falls below p, after the randomized back-off described in Section 3.1.3. All inputs to Equation (3.1) are locally observable at zero communication cost: RSSI is reported by the transceiver with every reception, the neighbor count is maintained by passive overhearing, and the copy counter is a by-product of the duplicate cache. The algorithm therefore adds no control traffic whatsoever.

Rationale of Each Factor

The rssi factor. Equation (3.2) encodes the edge-of-coverage heuristic introduced above: it approaches 1 as the RSSI approaches the sensitivity limit (−131.5 dBm), where the receiver is likely one of very few nodes to have heard the packet, and approaches 0 for strong signals from nearby transmitters whose coverage the receiver cannot meaningfully extend. A sigmoid is preferred over a hard threshold because RSSI in a shadowed channel is a noisy quantity; a continuous transition prevents abrupt behavioral changes from decibel-level fluctuations, and its shape is consistent with the coverage probability induced by log-normal fading [SYA11; CBR17]. The parameter α controls the steepness of the transition and $\mathrm{RSSI}_{\mathrm{ref}}$ its center.

The density factor. Equation (3.3) is the direct counter-measure to the broadcast storm [Ni+99; Tse+02]: it scales the individual forwarding probability inversely with the local neighbor count n, so that the *expected number of relays* in a neighborhood—roughly $n \cdot f_{\mathrm{density}}$, which approaches the constant $1/\beta$ for large n—remains bounded as density grows, instead of growing linearly as under flooding. The rational form $1/(1+\beta n)$ is the simplest function with value 1 at $n = 0$, smooth decay and a zero asymptote.

The duplicate factor. Equation (3.4) is a continuous reformulation of the counter-based scheme of Tseng et al. [Tse+02]: every copy of a packet a node overhears is direct evidence that the packet is already well disseminated in its vicinity, so the incentive to add yet another copy decays exponentially. Unlike the original binary threshold, the exponential form degrades gracefully, halving the probability roughly every two additional copies (for the optimized γ).

The probability floor. Finally, p_{floor} guarantees that no combination of inputs ever silences a node completely. It is the safeguard against gossip die-out identified in Section 2.5: in pathological cases—a sparse bridge node that happens to receive a strong signal, for instance—the multiplicative factors could otherwise drive p arbitrarily close to zero and sever the network. The floor also provides a single, intuitive operational knob: raising p_{floor} toward 1 continuously morphs Smart Gossip back into flooding, a property exploited in Section 3.3.8.

Behavior in Representative Scenarios

Table 3.1 evaluates Equation (3.1) with the optimized parameters of Section 3.3.5 in five representative situations, illustrating how the factors interact. A node at the coverage edge with few neighbors forwards with high probability (0.70); an average node halves that; observed duplicates suppress it further; and well-connected nodes near the transmitter fall to the floor, retransmitting only 11% of the time.

3.3.5. Evolutionary Optimization Process

Why Manual Tuning Is Insufficient

Equation (3.1) introduces five free parameters: $\theta = (\alpha, \mathrm{RSSI}_{\mathrm{ref}}, \beta, \gamma, p_{\mathrm{floor}})$. Because their effects are strongly coupled—a steeper sigmoid changes the operating point at which the density factor matters, the duplicate penalty interacts with the floor—tuning them one at a time is methodologically unsound. Also, exhaustive search fares no better: a modest grid of ten values per dimension yields 10^5 combinations, each requiring multiple full network simulations to

Table 3.1.: Smart Gossip forwarding probability in representative scenarios (optimized parameters).

Scenario	RSSI (dBm)	n	c	f_{rssi}	$f_{\text{density}} \cdot f_{\text{dupe}}$	p
Coverage edge, sparse	−130	2	1	0.91	$0.77 \cdot 1.00$	**0.70**
Mid-range, average density	−115	5	1	0.73	$0.58 \cdot 1.00$	**0.42**
Mid-range, redundancy seen	−115	5	3	0.73	$0.58 \cdot 0.54$	**0.23**
Near transmitter, dense	−95	10	2	0.43	$0.41 \cdot 0.73$	**0.11**†
Adjacent transmitter, dense	−80	15	4	0.11	$0.31 \cdot 0.39$	**0.11**†

†clamped to p_{floor}

average out stochastic noise. Finally, the objective surface is non-convex and *noisy*, since every evaluation is itself a random experiment, which rules out gradient-based methods. The problem calls for a derivative-free global optimizer that is sample-efficient in moderate dimensions.

Differential Evolution

Differential evolution (DE) [SP97; PSL05] satisfies these requirements and was adopted through its `scipy.optimize` implementation [Vir+20]. DE maintains a population of candidate parameter vectors and evolves it through three operators. *Mutation* creates, for each individual $\mathbf{x}_i$, a mutant vector from three distinct randomly chosen members of the population,

$$\mathbf{v}_i = \mathbf{x}_a + F \cdot (\mathbf{x}_b - \mathbf{x}_c), \tag{3.5}$$

where the differential weight F was dithered in $[0.5, 1.0]$ to balance exploration and convergence [PSL05]. *Crossover* then mixes $\mathbf{v}_i$ with $\mathbf{x}_i$ coordinate-wise with recombination probability 0.7, producing a trial vector. *Selection* is greedy: the trial replaces $\mathbf{x}_i$ in the next generation only if it achieves a better fitness. Because mutation step sizes derive from the population's own spread, DE self-adapts its search scale, behaving exploratively while the population is dispersed and refining once it concentrates around an optimum—a property well suited to noisy, non-convex landscapes [DS11].

Fitness Function and Configuration

Each candidate θ is evaluated by running full network simulations at sizes of 15 and 30 nodes—the regime where the routing decision matters most—over multiple seeded repetitions, and aggregating four metrics into the scalar fitness

$$\text{fitness}(\theta) = 0.40\,\overline{\text{reach}} + 0.40\,\overline{\text{useful}} - 0.10\,\overline{\text{col}} - 0.10\,\overline{\text{tx}}, \tag{3.6}$$

where $\overline{\text{reach}}$ is the mean reachability, $\overline{\text{useful}}$ the mean usefulness, $\overline{\text{col}}$ the mean collision rate (all in %) and $\overline{\text{tx}}$ the mean per-node transmission airtime normalised to a comparable scale. The weights encode the design intent: coverage and channel efficiency are rewarded equally and dominantly, while collisions and raw airtime act as soft penalties. The search ranges of the five parameters (Table 3.8) were chosen generously around physically meaningful values. The optimization converged after approximately 160 fitness evaluations, completing in under ten minutes of computation—itself a demonstration of why the exploration had to be conducted in simulation.

Limitations of the Optimization

Three limitations of this process must be acknowledged. First, to keep the campaign tractable, each fitness evaluation averaged over a small number of repetitions, so the optimum is estimated under residual noise; a larger budget would yield a more robust optimum. Second, fitness was evaluated only at 15 and 30 nodes; including the extremes (5 and 50) might have shifted the optimum toward higher coverage in small networks. Third, Equation (3.6) does not impose a hard reachability constraint, which explains the coverage deficit observed in the smallest networks in Section 3.3.7. These limitations delimit the validity of the chosen parameters and directly motivate the future-work directions of Chapter 5.

3.3.6. Performance Metrics

Four metrics, computed identically for every algorithm and scenario, quantify the quality of a forwarding policy. Together they capture the two sides of the fundamental trade-off—how much of the network is reached, and at what cost in channel resources.

Reachability

Reachability is the fraction of origin–destination pairs for which a generated message was correctly delivered, expressed as the number of useful receptions over $M\,(N-1)$, where M is the number of generated messages and N the network size. It is the primary effectiveness metric: it answers whether the network performs its function. Absolute values must be interpreted against the topology; in the sparse 15×15 km scenarios simulated here, some node pairs lack physical connectivity altogether, bounding reachability below 100% for *any* algorithm.

Average Delay

Delay is measured per delivered message as the time elapsed between generation at the source application and reception at the destination, and averaged over all deliveries. Beyond user experience, delay is a sensitive *congestion indicator*: in a listen-before-talk MAC, channel saturation manifests as packets waiting tens of seconds for clear air, so a diverging delay curve reveals a collapsing channel even when delivery eventually succeeds. Note that the metric is conditioned on delivery; messages never delivered do not contribute.

Transmission Air Utilization

Air utilization is the total time each node's radio spends transmitting over the simulation, averaged per node. It is the direct cost metric of a forwarding policy and carries a double physical meaning: it is proportional to the radio's energy expenditure—linking the protocol to the power requirement of Section 3.1.4—and it measures consumption of the regulatory duty-cycle budget of the EU 868 band, the hard ceiling identified in Section 3.1.4.

Usefulness

Usefulness is the fraction of received packets that carried new information to their receiver, i.e. that were not duplicates already present in its cache. It is the *efficiency* metric of the channel: a low value means the medium is being filled with copies nobody needs. Usefulness isolates precisely the redundancy that gossip is designed to remove, making it the most discriminating

single indicator of forwarding intelligence. The collision rate—the fraction of sensed packets destroyed by concurrent transmissions—is additionally reported as a secondary congestion indicator.

3.3.7. Simulation Results and Analysis

The final evaluation campaign compared Smart Gossip, with the optimized parameters of Table 3.8, against Meshtastic's managed flood across four network sizes (5, 15, 30 and 50 nodes), with 30 seeded repetitions per algorithm and size—240 simulations of 30 minutes each. Both algorithms faced identical topologies, traffic, and channel realizations in every repetition. Tables 3.2–3.6 report means and standard deviations; Figures 3.5–3.9 present the same data graphically.

Table 3.2.: Reachability (%), mean ± standard deviation over 30 repetitions.

Nodes	Smart Gossip	Managed Flood	Δ abs.	Δ rel.
5	68.7 ± 19.0	76.8 ± 19.1	−8.1 pp	−10.6%
15	50.0 ± 11.1	62.5 ± 11.5	−12.4 pp	−19.9%
30	38.2 ± 7.3	47.1 ± 6.4	−8.9 pp	−18.9%
50	29.3 ± 4.1	33.8 ± 3.0	−4.5 pp	−13.4%

Table 3.3.: Collision rate (%), mean ± standard deviation over 30 repetitions.

Nodes	Smart Gossip	Managed Flood	Δ abs.	Δ rel.
5	4.0 ± 2.8	6.3 ± 3.9	−2.3 pp	−36.6%
15	11.5 ± 2.5	17.1 ± 4.4	−5.6 pp	−32.8%
30	16.3 ± 2.8	19.9 ± 2.2	−3.7 pp	−18.3%
50	20.6 ± 2.4	19.1 ± 2.3	+1.5 pp	+7.9%

Table 3.4.: Usefulness (%), mean ± standard deviation over 30 repetitions.

Nodes	Smart Gossip	Managed Flood	Δ abs.	Δ rel.
5	48.0 ± 3.1	47.9 ± 2.7	+0.1 pp	+0.2%
15	50.1 ± 2.5	45.8 ± 4.0	+4.3 pp	+9.5%
30	51.0 ± 2.7	45.4 ± 3.4	+5.6 pp	+12.3%
50	49.9 ± 2.3	45.2 ± 2.4	+4.8 pp	+10.6%

Channel Efficiency

The most robust result of the campaign is the reduction in air utilization: for networks of 15 nodes and above, Smart Gossip occupies 27–32% less channel time per node than managed flood, with small variance across 30 repetitions. The practical significance is twofold. In energy terms, transmission is the dominant consumer in a LoRa node, so this reduction translates

Table 3.5.: Per-node transmission air utilization (ms), mean ± standard deviation over 30 repetitions.

Nodes	Smart Gossip	Managed Flood	Δ abs.	Δ rel.
5	54 498 ± 5 628	61 246 ± 9 847	−6 748	−11.0%
15	67 862 ± 4 519	97 268 ± 10 525	−29 406	−30.2%
30	73 873 ± 4 574	109 104 ± 6 396	−35 232	−32.3%
50	79 802 ± 2 479	109 958 ± 7 701	−30 156	−27.4%

Table 3.6.: Average end-to-end delay (ms), mean ± standard deviation over 30 repetitions.

Nodes	Smart Gossip	Managed Flood	Δ abs.	Δ rel.
5	5 556 ± 867	4 318 ± 1 356	+1 238	+28.7%
15	6 766 ± 925	16 199 ± 8 123	−9 433	−58.2%
30	11 362 ± 5 623	78 592 ± 34 365	−67 230	−85.5%
50	27 112 ± 16 709	160 440 ± 50 484	−133 328	−83.1%

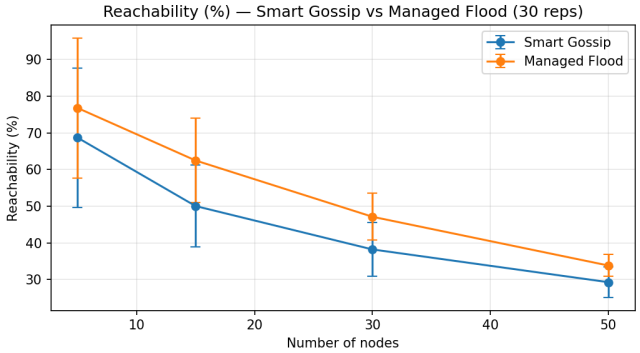


Figure 3.5.: Reachability (%) versus network size for Smart Gossip and Managed Flood, averaged over 30 repetitions.

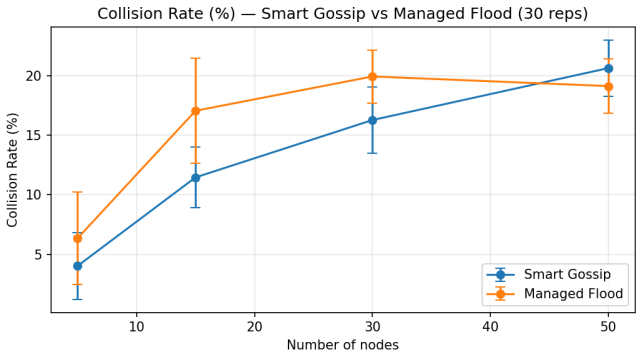


Figure 3.6.: Collision rate (%) versus network size for Smart Gossip and Managed Flood, averaged over 30 repetitions.

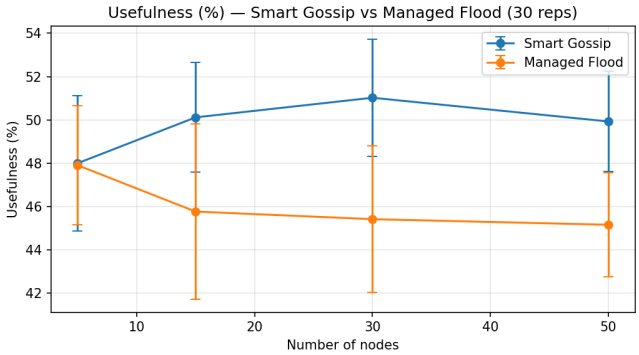


Figure 3.7.: Usefulness (%) versus network size for Smart Gossip and Managed Flood, averaged over 30 repetitions.

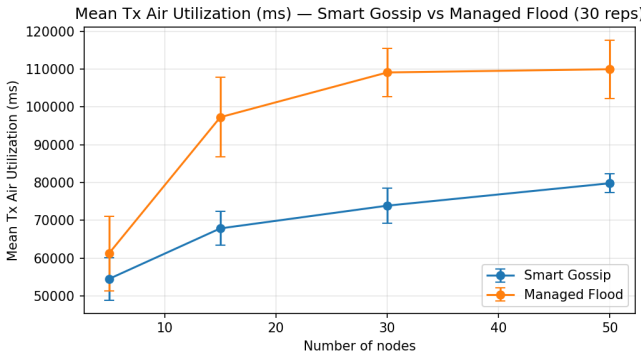


Figure 3.8.: Per-node transmission air utilization (ms) versus network size for Smart Gossip and Managed Flood, averaged over 30 repetitions.

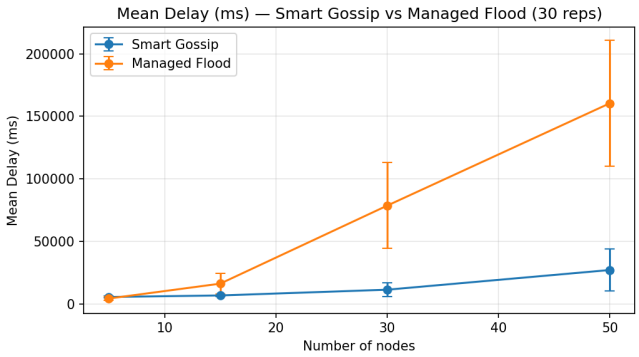


Figure 3.9.: Average end-to-end delay (ms) versus network size for Smart Gossip and Managed Flood, averaged over 30 repetitions.

directly into battery lifetime. In regulatory terms, with the `LONG_FAST` airtime of ∼550 ms per packet, managed flood at 15 nodes already consumes 5.4% of duty cycle over the 30-minute window against Smart Gossip's 3.8%; under heavier traffic or larger networks, flooding approaches the regulatory ceiling first, at which point nodes are legally forced to silence. The usefulness metric explains where the savings come from: under managed flood, fewer than half of all received packets carry new information, whereas Smart Gossip raises usefulness by a consistent ∼10% relative for all networks of 15 nodes and above—the same information is disseminated with substantially fewer transmissions.

Congestion Collapse under Flooding

The delay results expose the broadcast storm in its most dramatic form. At 30 nodes, managed flood exhibits a mean delay of 78.6 s—and 160 s at 50 nodes—against 11.4 s and 27.1 s for Smart Gossip, reductions of 85% and 83% respectively. The mechanism is the one anticipated in Section 2.5: when every node contends to retransmit, the listen-before-talk MAC forces packets to queue for tens of seconds awaiting clear channel, and collisions force further retries. Smart Gossip removes the bulk of this contention at the source by silencing redundant relays, so the packets that are forwarded traverse a comparatively idle channel. The collision-rate results corroborate the picture at 5–30 nodes (reductions of 18–37%); only at 50 nodes does managed flood show marginally fewer collisions (+1.5 pp for Smart Gossip), an artifact of its channel being so saturated that its contention windows serialize transmissions—at the cost of the 160-second delays just described.

The Coverage Trade-off and Failure Scenarios

The cost of these gains is a reachability deficit of 4.5–12.4 percentage points, largest at intermediate sizes. This is the inherent gossip trade-off anticipated theoretically [HHL06]: every suppressed retransmission is also a suppressed redundancy opportunity, and in a sparse topology some of those opportunities were load-bearing. Three observations contextualize the deficit. First, absolute reachability is bounded by physical connectivity in these sparse scenarios—both algorithms range from ∼30 to ∼77%—so part of the gap reflects marginal links that only brute-force redundancy can cross. Second, the deficit *shrinks* in the largest network (−4.5 pp at 50 nodes) while the efficiency advantages persist, indicating that the trade-off improves exactly where it matters. Third, the deficit is largest precisely in the configurations excluded from the optimization fitness, suggesting it is partly an artifact of the tuning procedure rather than of the algorithm itself.

The clear failure scenario for Smart Gossip is the small, sparse network. At 5 nodes, congestion is a non-problem—there is nothing to optimize away—while every retransmission matters for coverage; managed flood is the better algorithm there, also winning the composite comparison below and incurring lower delay (+28.7% for Smart Gossip). Conversely, the failure scenario for managed flood is density: its delay and airtime diverge with network size, which is incompatible with the scalability requirement of Section 3.1.4.

Composite Comparison

To aggregate the four metrics into a single comparative figure, a composite score was defined weighing effectiveness and efficiency equally and penalizing channel costs:

$$\text{score} = 0.35\,\text{reach} + 0.35\,\text{useful} - 0.15\,\text{col} - 0.15\,\frac{\text{tx}_{\text{air}}}{1000}. \tag{3.7}$$

The weighting differs slightly from the optimization fitness of Equation (3.6), placing heavier emphasis on channel costs, precisely to verify that the optimized parameters do not merely overfit their own objective. Table 3.7 shows the outcome: Smart Gossip wins in three of the four network sizes, with an advantage that grows with density, and loses only in the 5-node scenario where coverage dominates.

Table 3.7.: Composite score (Equation (3.7)) per network size.

Nodes	Smart Gossip	Managed Flood	Winner	Margin
5	32.06	**33.50**	MF	+1.44
15	**23.15**	20.73	SG	+2.42
30	**17.70**	13.03	SG	+4.68
50	**12.65**	8.28	SG	+4.38

In summary, the simulations demonstrate that Smart Gossip delivers comparable information dissemination at a 27–32% lower channel and energy cost, while avoiding the congestion collapse that renders flooding-based forwarding unusable—delays beyond a minute—at the network sizes the system is designed to scale to.

3.3.8. Final Parameter Selection

Table 3.8 lists the parameter values produced by the optimization, which are adopted as the firmware defaults of Chapter 3.5.

Table 3.8.: Optimized Smart Gossip parameters, search ranges, and physical interpretation.

Parameter	Symbol	Value	Search range	Interpretation
RSSI sigmoid steepness	α	0.145 dBm^{-1}	[0.02, 0.20]	high-to-low transition over ∼28 dB
RSSI inflection point	RSSI_{ref}	−103.8 dBm	[−120, −80]	∼600–800 m in the 3GPP model
Density weight	β	0.146	[0.01, 0.15]	7 neighbors → $f_{\text{density}} \approx 0.49$
Duplicate penalty	γ	0.311	[0.10, 0.60]	each extra copy reduces p by ∼27%
Probability floor	p_{floor}	0.111	[0.10, 0.40]	guaranteed minimum relay probability

Each value admits a coherent physical reading, which lends confidence that the optimizer converged to a meaningful operating point rather than a noise artifact. The inflection point of −103.8 dBm places the center of the RSSI transition at roughly 600–800 m under the simulated propagation model—comfortably inside coverage, so that only genuinely close receivers are suppressed. The steepness $\alpha = 0.145$ spreads the transition over approximately 28 dB, wide enough to absorb shadowing fluctuations. The density weight $\beta = 0.146$ halves the forwarding probability at around 7 neighbors, bounding the expected relay count per neighborhood near

$1/\beta \approx 7$ for very dense regions. The duplicate penalty $\gamma = 0.311$ attenuates each additional overheard copy by roughly 27%, and the floor of $p_{\text{floor}} = 0.111$ guarantees that even maximally suppressed nodes relay about one packet in nine. Notably, the optimizer drove both β toward the top of its search range and p_{floor} toward the bottom, confirming that in the evaluated regime the dominant lever is aggressive density-based suppression.

Two real-world implications follow. First, because all inputs to the decision are already available in the firmware (transceiver RSSI, neighbor table, duplicate cache), the algorithm's implementation cost is a handful of floating-point operations per received packet—negligible on the ESP32—and its parameters are stored as configuration constants that can be updated without protocol changes. Second, p_{floor} doubles as an application-level policy knob: deployments prioritizing delivery certainty over efficiency (for instance, emergency alert dissemination) can raise the floor toward 1, continuously trading channel efficiency back for the redundancy of full flooding without any other modification. The validation of these parameters on real hardware is presented in Chapter 3.7.

3.4. Hardware Design

3.4.1. Hardware Requirements

The system-level requirements established in Section 3.1.4 translate into a concrete set of hardware constraints that shaped every design decision in the custom printed circuit board (PCB). Four principal constraints govern the design space.

Autonomy. The device is intended for deployment scenarios in which mains power is unavailable for extended periods. A target runtime of several days of continuous mesh operation on a single lithium polymer (LiPo) cell demands that the combined average current draw of the microcontroller, LoRa transceiver, BLE radio, and all ancillary circuitry remains in the range of a few milliamperes.

Compactness. The node is designed to be carried alongside a smartphone, so the PCB form factor must be kept to a minimum consistent with the required component area. The resulting board measures 74.5 mm × 89.8 mm, a footprint that accommodates the integrated PCB antenna, the USB-C connector, the battery header, and all active circuitry without requiring multi-board stacking.

RF compliance. Operation in the EU 868 MHz ISM band is governed by ETSI EN 300 220 [Eur20]; most sub-bands within 863–870 MHz impose a maximum radiated power of 25 mW (14 dBm ERP) and a duty-cycle ceiling of 1 %. The firmware enforces the duty-cycle budget in software (Chapter 3.5), while the RF subsystem is sized to remain within the radiated limits achievable by the on-board PCB antenna at the configured transmit power.

Cost and manufacturability. As a prototype research platform, the design targets standard two-layer PCB fabrication and commercially available, off-the-shelf components sourced from LCSC. The complete bill of materials contains fewer than thirty distinct line items, all available in standard SMD packages ranging from 0402 to SOIC-16, check out appendix C.

3.4.2. System Block Diagram

The hardware platform is organised around four functional subsystems that interact through well-defined electrical interfaces, as illustrated in Figure 3.10. First, the *processing subsystem* is anchored by the Espressif ESP32-WROOM-32UE module [Esp23b], which hosts the application processor, the BLE radio, and the SPI master that controls the LoRa transceiver. Second, the *RF subsystem* comprises the HopeRF RFM95W-868S2 module [Hop18]—built around the Semtech SX1276 die [Sem19]—together with the integrated PCB monopole antenna derived from Texas Instruments Design Note DN024 [Tex08]. Third, the *power subsystem* converts a single-cell LiPo battery or a USB 5 V supply into a regulated 3.3 V rail through a cascaded charging, protection, and buck-boost stage. Finally, the *programming and user-interface subsystem* provides USB connectivity through a CH340C USB-to-UART bridge [Jia23], two tactile pushbuttons for manual boot selection and reset, and two indicator LEDs reflecting battery charging status.

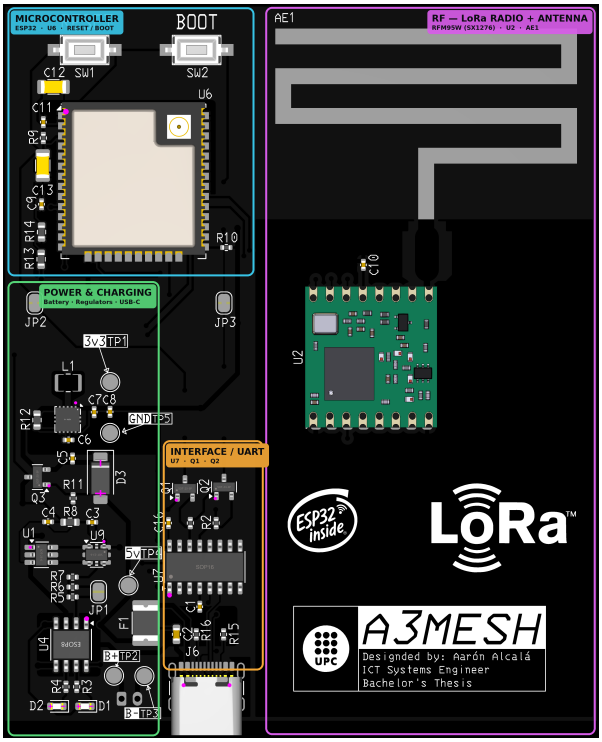


Figure 3.10.: PCB subsystems overlay.

3.4.3. Power Subsystem

The power subsystem is the most topologically complex portion of the design. Its function is threefold: to recharge a removable LiPo cell safely from a USB supply, to protect the cell against the electrical stress conditions that shorten its service life, and to deliver a regulated 3.3 V output whose quality is maintained across the entire discharge range of the battery. These three responsibilities are allocated to three dedicated stages arranged in series.

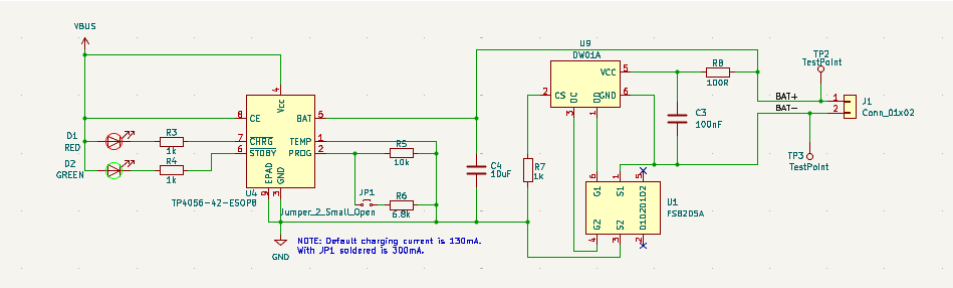


Figure 3.11.: Schematic of the power subsystem, showing the three cascaded stages that handle charging, protection, and voltage regulation.

Battery Selection

A single-cell lithium polymer cell was selected as the energy source. Lithium polymer chemistry offers an energy density superior to nickel-metal hydride or alkaline alternatives, a relatively flat discharge curve that simplifies state-of-charge estimation, and a well-established safety record when combined with dedicated protection electronics [Nan23]. The single-cell topology (nominal 3.7 V, 4.2 V fully charged, 3.0 V recommended cut-off) is directly compatible with the TP4056 charger and the TPS63001 buck-boost regulator without requiring series cell-balancing circuits. The cell is connected through a two-pin JST-compatible header (J1, mounted on the board underside) so that different cell capacities can be fitted to the same platform without hardware changes.

Charging Circuit

Charging is handled by the TP4056-42 (U4), a fully integrated constant-current constant-voltage linear charger for single-cell LiPo and Li-ion batteries in an ESOP-8 package with an exposed thermal pad [Nan23]. The IC accepts an input voltage between 4.0 V and 8.0 V, making it directly compatible with the USB 5 V supply, and terminates charging at 4.2 V with $\pm 1\,\%$ accuracy. A 500 mA polyfuse (F1, 1812 package) is inserted in series with the VBUS rail upstream of the charger to protect the USB supply from a downstream overcurrent fault.

The charge current is programmed by a single resistor connected between the PROG pin and ground. With $R6 = 6.8\,\text{k}\Omega$ alone (solder jumper JP1 open), the resulting charge current is approximately 130 mA—conservative by design, imposing no thermal demands on the compact package. Additionally, an optional solder jumper JP1 places a second resistor in parallel with R6, reducing the equivalent PROG resistance and raising the charge current to approximately 300 mA for applications where faster turnaround is preferred. Furthermore, the open-collector $\overline{\text{CHRG}}$ and $\overline{\text{STDBY}}$ outputs of the TP4056 drive Q1 and Q2 (SS8050 NPN transistors) through $1\,\text{k}\Omega$ current-limiting resistors, which in turn illuminate D1 (red) during active charging and D2 (green) when the cell has reached full charge.

A Schottky diode D3 (SS34, SMA package, 3 A rated) is placed in the power path to prevent reverse conduction from the battery back into the USB supply when the latter is absent or at a lower potential than the battery, ensuring that the system can be powered from either source without contention between the two rails.

Voltage Regulation

The regulated 3.3 V rail is generated by the Texas Instruments TPS63001 (U8), a synchronous buck-boost converter in the DRC0010J leadless package with an exposed thermal pad [Tex12]. The choice of a buck-boost topology is dictated by the battery voltage window: a freshly charged LiPo cell delivers 4.2 V (requiring step-down conversion), while a partially discharged cell may present 3.3 V or less (requiring step-up conversion). Hence, a purely buck or purely boost converter would lose regulation at one extreme of this range, while the TPS63001 transitions seamlessly between both modes as the cell discharges, maintaining a regulated 3.3 V across the full battery window from approximately 1.8 V to 5.5 V input.

The TPS63001 operates at a fixed switching frequency of approximately 1.5 MHz, which allows the external inductor (L1, 2.2 μH, 1008 package) and output capacitors to be kept small. In addition, the converter's typical peak efficiency exceeds 90 % across the battery's mid-discharge range [Tex12], a critical figure for meeting the multi-day autonomy target. Following the converter output, two bulk capacitors (C12, 100 μF, 1206; C13, 2.2 μF, 1206) are placed, complemented by distributed 100 nF and 10 μF decoupling capacitors close to the supply pins of all active devices.

Power Monitoring

In order to monitor the state of charge of the LiPo cell, the firmware measures its terminal voltage through a resistive divider connected to one of the ESP32's 12-bit analogue inputs. Given that the resistors R13 (330 kΩ) and R14 (470 kΩ) form a divider with ratio $470/(330 + 470) \approx 0.59$, scaling the 3.0–4.2 V battery range to approximately 1.76–2.47 V, comfortably within the 0–3.3 V ADC input range of the ESP32 [Esp23a]. Thus, the firmware applies a multiplicative correction factor to reconstruct the actual battery voltage from the sampled reading, and reports a low-battery warning when the reconstructed voltage falls below 3.0 V. The high total impedance of the divider ($\approx$800 kΩ) ensures that the monitoring path dissipates less than 6 μW at full charge, with a negligible impact on the overall energy budget.

3.4.4. Processing Subsystem

ESP32 Architecture

The central processing element is the Espressif ESP32-WROOM-32UE (U6), a fully-integrated system-in-package embedding a dual-core Xtensa LX6 processor clocked at up to 240 MHz, 520 KB of on-chip SRAM, 448 KB of ROM, and 4 MB of serial flash memory in a 38.5 mm $\times$ 18 mm metal-shielded module [Esp23b; Esp23a]. Moreover, the module integrates the ESP32 SoC, a 40 MHz crystal oscillator, a power amplifier, a low-noise amplifier, band-pass filters, and a RF switch for the 2.4 GHz Wi-Fi and Bluetooth radios, together with the matching network required for antenna connection. The *UE* variant features a U.FL connector for an external antenna rather than a printed trace antenna, which was selected to avoid desensitisation of the 2.4 GHz radio from the proximity of the 868 MHz LoRa circuitry.

The ESP32 communicates with the RFM95W LoRa module via a dedicated SPI bus, using the GPIO assignments listed in Table 3.9. The pin mapping follows the constraints of the ESP32's IOMUX multiplexer, which pre-assigns high-speed SPI signals to specific GPIO banks to avoid the additional propagation delay of the GPIO matrix [Esp23a].

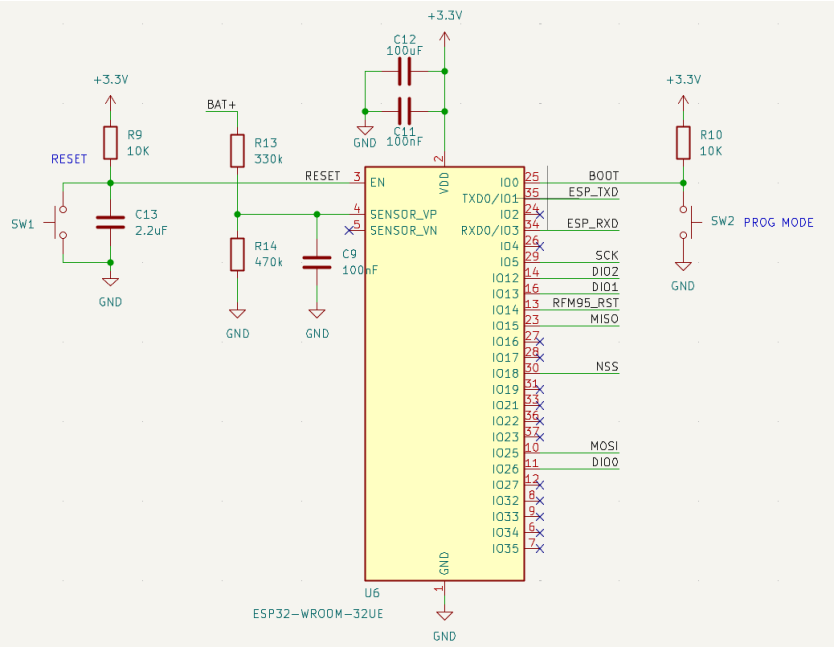


Figure 3.12.: Schematic of the processing subsystem, showing the ESP32-WROOM-32UE module.

Table 3.9.: SPI and control GPIO assignments between the ESP32 and the RFM95W.

Signal	Function	ESP32 GPIO
SCK	SPI clock	5
MOSI	Master Out Slave In	25
MISO	Master In Slave Out	15
NSS	Chip Select (active low)	18
RST	Hardware Reset (active low)	14
DIO0	RX/TX interrupt	26

BLE Integration

The ESP32's integrated 2.4 GHz radio supports Bluetooth 4.2, including the BLE stack required for the smartphone interface described in Chapter 3.6. The BLE subsystem operates exclusively in the peripheral role, advertising as a GATT server and maintaining a single connection to the paired smartphone. No external RF components are required for the BLE link: the module's internal antenna provides sufficient gain for the short-range smartphone pairing scenario.

Memory Considerations

The 4 MB flash within the module is partitioned into a bootloader region, the primary firmware image, and a non-volatile storage (NVS) partition that persists configuration parameters. The 520 KB of on-chip SRAM is shared between the FreeRTOS task stacks, the Bluedroid BLE host stack, and the application data structures, including the duplicate packet cache and

the neighbour table. Under the profiling conducted during firmware development, peak heap occupancy remained below 200 KB, leaving a margin sufficient for future protocol extensions without requiring external SRAM.

3.4.5. RF Subsystem

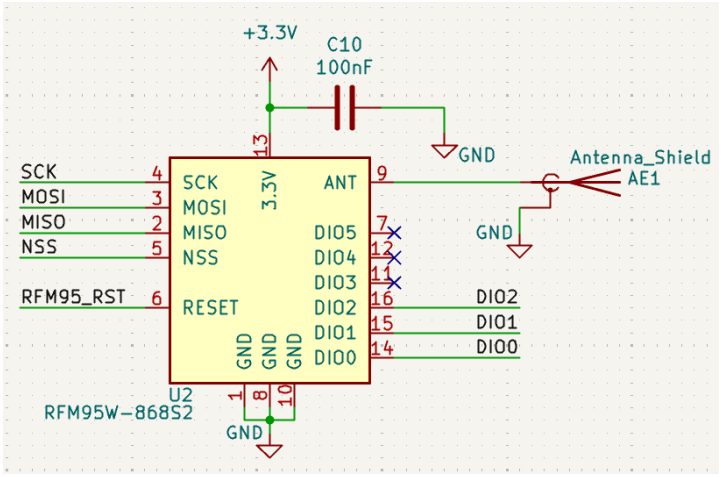


Figure 3.13.: Schematic of the RF subsystem, showing the RFM95W LoRa module.

LoRa Transceiver Selection

The LoRa radio is implemented by the HopeRF RFM95W-868S2 (U2), a compact SMD module measuring 16 mm × 17 mm that integrates the Semtech SX1276 transceiver die, a TCXO-grade crystal oscillator, RF matching network, and RF switch within a shielded metal enclosure [Hop18; Sem19]. The SX1276 is the same silicon underpinning the Meshtastic platform reviewed in Section 2.4.1, ensuring PHY-level compatibility with its packet format and enabling interoperability tests between A3MESH nodes and commercial Meshtastic hardware. The module operates from a 3.3 V supply and draws a transmit current of approximately 120 mA at maximum output power in PA_BOOST mode, and a receive current of approximately 10.3 mA [Sem19]; these figures directly inform the firmware duty-cycling strategy.

The module exposes a 16-pad castellated interface providing power supply, SPI bus, interrupt outputs, and an RF port that connects directly to the integrated PCB antenna, forgoing an external coaxial connector and its associated insertion loss.

Antenna Design Considerations

The RF radiator (AE1) is an integrated PCB monopole antenna implemented according to Texas Instruments Design Note DN024 [Tex08], which documents a compact meandered monopole geometry validated for 868 MHz, 915 MHz, and 955 MHz operation. The use of a printed antenna eliminates the cost, mechanical complexity, and failure mode of a coaxial connector, and is consistent with the compact form factor of the device. The same antenna geometry is widely deployed in commercially produced ISM-band sensor nodes, making it a well-characterised, industrially proven choice for this application.

A quarter-wavelength monopole resonant at 868 MHz would require a straight conductor of approximately 86 mm in free space. The DN024 geometry compresses this electrical length into a meandered copper trace occupying a footprint of 38 mm × 24 mm by folding the radiating element above a segmented ground plane, whose sections act as the return-current mirror and determine the radiation pattern and input impedance. The antenna's single feed pad connects directly to the RFM95W RF output pad, which includes the on-module matching network that presents a 50 Ω interface to the antenna feed point.

RF Matching and Layout

A keep-out zone of 38 mm × 24 mm is enforced on both copper layers beneath the antenna footprint; no vias, signal traces, or power planes encroach on this region, as partial copper coverage within the keep-out degrades the radiation pattern and lowers antenna efficiency [Tex08]. The SPI bus traces between the ESP32 and the RFM95W are routed on the front copper layer as short, direct connections to minimise series resistance and stray capacitance at the 10 MHz SPI clock frequency. All power supply traces to the RFM95W and ESP32 are supplemented with local 100 nF decoupling capacitors placed within 0.5 mm of each supply pin.

Regulatory Constraints

Operation in the EU 868 MHz band is subject to ETSI EN 300 220, which subdivides the 863–870 MHz range into sub-bands with individual power and duty-cycle limits [Eur20; Ade+17]. The majority of these sub-bands permit a maximum ERP of 14 dBm (25 mW) with a 1 % duty cycle. At a transmitter output power of 20 dBm and a PCB antenna with gain of approximately 0 dBi, the resulting EIRP marginally exceeds the 14 dBm sub-band limit of the standard channels.[1] The firmware's duty-cycle enforcement subsystem (Chapter 3.5) tracks the cumulative airtime over a sliding one-hour window and prevents the node from transmitting beyond the applicable regulatory budget, irrespective of the configured RF power level.

3.4.6. USB Programming Interface

Firmware development and debugging require reliable access to the ESP32's UART bootloader without manual intervention. This is accomplished by pairing the CH340C (U7) USB-to-UART bridge with a two-transistor auto-reset circuit.

The CH340C is a crystal-less USB-to-UART converter in a SOIC-16 package that derives its clock from the USB host bus, eliminating the tolerance sensitivity and cost of an external crystal [Jia23]. Its UART output is connected to the ESP32's default UART0 (GPIO pins 1 and 3), which serves both as the firmware programming channel and as the debug serial console at 115 200 baud.

The auto-reset circuit replicates the approach used on standard ESP32 development boards: the DTR and RTS control lines of the CH340C drive the base terminals of two SS8050 NPN transistors (Q1, Q2) through 100 Ω base resistors (R12, R8). The collectors are connected to the BOOT (GPIO0) and EN (reset) pins of the ESP32 respectively, each with a 10 kΩ pull-up to 3.3 V. When a host initiates a programming sequence, the USB-to-serial driver asserts a specific DTR/RTS toggle pattern that pulls EN low to reset the ESP32, then holds GPIO0 low

[1] Sub-band 869.4–869.65 MHz is an exception under ETSI EN 300 220, permitting up to 500 mW ERP with a 10 % duty cycle [Eur20]; the 20 dBm transmit level of the RFM95W falls within this limit. Formal EMC testing and a Declaration of Conformity are outside the scope of this prototype platform.

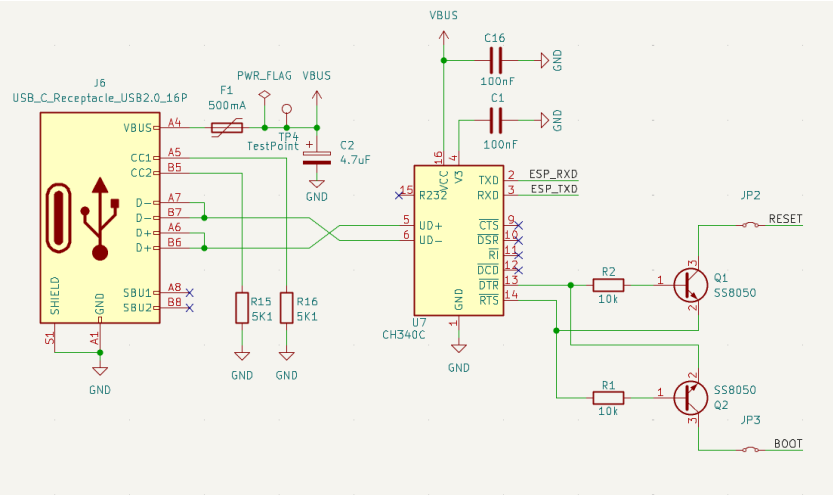


Figure 3.14.: Schematic of the USB programming and user interface subsystem, showing the CH340C USB-to-UART bridge and auto-reset circuit.

to latch it into bootloader mode upon reset release—all without the user pressing any button. Manual override remains available through SW1 (BOOT) and SW2 (RST) tactile switches.

The USB interface uses a 16-pin USB-C receptacle (J6, GCT USB4105 series) supporting USB 2.0 signals only. Two 5.1 kΩ resistors (R15, R16) pull down the CC1 and CC2 lines, which is the USB Type-C specification's mechanism for a device to signal its power sink requirements to a compliant charger or host port.

3.4.7. PCB Design

The board was designed in KiCad 9.0 and manufactured on a standard two-layer FR4 laminate, 1.6 mm nominal thickness. The copper stackup comprises a front signal layer (F.Cu) carrying the majority of the active components, and a back copper layer (B.Cu) used as a return-current reference and ground plane beneath the low-frequency digital circuitry.

Component placement follows a functional partitioning strategy. The power subsystem occupies the lower-left quadrant, with the TP4056, protection MOSFETs, and TPS63001 grouped to minimise the area of the high-frequency switching node of the buck-boost converter; the 2.2 μH inductor (L1) is placed as close as possible to the TPS63001's switching pins to reduce the current loop area and the associated electromagnetic emission. The ESP32 module is centred on the board, with SPI bus traces to the RFM95W module kept short. The CH340C and auto-reset transistors are clustered near the USB-C connector to minimise the length of the USB differential pair D+/D-.

The antenna occupies the top-right corner of the board, with the 38 mm × 24 mm keep-out zone maintained free of copper on both layers. The battery connector J1 is placed on the underside of the board to allow the cable to exit without obstruction.

Decoupling follows a two-tier strategy: 100 nF X5R capacitors in 0402 packages are placed within 0.5 mm of each active device supply pin to suppress switching transients, while 10 μF bulk capacitors distributed across the board buffer medium-frequency current demand. The 100 μF output capacitor of the TPS63001 provides low-frequency reservoir capacity.

The complete electrical schematic and the front and back copper layers of the resulting board are provided in Appendix B for reference.

3.4.8. Prototype Manufacturing

The KiCad layout was exported to Gerber files in RS-274-X format together with NC drill files, which were then submitted to JLCPCB for fabrication of a two-layer PCB. All SMD passives use 0402 or 0603 packages, except for bulk capacitors, which require the larger 1206 package to accommodate their voltage ratings. Integrated circuits range from SOT-23-6 to SOIC-16, while the USB-C receptacle and battery header are semi-through-hole parts, chosen for the mechanical robustness they provide against repeated connector cycles.

Furthermore, the BOM was compiled with LCSC part numbers so that JLCPCB could source every component from a single distributor, streamlining assembly. Finally, seven test-point pads were distributed across the board to facilitate in-circuit measurement of the key power rails (VBUS, 3.3 V, BAT+) and programming signals during bringup.

3.4.9. Mechanical Integration

The mechanical layer closes the loop between the PCB of Section 3.4.7 and the accessibility requirement of Section 3.1.4: without an enclosure the user perceives as a natural part of the smartphone, the hardware effort remains a bare board. The subsections that follow state the design objectives, the ergonomic and integration decisions taken to meet them, the 3D-modelling process, and the resulting enclosure geometry.

Design Objectives

The primary objective was an accessory the user would not need to consciously manage: an "install once, forget it's there" experience, in which the board becomes a permanent extension of the phone rather than a separate object to keep track of. To achieve this, the PCBA is housed in a dedicated shell that is permanently bonded to the back of the phone's existing case with an adhesive layer, following the same attachment principle used by smart-wallet card holders. This removes any manual connection step and satisfies the core usability goal of the project.

Ergonomic Considerations

Building on the goal of making the accessory feel like a natural part of the phone, two main ergonomic decisions were made. First, all external edges of the enclosure were rounded, eliminating sharp corners and edges so that the device sits comfortably in the hand during normal phone use, in the same way that commercial phone cases avoid hard edges.

Second, the enclosure was modelled to be 3D printed in a flexible, rubber-like plastic, matching the material commonly used in standard phone cases. This material choice provides a soft tactile feel consistent with the rest of the phone accessory, while also adding a degree of shock absorption to protect the PCBA inside.

Smartphone Integration

Several attachment strategies were evaluated before settling on the adhesive-mounted shell described in Section 3.4.9. One alternative considered was integrating a small dedicated pocket

directly into the phone case, sized to hold the board internally. This approach would reduce the overall added thickness and improve comfort, but it was discarded because it is not scalable: a custom case would need to be designed and manufactured for every phone model, which is impractical for a general-purpose accessory.

A second alternative considered was a MagSafe-style magnetic attachment. This option was ultimately rejected due to the risk of the magnets' electromagnetic field interfering with the radiofrequency signals handled by the board, which could compromise the device's wireless performance.

Given these constraints, the adhesive-backed shell was selected as the final solution, as it offers a reasonable compromise between universality, ease of installation, and signal integrity.

3D Modelling Process

The enclosure was designed using Autodesk Fusion 360 as the primary CAD tool. To ensure the design would integrate correctly with a real device, a 3D model of a phone similar to the target unit was sourced online and used as a reference geometry throughout the modelling process, allowing the enclosure's dimensions and curvature to closely match the actual phone form factor.

In parallel, the 3D model of the PCBA was exported directly from KiCad. This ensured that the internal cavity of the enclosure was dimensioned to match the real board geometry as closely as possible, avoiding the tolerance issues that can arise when modelling component placement from datasheets alone.

Enclosure Design

Figure 3.15 shows the final enclosure design housing the PCBA. The rounded edges, the rubber-like surface finish, and the discreet ventilation slots can be appreciated in the render, along with the engraved logo on the rear face. The bottom edge integrates the connector cutout required for charging and data access, keeping the silhouette as close as possible to that of a standard phone case.

Additionally, two small elements made of a transparent material can be observed. These act as light diffusers for the red and green charging status LEDs mounted on the PCBA: since the LEDs themselves are fully enclosed within the opaque shell, these transparent inserts channel and scatter their light to the outer surface, allowing the user to clearly see the charging status indicators from outside the enclosure.

3.5. Firmware Architecture and Implementation

This chapter describes the design philosophy, the layered decomposition, and the concrete implementation of each subsystem, drawing directly from the `tfg_firmware/firmware` PlatformIO project.

3.5.1. Firmware Design Philosophy

Three principles govern the design of the A3MESH firmware: strict layer separation, non-blocking interrupt handling, and static memory allocation.



Figure 3.15.: Final enclosure design housing the PCBA.

Strict layer separation. Each software layer communicates with adjacent layers exclusively through FreeRTOS queues; no layer directly invokes functions from a non-adjacent layer. The sole deliberate exception is the C-linkage wrapper in `PassiveACK.h`, which allows the LoRa transmit task—residing in the `lora` PlatformIO library directory—to call into the mesh layer without creating a circular header dependency between library directories.

Non-blocking interrupt handling. Follows the deferred-processing pattern standard in FreeRTOS embedded designs [24]. In particular, the SX1276's DIO0 pin asserts on RxDone and triggers an ISR declared `IRAM_ATTR`, so that it executes from instruction RAM regardless of flash cache state. The ISR itself does nothing but release a binary semaphore and optionally request a context switch. All SPI transfers and frame parsing occur in a dedicated high-priority task, ensuring that the radio FIFO is emptied before a subsequent preamble can corrupt it.

Static memory allocation. Eliminates heap fragmentation and non-deterministic latency on the critical path. For instance, a pool of eight statically allocated frame slots accommodates bursts of arriving frames; inter-task transfers then pass a 32-bit pointer to a pool slot rather than copying the payload, reducing queue-transfer overhead to a single word. As a result, no dynamic allocation occurs on any path between frame reception and forwarding decision.

3.5.2. Layered Architecture

The firmware is decomposed into four horizontal layers. Firstly, the *Physical Communication Layer* encapsulates the LoRa and BLE radio interfaces. Secondly, the *Mesh Logic Layer* implements packet validation, duplicate suppression, neighbour management, and the Smart Gossip forwarding algorithm. Thirdly, the *System Services Layer* handles regulatory duty-cycle compliance, battery state estimation, and periodic housekeeping. Finally, the *Application and Interface Layer* exposes an AT-command interface over BLE to the companion mobile application and manages the bidirectional data pipelines between radio and smartphone. Figure 3.16 illustrates the layered architecture.

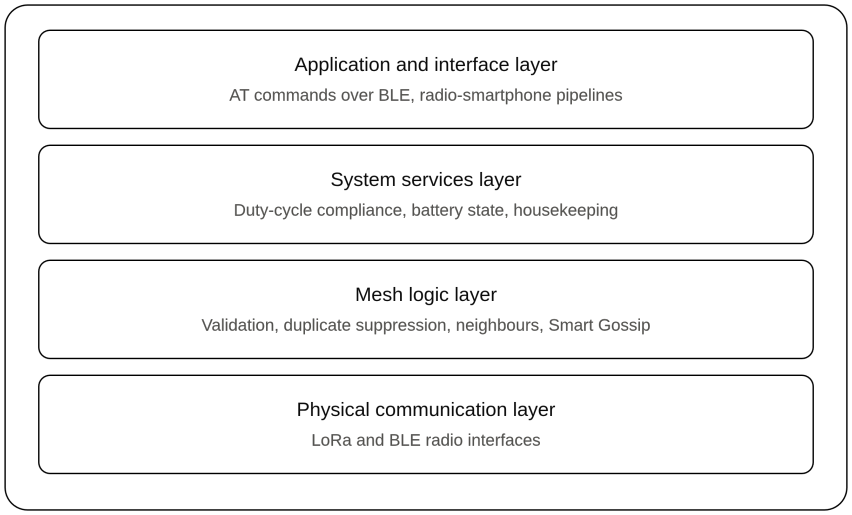


Figure 3.16.: Firmware layer diagram.

Eight FreeRTOS tasks are created at boot time. Four are pinned to Core 0, dedicated to all radio activity; four are pinned to Core 1, which handles BLE, pipeline processing, and power management (Table 3.10).

Table 3.10.: FreeRTOS task inventory.

Task	**Core**	**Prio**	**Role**
`vLoRaRxTask`	0	2	ISR deferred handler; frame pool allocation
`vLoRaTxTask`	0	1	Jittered forwarding and duty-cycle gate
`vSendDiscoveryTask`	0	1	Periodic DISCOVERY beacon injection
`vNeighborCheckTask`	0	1	Neighbour-table timeout eviction
`vBLETxTask`	1	1	BLE GATT notification drain
`vBLE2LoRaTask`	1	1	AT command parser; upstream packet injector
`vLoRa2BLETask`	1	1	Mesh receive state machine
`vPowerTask`	1	1	Adaptive CPU frequency scaling

Four queues provide all inter-layer communication:

- `xLoRaRxQueue` carries pointers to pool-allocated frame descriptors from the radio ISR handler to the mesh pipeline.
- `xLoRaTxQueue` carries outbound frame descriptors from the mesh and application layers to the radio transmitter.
- `xBLERxQueue` carries AT-command strings from the BLE stack to the command processor.
- `xBLETxQueue` carries AT response and notification strings from any task towards the BLE notification sender.

All queues have a depth of ten items and are statically allocated at initialisation.

3.5.3. Physical Communication Layer

LoRa Driver

The LoRa driver wraps the Arduino-LoRa SPI library with an interrupt-deferred receive path, a static frame pool, and regulatory airtime accounting. It is instantiated as a global singleton in `main.cpp` and configured at boot with the parameters defined in `config.h`: 868 MHz carrier, spreading factor 8, bandwidth 125 kHz, coding rate 4/5, 20 dBm transmit power via PA_BOOST, sync word 0x12, and a preamble of eight symbols. These values yield a raw data rate of approximately 3.1 kbit/s and a practical ground-level range of several kilometres while keeping the time-on-air of a maximum-length frame below one second.

Receive path. The SX1276 DIO0 line is connected to GPIO26 and configured as a rising-edge interrupt. When it fires, the ISR `onLoRaRxIRQ()`, placed in IRAM, calls `xSemaphoreGiveFromISR()` and, if this unblocks a higher-priority task, sets `xHigherPriorityTaskWoken` accordingly. This semaphore is what `vLoRaRxTask` is waiting on: running at priority 2 on Core 0, the task blocks indefinitely on the corresponding `xSemaphoreTake()` until the ISR wakes it. Once woken, it first calls `LoRa.parsePacket()` to confirm frame availability and obtain its length, then allocates a pool slot via `alloc_frame()` using a volatile round-robin index [TB15, Ch. 2], and reads the payload bytes with `LoRa.readBytes()`. It also records the RSSI, before re-entering continuous-receive mode with `LoRa.receive()` and posting the pool-slot pointer to `xLoRaRxQueue`. Throughout this path, no copying takes place: the bytes written by the SPI driver into the pool slot are the same bytes later consumed by the mesh pipeline.

Transmit path. `vLoRaTxTask` blocks on `xLoRaTxQueue`, waiting for items to dequeue. Each dequeued item is a `tx_LoRa_msg_t` descriptor containing the payload bytes, a jitter delay in milliseconds, a packet identifier, the original source address, and a forwarding flag. The task then calls `vTaskDelay()` for the specified jitter period, which in turn creates the window during which a passive ACK cancellation may suppress the transmission (Section 3.5.4). Once this delay elapses, it calls `mesh_consume_pending_forward()`: if the forward has been cancelled in the meantime, the descriptor is simply discarded; otherwise, `check_and_record_airtime()` is invoked first, followed by `lora.send_raw_pkt(data, len)`.

Duty-cycle enforcement. Under the EU868 general sub-band, the maximum allowed on-air time is 1 % per hour [Eur20], or equivalently 36 000 ms per rolling 3600-second window. To enforce this limit, the driver maintains a ring buffer of 20 transmission records (`_airtime_log`), where each entry holds a timestamp together with the airtime estimated by `compute_airtime_ms(pkt_len)` for that packet. Before each transmission, `check_and_record_airtime()` sums the airtime of all entries whose timestamp falls within the last 3600 s; if adding the pending packet would then push the total above 36 000 ms, the transmission is blocked silently. By design, this sliding window avoids the compliance gaps that a fixed-period counter would otherwise introduce around midnight or after a device restart.

BLE Driver

The BLE driver implements a Nordic UART Service (NUS)-compatible GATT server using the ESP-IDF Bluedroid stack [23]. In particular, a single GATT service hosts two characteristics: a TX characteristic with the NOTIFY property, used to push data from node to smartphone, and an RX characteristic with the WRITE WITHOUT RESPONSE property, used by the smartphone to send AT commands. This two-characteristic layout is in fact the standard for transparent serial emulation over BLE, and as such is natively supported by serial terminal applications across Android and iOS [Blu23].

Connection lifecycle, in turn, is managed by `ServerCallbacks`: `onConnect` sets the internal `_deviceConnected` flag, while `onDisconnect` clears it and immediately restarts advertising, so that the node remains always connectable without requiring a reboot. When the smartphone writes to the RX characteristic, `RXCallbacks::onWrite()` copies the received value into a 256-byte buffer and then posts it to `xBLERxQueue` via a non-blocking `xQueueSend`; if the queue happens to be full, indicating transient saturation of the command processor, the frame is dropped rather than blocking the Bluedroid callback thread. Outbound data, meanwhile, travels the reverse path: `vBLETxTask` on Core 1 drains `xBLETxQueue` and, for each dequeued string, calls `ble.send()`, thereby issuing a GATT notification to the connected peer. By decoupling BLE transmission into a dedicated task, long notify round trips are prevented from stalling time-sensitive state-machine transitions.

Packet Encoding

Every frame transmitted over the LoRa channel consists of a fixed four-byte `A3MESH_header_t` followed by a variable-length `A3MESH_packet_t`. The header carries a one-byte protocol version, a one-byte frame type (`a3mesh_msg_type_t`), and a two-byte packet length as shown in Table 3.11.

Table 3.11.: Fields of the `A3MESH_header_t` wire structure.

Field	Size	Description
`version`	1 B	Protocol version identifier.
`type`	1 B	Frame type (`a3mesh_msg_type_t`); determines how the following payload is interpreted.
`packet_length`	2 B	Length in bytes of the following `A3MESH_packet_t`.

While the packet structure itself is declared `__attribute__((packed))` to match the on-wire layout exactly. Regarding to its fields are listed in Table 3.12.

Table 3.12.: Fields of the `A3MESH_packet_t` wire structure.

Field	Size	Description
`pkt_id`	2 B	Per-source monotonic counter; paired with `src` for global duplicate identification.
`src`	4 B	Node ID of the original source; never modified during forwarding.
`node_id`	4 B	Node ID of the last forwarder; updated at each hop.
`dst`	4 B	Destination node ID, or `0xFFFFFFFF` for broadcast.
`ttl`	1 B	Time-to-live; decremented at each hop; frame dropped when zero.
`type`	1 B	Packet sub-type (`a3mesh_enc_type_t` for data frames).
`flags`	1 B	Control bits: bit 0 is the direct-unicast hint, which suppresses gossip forwarding.
`payload_len`	1 B	Number of valid bytes in the payload array.
`crc`	2 B	CRC-16/CCITT (poly 0x1021, init 0xFFFF) over `payload[0..payload_len-1]`.
`payload`	N B	Application data or ciphertext, up to `A3MESH_MAX_PAYLOAD` bytes.

The four frame types defined by `a3mesh_msg_type_t` are:

- `k_data_plain`: unencrypted user message.
- `k_discovery_type`: neighbour beacon.
- `k_key_exchange`: in-band key provisioning.
- `k_data_encrypted`: AES-256-GCM ciphertext; the encryption scope is further encoded in the `type` sub-field as `a3mesh_enc_type_t`:
  - `k_enc_node2node`: unicast between two nodes
  - `k_enc_group`: multicast to a group of nodes sharing the same key
  - `k_enc_network`: broadcast to all nodes in the network

Serialisation, performed by `A3MESHPacket::serialize_header_pkt()`, fills the header's `packet_length` field automatically and writes both structures into a caller-provided buffer in packed order. Deserialisation, by `A3MESHPacket::deserialize_pkt()`, applies four integrity checks before accepting a frame: minimum buffer length against `HEADER_SIZE`, buffer length

against the declared `packet_length`, payload length against `A3MESH_MAX_PAYLOAD`, and CRC-16/CCITT verification of the payload; any failing check discards the frame without further processing. The node identifier used in `src` and `node_id`, in turn, is derived from the factory Bluetooth MAC address as described in Section 3.5.5.

3.5.4. Mesh Logic Layer

Reception State Machine

All received frames are processed by a ten-state explicit finite state machine (`MeshRxState`) running inside `LoRa2BLETask`. Every frame traverses exactly one path from `k_rx_idle` to `k_rx_done`; the only branching points are the validate, classify, data-process, and forward handlers. The states are summarised in Table 3.13.

Table 3.13.: `MeshRxState` states and their actions.

State	Action
`k_rx_idle`	Block on `xLoRaRxQueue`; dequeue pool pointer on arrival.
`k_rx_validate`	Deserialise; apply self-source filter, TTL check, duplicate cache lookup, and passive-ACK cancel.
`k_rx_classify`	Route by `header.type` to the appropriate handler state.
`k_rx_process_discovery`	Update neighbour table; notify BLE on new neighbour discovery.
`k_rx_process_data`	Check destination; decide local delivery and/or gossip forwarding.
`k_rx_process_key`	Check destination; deliver key-exchange payload to BLE locally.
`k_rx_local_delivery`	Decrypt if encrypted; format and post AT notification to `xBLETxQueue`.
`k_rx_forward`	Patch `node_id`; decrement TTL; compute jitter; register pending forward; enqueue to `xLoRaTxQueue`.
`k_rx_drop`	Release pool slot; discard frame.
`k_rx_done`	Release pool slot; return to `k_rx_idle`.

Packet validation. First, at `k_rx_validate`, four guards are applied in order: a self-source filter which drops any frame whose `src` matches the local node ID, preventing a node from processing its own transmissions returning via a relay. Then, a TTL check discards frames with `ttl` $= 0$. Next, the duplicate cache is then consulted via `get_copies()`: if the (`pkt_id`, `src`) pair has been seen more than `GOSSIP_MAX_COPIES` $= 3$ times, the frame transitions to `k_rx_drop`. Finally, `cancel_pending_forward()` is called unconditionally for every valid frame that matches a message in the cache: a node that has already queued a forward of this packet and now observes a neighbour retransmitting it cancels its own copy before it reaches the antenna (Section 3.5.4).

Packet classification. Following validation—at `k_rx_classify`—the state machine branches to one of three handlers based on the `header.type` field.

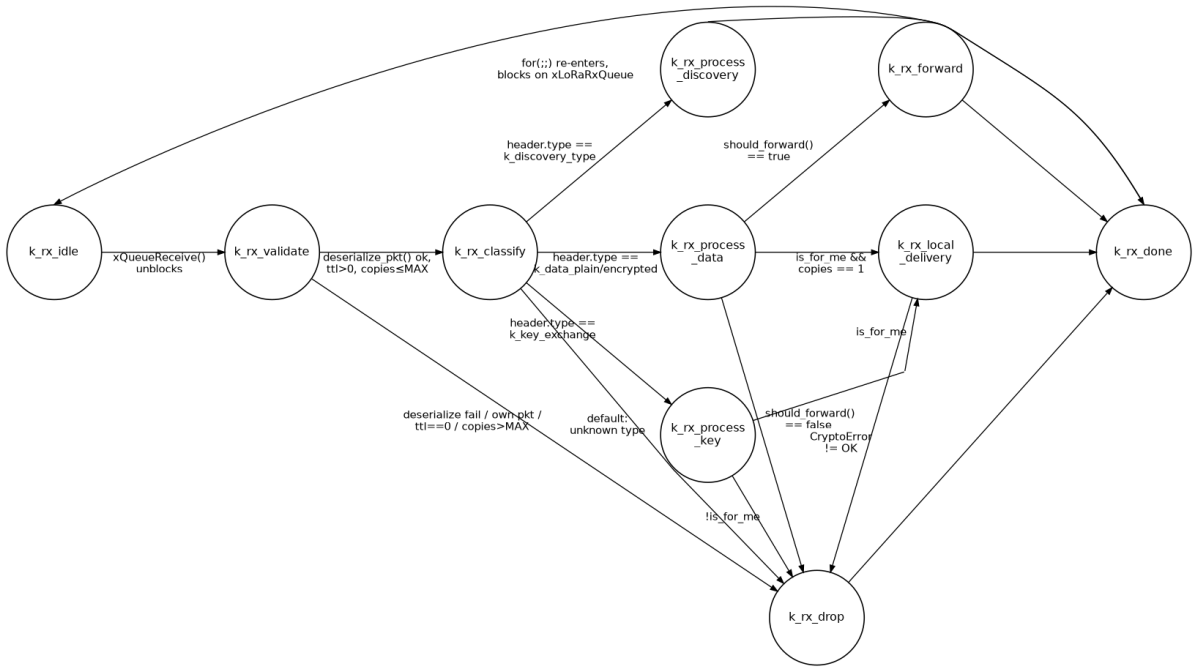


Figure 3.17.: Reception state machine diagram.

Data packet handling. At `k_rx_process_data`, broadcast and locally-addressed frames proceed to `k_rx_local_delivery` only when `copies` = 1, preventing the user from receiving the same message twice via different relay paths. Frames carrying the `DIRECT_UNICAST_FLAG` in the `flags` byte are not gossiped; all other data frames enter `should_forward()` for the probabilistic forwarding decision.

At `k_rx_forward`, the forwarding node patches `node_id` with its own identifier and decrements `ttl` by one. An RSSI-based jitter delay is then computed with the linear mapping consistent with the distance-aware relay selection studied in [SYA11]:

$$d_{\text{jitter}} = \text{map}(\text{RSSI},\ -120,\ -40,\ 20,\ 200)\ \ [\text{ms}]$$

Neighbour Discovery

The mesh layer maintains a neighbour table of up to `MAX_NEIGHBORS` = 10 entries, each recording the 32-bit node identifier, the most recently observed RSSI, and the timestamp of the latest reception; entries expire after `NODE_TIMEOUT` = 180 s of silence.

`vSendDiscoveryTask` injects beacons, waking every `DISCOVERY_INTERVAL_MS` = 30 s plus a random offset of up to 10 s, which desynchronises simultaneous power-ons and reduces collisions between concurrent DISCOVERY transmissions from neighbouring nodes. Each beacon carries `ttl` = `MIN_TTL` = 1, limiting propagation to a single hop so that the table reflects only directly reachable nodes. On reception, the validate state updates the corresponding entry; if the node is new, `k_rx_process_discovery` also posts a `+DISC=⟨id⟩` notification to `xBLETxQueue` to inform the companion application.

Stale entries are evicted by `vNeighborCheckTask`, which uses `vTaskDelayUntil()` for drift-free periodicity at a wakeup interval of `NODE_TIMEOUT`/10 ms. Its eviction function `rm_tout_neigbhors()` applies swap-last deletion—overwriting the expired slot with the last table entry—completing in O(1) time without shifting the array.

Smart Gossip Integration

The forwarding probability is computed by `gossip_forward_probability()`, the firmware implementation of the algorithm described in Chapter 3.3 using the compiled parameter values that correspond to the differential evolution optimum and are embedded as compile-time constants in `config.h`, that returns a fixed-point percentage.

The forwarding decision in `should_forward()` draws a uniform random integer from `esp_random()` % 100 and forwards if it falls below the threshold. `esp_random()` reads the ESP32 hardware random number generator, which is continuously seeded from RF thermal noise when the BLE radio is active [Esp23a], providing statistically independent draws without the sequential correlations of a software PRNG.

Duplicate Detection and Passive ACK Suppression

Duplicate detection. The duplicate cache stores up to `DUP_CACHE_SIZE` = 50 entries, each keyed by the (`pkt_id`, `src`) pair and carrying a copy count c and a reception timestamp; entries expire after `DUP_TIMEOUT` = 30 s, and once the cache is full, a new entry evicts the least-recently-used one. Since the copy count fed into f_{dupe} is read before the forwarding probability is evaluated, that probability decays monotonically as additional relay paths deliver copies of the same message, naturally throttling rebroadcast in well-connected regions.

Passive ACK suppression. When the state machine transitions to `k_rx_forward`, it calls `mesh_register_pending_forward()` to insert a `PendingForward` record—holding the packet identifier and an `active` boolean initialised to `true`—into a mutex-protected table of up to `MAX_PENDING_FORWARDS` = 8 entries, before posting the `tx_LoRa_msg_t` descriptor to `xLoRaTxQueue`. If, during the jitter delay, the same message arrives again from a neighbour that forwarded it independently, the validate state calls `cancel_pending_forward()`, clearing `active`; when `vLoRaTxTask` later calls `mesh_consume_pending_forward()`, it then finds the record inactive and discards the descriptor without transmitting—eliminating a redundant on-air copy with no explicit coordination between nodes as illustrated in Figure 3.18.

TTL and Loop Prevention

Network loops are controlled by two independent mechanisms. At the packet level, every forwarding node decrements the TTL field by one, and once it reaches zero the frame is discarded at `k_rx_validate` before any duplicate-cache update; with `MAX_TTL` = 5, this bounds the propagation diameter to five hops per transmission event.

At the message level, the duplicate cache provides a complementary bound: even if a packet re-enters a previously visited node through an unexpected cycle, the cache increments its copy count, and once that count exceeds `GOSSIP_MAX_COPIES` = 3, the packet is dropped unconditionally. Together, the two limits bound the worst-case replication factor of any message to at most $3 \times 5 = 15$ on-air copies within its 30-second cache lifetime.

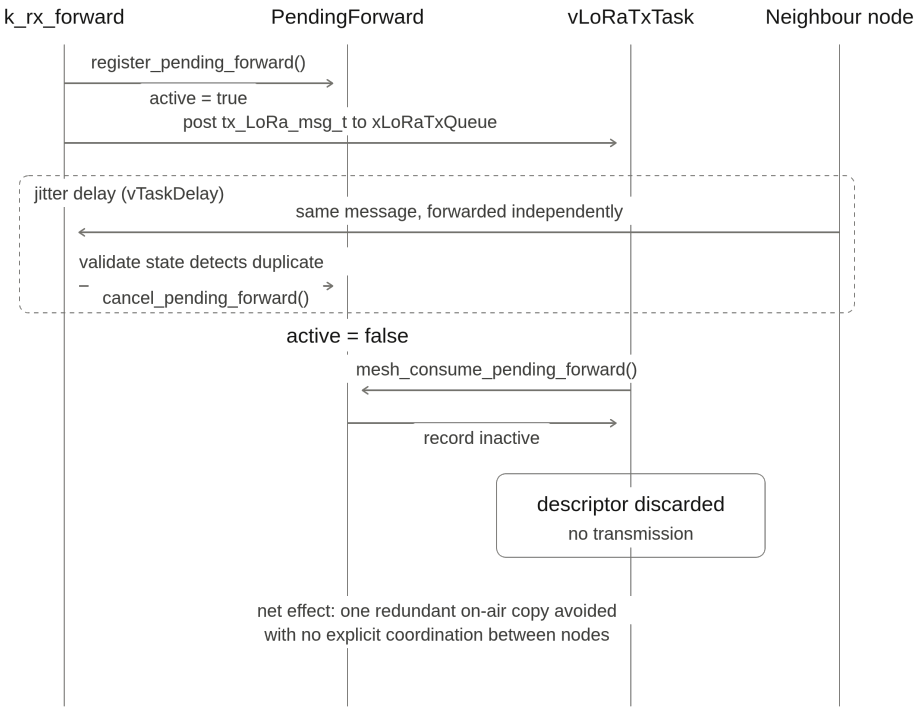


Figure 3.18.: Passive ACK suppression sequence-diagram.

3.5.5. System Services Layer

Power Management and Battery Monitoring

Sleep modes and their unsuitability. The A3MESH node must remain continuously active as a mesh relay, which rules out both ESP32 sleep modes. Deep sleep would discard all volatile state—the passive-ACK pending-forward table, the neighbour table, and the duplicate cache—and terminate the BLE GATT connection, requiring a full re-initialisation on each wake-up cycle and making sustained mesh participation impossible. Light sleep is equally unsuitable for two independent reasons. First, the ESP32 BLE radio requires a minimum CPU clock of 80 MHz for its timing constraints to be met [Esp23a]; below this threshold the Bluetooth controller ceases to function correctly. Second, the SX1276 LoRa transceiver has no autonomous wake-up capability over SPI: packets received while the CPU is asleep fill the transceiver FIFO, which overflows silently if the processor does not service the DIO0 interrupt promptly, causing frame loss. Transmit energy is bounded through the duty-cycle enforcement mechanism of Section 3.5.3.

Adaptive CPU frequency scaling. Since the node must remain awake but is frequently idle between mesh events, dynamic power consumption is reduced through adaptive clock scaling, implemented by the `PowerManager` class. The CPU nominally runs at `POWER_ACTIVE_MHZ` = 240 MHz. A dedicated FreeRTOS task (`PowerTask`), pinned to Core 1 at priority 1 and polling every `POWER_CHECK_INTERVAL_MS` = 2 min, monitors the elapsed time since

the last call to `notify_activity()`. When this interval exceeds `POWER_IDLE_TIMEOUT_MS` = 10 min—indicating that no valid LoRa packet has been received—the task invokes `setCpuFrequencyMhz(POWER_IDLE_MHZ)`, reducing the clock to 80 MHz. This is precisely the minimum frequency at which both the BLE radio and the LoRa SPI driver continue to operate correctly, making it the lowest safe operating point with no radio disruption. On the next valid packet reception, the receive path calls `notify_activity()`, which immediately restores the clock to 240 MHz. The idle-to-active transition is detected by an atomic compare-and-exchange on a `std::atomic<bool>` flag, so no mutex is required on the forwarding hot path.

Battery state estimation. The `Battery` class converts a raw ADC sample into a state-of-charge percentage using a 20-point look-up table of (voltage, percentage) pairs that characterises a typical LiPo discharge curve, spanning 4.20 V at 100 % down to 3.00 V at 0 %. A 12-bit ADC reading from pin A0 (GPIO36) is first scaled to the actual cell voltage by $V_{\text{bat}} = (\text{raw}/4095) \times 3.3 \times k$, where $k =$ `BATTERY_ADC_FACTOR` $= 2.0$ compensates for the R13/R14 resistive divider described in Chapter 3.4. The corresponding look-up entry is then located by a linear scan; if the measured voltage falls between two adjacent entries, the percentage is determined by linear interpolation, yielding a smooth continuous estimate rather than a coarse staircase. The resulting percentage is reported to the companion application via the `AT+INFO?` command. As a hardware safeguard independent of firmware state, the DW01A + FS8205A protection stage provides a hard cut-off near 2.4 V to prevent deep discharge.

Node Identity

Each node derives its 32-bit identifier deterministically from the factory-programmed six-byte Bluetooth MAC address using the FNV-1a non-cryptographic hash function [EH21], computed iteratively as $h_0 = \text{FNV_OFFSET_BASIS}$ and, for $i = 0, \ldots, 5$:

$$h_{i+1} = (h_i \oplus \text{MAC}[i]) \times \text{FNV_PRIME} \pmod{2^{32}}$$

with ID $= h_6$. Since all six MAC bytes are folded into the accumulator before the final multiplication, the 32-bit output is sensitive to the full MAC address while remaining negligibly cheap to compute at boot time, and because it stays invariant across firmware updates and power cycles, peer devices and the duplicate cache can maintain consistent state without manual reconfiguration.

3.5.6. Application and Interface Layer

AT Command Interface

The companion application communicates with each node through a Hayes-style AT command protocol [Int03] carried over the BLE NUS service. Each command is a CRLF-terminated ASCII string, and the node replies with zero or more unsolicited result lines followed by a final status code: `+OK` indicates that the command was executed successfully, while `+ERR=n` indicates failure, with the integer `n` identifying the specific error condition. Appendix A lists the complete command set.

BLE-to-LoRa Pipeline

On the upstream path, `BLE2LoRaTask` on Core 1 blocks on `xBLERxQueue` and, for each command received from the smartphone, performs a prefix match against the command table from Section 3.5.6 to determine the appropriate action. Transmission commands construct and serialise an `A3MESH_packet_t` (delegating encryption to `CryptoManager` when required) before posting it to `xLoRaTxQueue`; query commands read from the mesh manager, battery driver, or node-ID register and write their responses directly to `xBLETxQueue`.

Figure 3.19 illustrates the sequence of events described above.

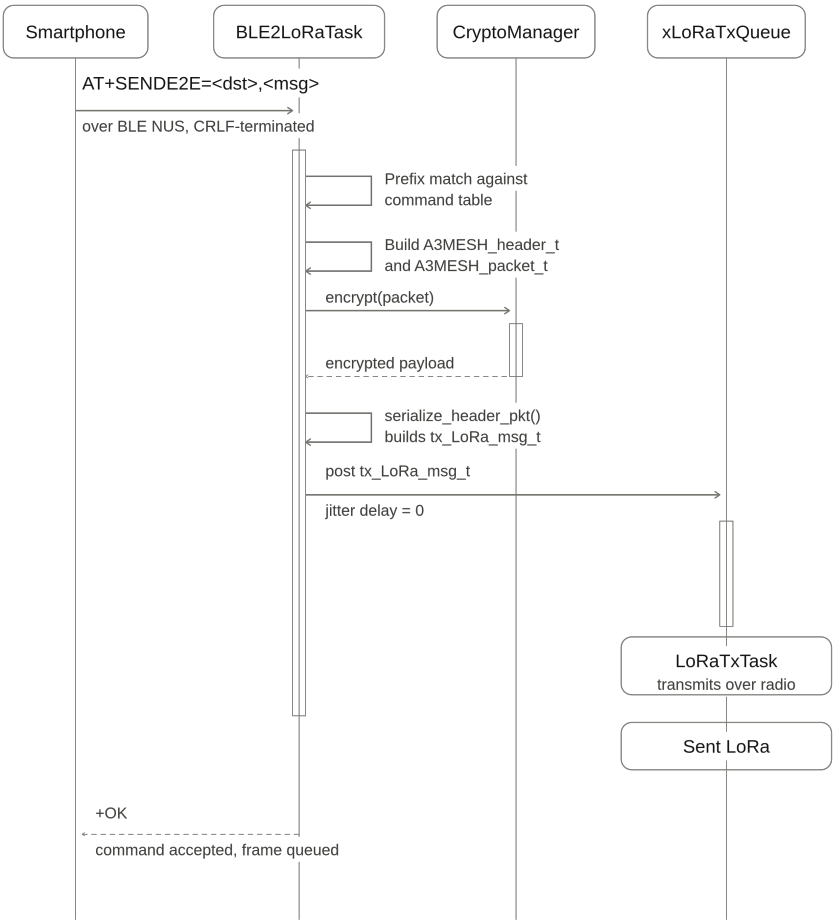


Figure 3.19.: BLE-to-LoRa sequence diagram.

LoRa-to-BLE Pipeline

The downstream pipeline carries received mesh traffic to the smartphone (Figure 3.20). A frame arriving over the air triggers the DIO0 ISR, which wakes `vLoRaRxTask`; the task reads the frame into a pool slot and posts the pointer to `xLoRaRxQueue`. `LoRa2BLETask` then dequeues the pointer on Core 1 and drives the reception state machine described in Section 3.5.4. If

the destination matches the local node (or the packet is broadcast), the machine reaches `k_rx_local_delivery`: the formatted AT notification string is placed on `xBLETxQueue`, and `vBLETxTask` delivers it to the smartphone via a GATT notification. If the packet must instead be relayed, the machine reaches `k_rx_forward`: a `tx_LoRa_msg_t` descriptor is posted to `xLoRaTxQueue`, re-entering the transmit path on Core 0. These two transitions are independent rather than mutually exclusive: a broadcast packet with an affirmative gossip decision triggers both local delivery and a forwarded copy in the same pass through the state machine.

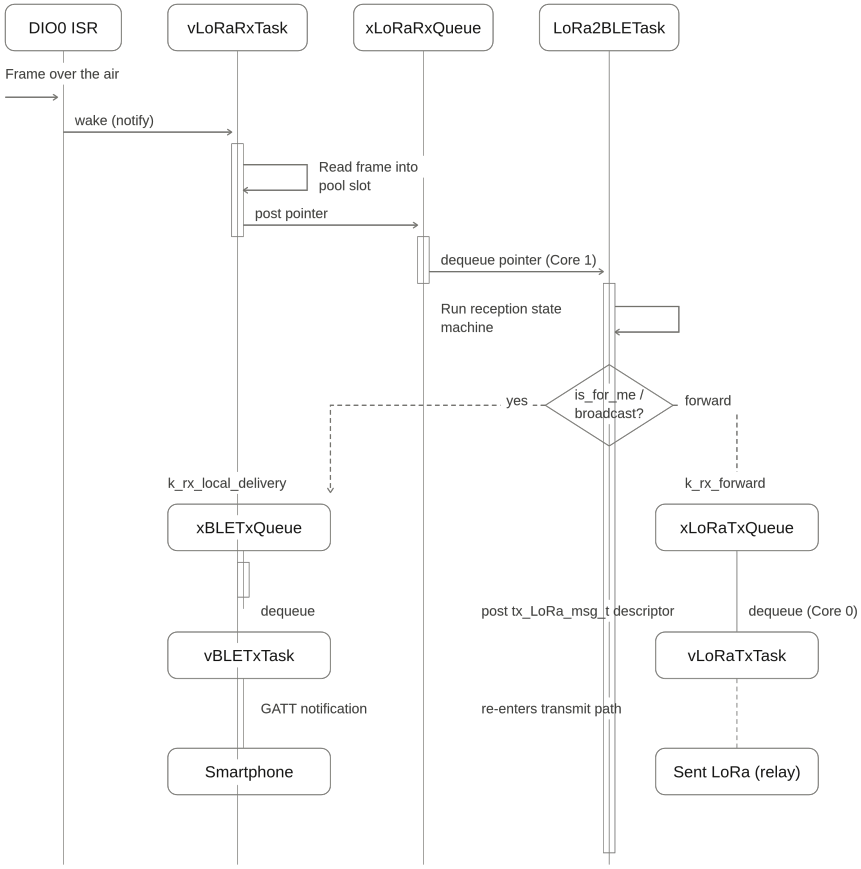


Figure 3.20.: LoRa-to-BLE sequence diagram.

3.5.7. Security Architecture

Encryption Model

All encrypted payloads use AES-256-GCM as specified in NIST SP 800-38D [Dwo07] and implemented through the mbedTLS library (`mbedtls_gcm_*` functions), which on the ESP32 maps to the hardware AES accelerator [Esp23a]. AES-256-GCM is an authenticated encryption with associated data (AEAD) scheme [McG08] providing confidentiality and message authentication in a single pass, without a separate HMAC step. Keys are 256 bits (32 bytes); nonces are

96 bits (12 bytes); authentication tags are 128 bits (16 bytes). Decryption that produces a tag mismatch returns a `CryptoError::AUTH_FAIL` and the plaintext is discarded.

Three encryption scopes are supported. **Node-to-node encryption** (`k_enc_node2node`) uses a symmetric key pre-shared between a specific pair of nodes, providing end-to-end confidentiality for unicast conversations. **Group encryption** (`k_enc_group`) uses a single key distributed to a cohort of nodes, enabling private multicast without sending separate unicasts per recipient. **Network encryption** (`k_enc_network`) uses a mesh-wide shared key as a baseline confidentiality layer against passive radio observers. Unencrypted messaging (`k_data_plain`) is preserved to ensure interoperability when key provisioning has not been completed.

Nonce Construction and Replay Protection

Nonce uniqueness is guaranteed by the `NonceManager` class. Each 96-bit nonce is composed of the 32-bit sender node ID concatenated with a 64-bit monotonically increasing counter, as recommended in RFC 5116 [McG08]:

$$\text{nonce}_{96} = [\,\text{node_id}_{32} \parallel \text{counter}_{64}\,]$$

The node-ID prefix ensures that two senders sharing a group or network key can never produce the same nonce even if their per-sender counters momentarily coincide. The 64-bit counter is transmitted in-band as part of the encrypted payload, allowing the receiver to reconstruct the exact nonce without an additional header field. The counter is stored as a `std::atomic<uint64_t>` and incremented atomically before each encryption call.

Replay attacks are prevented by the `ReplayProtection` class, which maintains per-sender state for up to 16 simultaneous senders. For each sender, it tracks the highest counter value seen and accepts a new packet only if its counter is strictly greater than the recorded maximum, or falls within a backward window of 64 positions to accommodate legitimate out-of-order delivery. Counters falling below the window floor are unconditionally rejected as replays.

Key Management

Key material is provisioned through two complementary mechanisms. The AT command interface allows the companion application to push keys directly over the authenticated BLE connection (`AT+SETPEERKEY=`, `AT+SETGRPKEY=`, `AT+SETNETKEY=`), storing them in the `CryptoManager` singleton. Alternatively, `AT+SENDKX=⟨dst⟩,⟨key⟩` constructs a `k_key_exchange` frame and delivers it over the mesh, enabling one node to provision a peer key on a remote device without a direct BLE connection to that device; the recipient's companion application receives a `+KX=` notification and confirms the key via `AT+SETPEERKEY=`. All keys reside in volatile memory only: `AT+CLRKEYS` and any power cycle erase the entire key store, requiring re-provisioning on the next boot.

3.6. Mobile Application Development

3.6.1. Application Objectives

Four principal objectives shaped the design of the application. First, it must establish and maintain a reliable BLE connection with the paired A3MESH node, transparently recovering from transient disconnections. Second, it must translate user actions—composing a message, opening a conversation, and managing contacts—into the AT commands defined by the

firmware interface described in Section 3.5.6, and dispatch incoming AT notifications to the relevant conversation. Third, it must manage the session cryptographic material required for end-to-end encryption, performing ECDH key exchange and daily network key derivation on behalf of the user without requiring any manual intervention. Fourth, it must persist all conversations and contact entries locally in a structured database so that message history remains accessible regardless of BLE connectivity status.

3.6.2. User Experience Design

The interaction model adopted for A3MESH deliberately mirrors the conventions of mainstream instant messaging applications, a design decision that flows directly from the accessibility requirement discussed in Section 3.1.4. Hence, users are presented with a conversation list on launch, from which they can open individual chat threads or initiate a new exchange. Within a thread, messages are rendered as asymmetric chat bubbles—outgoing messages aligned to the right with a filled blue background, and incoming messages aligned to the left with a neutral surface colour—a layout that reproduces the visual grammar established by widely adopted messaging platforms and is immediately recognisable without instruction.

The visual framework follows the Material Design 3 specification [Goo24b], which provides a coherent set of colour tokens, typographic scales, and component behaviours optimised for Android. The primary structural element is a floating, pill-shaped bottom navigation bar hosting four destinations: *Chats*, *Contacts*, *Device*, and *Settings*. The pill form is implemented through a custom drawable with 28 dp corner radius, giving the element a visually distinct character consistent with the rounded aesthetic introduced in Material 3. The bar floats over the content layer with a 24 dp bottom margin; all scrollable lists include a matching bottom padding to prevent content from being obscured behind it.

The four-tab architecture maps cleanly to the user's mental model: the Chats tab addresses communication, the Contacts tab manages the address book, the Device tab surfaces connectivity status, and the Settings tab exposes the small subset of radio parameters and other aesthetic features that a user might legitimately wish to adjust.

3.6.3. Android Communication Architecture

The application follows the Model–View–ViewModel (MVVM) architectural pattern recommended by Google's Android Architecture Guidelines [Goo24a], in which UI components observe reactive data streams rather than pulling state imperatively. This choice was motivated by two properties of the BLE communication domain: incoming events arrive asynchronously on hardware callback threads, not on the Android main thread; and multiple UI components must react to the same events without being coupled to one another.

The central component is `BleViewModel`, an `AndroidViewModel` subclass scoped to the `MainActivity` lifecycle and shared across all fragments via the `activityViewModels()` delegate. `BleViewModel` exposes the application's observable state through `LiveData` objects: the currently connected `BluetoothDevice`, a human-readable connection status string, an event stream of newly received messages, the node's self-reported device information, and the list of discovered neighbouring nodes. Mutations to this state are posted from whichever thread detects them, and the Android framework guarantees that all observers are notified on the main thread, eliminating the need for explicit thread-marshalling in the UI layer.

Persistent state—messages and contacts—is managed through a Room database, the official

Android SQLite abstraction layer [Goo24c] where two entities are defined: `Contact`, keyed by the 32-bit integer LoRa node address, and `Message`, which records the remote address, textual content, timestamp, direction, an optional sender-address field for public-channel messages, and a read/unread flag. Messages are stored in decrypted form; the firmware performs AES-256-GCM decryption before forwarding plaintext over BLE as described in Section 3.5.7, so the application never handles ciphertext directly. Both DAOs return `LiveData`-wrapped query results, meaning that the conversation list and chat bubbles update reactively whenever the database changes, without any polling.

Furthermore, the BLE connection is maintained across the full Android activity lifecycle through a foreground `Service`—`BleConnectionService`—started as soon as a successful GATT connection is established.

Key management. The application hosts a layered key management subsystem that operates transparently to the user and provisions cryptographic material to the node firmware on demand. Three classes of key are managed. The first is the *network-wide daily key*, a 32-byte AES-256 secret derived deterministically from a fixed network seed and the current calendar date using HKDF-SHA256 [KE10]:

$$K_{\text{net}} = \text{HKDF-SHA256}(\text{seed}, \varnothing, \texttt{"A3MESH-NET-DAILY-v1"} \parallel \text{date}, 32)\,. \tag{3.8}$$

Because the derivation is deterministic and the seed is embedded in every node's firmware, all nodes in the network share the same daily key without any out-of-band coordination; the key rotates automatically at midnight, a transition scheduled by `BleViewModel` via a coroutine that sleeps until the next midnight boundary.

The second class is *peer-to-peer session keys*, one per remote node, derived from an ECDH handshake over the NIST P-256 curve (secp256r1) [Bar+18]. The handshake is initiated automatically whenever the application sends a first message to a node for which no session key exists, or when the firmware notifies the application of a newly discovered neighbour via a `+DISC=` unsolicited response. Both parties generate ephemeral P-256 key pairs, exchange the 64-byte raw public keys (X and Y coordinates concatenated, without DER encoding overhead) over the mesh via `AT+SENDKX` commands, and independently derive the session key as:

$$K_{\text{peer}} = \text{HKDF-SHA256}(\text{ECDH shared secret}, \varnothing, \texttt{"A3MESH-N2N-v1"}, 32)\,. \tag{3.9}$$

The resulting key is provisioned to the firmware via `AT+SETPEERKEY`, after which all subsequent messages to that peer are encrypted at the firmware level before transmission.

The third class is *group keys*, 32-byte secrets generated by a group creator using a cryptographically secure random number generator and distributed out-of-band via a QR code.

All keys are persisted in hexadecimal form through the `A3MeshKeyStore` class, which wraps `SharedPreferences`, and the full key set is re-provisioned to the firmware on every reconnection to survive firmware resets.

3.6.4. BLE Integration

The BLE communication layer is encapsulated in the `ConnectionManager` singleton, which owns the GATT connection lifecycle and serialises all operations through a `ConcurrentLinkedQueue`. This serialization is necessary because the Android BLE stack is not reentrant: issuing a second GATT operation before the callback for the previous one has

fired results in silent failure. The queue ensures that only one operation—connect, read, write, MTU negotiation, or notification subscription—is in flight at any given time, and that the next operation is dequeued only after `signalEndOfOperation()` is called from the relevant GATT callback. Consumers register `ConnectionEventListener` instances, stored via `WeakReference` to avoid leaking Activity contexts, and receive callbacks for connection completion, disconnection, and characteristic changes.

Upon successful GATT connection, `MainActivity` requests a maximum transmission unit (MTU) of 256 bytes to accommodate AT commands whose payloads may approach the BLE PDU size, then subscribes to notifications on the Nordic UART Service (NUS) RX characteristic [Nor23] (`UUID 6E400003`), which the firmware uses to push unsolicited responses. Commands are written to the NUS TX characteristic (`UUID 6E400002`) as UTF-8-encoded byte arrays terminated with `\r\n`, following the ITU-T V.250 convention adopted by the firmware command parser [Int03].

Incoming notifications are dispatched in `MainActivity`'s `onCharacteristicChanged` callback according to a set of unsolicited response codes defined by the firmware AT interface. Each code triggers the corresponding handler in `BleViewModel`, which updates the Room database, posts the appropriate `LiveData` event, or initiates a key exchange, depending on the type of notification.

3.6.5. Messaging Interface

The messaging interface is organised around two principal screens. `ChatsFragment` presents the conversation list as a `RecyclerView` powered by `ChatsAdapter`, which uses `DiffUtil` to compute item differences and animate insertions without redrawing the entire list. Each row displays the contact name or hexadecimal node address, the timestamp of the most recent message, and an unread message badge. A pinned entry at the top of the list represents the public broadcast channel, which is permanently visible. New conversations are initiated through a speed-dial floating action button that expands into three options: direct message to a specific node address, create a new group, and join an existing group.

`ChatActivity` hosts the active chat window. When the user submits a message, `BleViewModel.sendMessage()` inspects the key store and selects the appropriate AT command: `AT+SENDNET` for the public channel, `AT+SENDGRP` for a group conversation with a provisioned group key, `AT+SENDE2E` for a peer with an established session key, and `AT+SEND` for an unkeyed initial exchange, simultaneously triggering a background key exchange to establish encryption for subsequent messages. Outgoing messages are written to the Room database immediately and displayed in the bubble view, providing perceived instant feedback regardless of mesh propagation delay.

Group management relies on QR-code-based key distribution implemented with the ZXing library [ZXi24]. When a user creates a group, the application generates a random 32-byte AES-256 group key, encodes it together with the 32-bit group identifier and group name into a compact string of the form `a3g:<groupId>:<keyHex>:<name>`, and renders this string as a QR code for distribution. Members wishing to join scan the code using ZXing's `ScanContract` Activity Result API, whereupon the application extracts the group key, provisions it to the firmware via `AT+SETGRPKEY`, and opens the corresponding group chat.

3.6.6. Contact and Node Management

The Contacts subsystem bridges two abstractions: the 32-bit integer LoRa node addresses used by the firmware and the human-readable names with which users identify their peers. `ContactsFragment` allows the user to assign a name to any node address, modify existing entries, and delete contacts. Contacts are resolved against message records at display time; if no contact entry exists for a given address, the application falls back to the raw hexadecimal representation (e.g., `0xAABBCCDD`).

`DeviceFragment` provides visibility into the paired node's operational state. On connection, the application queries the node by issuing `AT+INFO?`, and the response is parsed and displayed. A separate `AT+NEIGHBORS?` query retrieves the current neighbour table, listing the identifiers, received signal strength, and last-heard timestamps of all nodes within direct radio range of the paired hardware. This list is also used proactively: on connection establishment and on each `+DISC=` notification, the application automatically initiates ECDH key exchange with newly discovered peers, so that end-to-end encrypted communication is available as soon as a neighbour enters range, without any user action.

3.6.7. Offline Interaction Model

A deliberate consequence of the architectural choices described above is that the application remains functional in a meaningful sense even when no A3MESH node is paired. The Room database retains the full conversation history and contact book, which are accessible and browsable at all times. Attempting to send a message with no BLE device connected is detected at the `ConnectionManager` level and results in a Material alert dialog prompting the user to establish a connection before proceeding. This clear feedback, rather than silent failure, is consistent with the accessible design philosophy that underpins the platform.

The foreground service ensures that BLE connectivity persists when the application moves to the background—a practical requirement in emergency scenarios where the device may be pocketed while the user is engaged in another activity. Should the BLE link be interrupted, the application displays an alert dialog on next resumption and the foreground service terminates itself cleanly.

3.6.8. Development Strategy

The mobile application is essential for validating the platform end-to-end but is not itself a research contribution of this thesis. To keep the effort proportionate to its role, the application was developed with the assistance of Claude Code [Dak+23], using an open-source BLE starter project as a scaffold [Pun24]. The structural layers—the Room schema, MVVM classes, Navigation graph, and RecyclerView adapters—were generated interactively, with my role limited to architecture and review. The cryptographic subsystem, by contrast, was specified at algorithm level and verified line by line against RFC 5869 and NIST SP 800-56A, because errors in key size, encoding, or error propagation are syntactically valid yet semantically incorrect and are not reliably flagged by any tool.

This approach compressed the total application effort to roughly two weeks of part-time work, against the two months invested in the simulator, the Smart Gossip algorithm, and the custom hardware. The result is functionally complete for the purposes of this thesis: it enables end-to-end field testing, satisfies the accessibility requirement of Section 3.1.4, and demonstrates that the firmware AT interface is expressive enough to support a fully featured

messaging application. Extensions such as iOS support, push notifications, and delivery indicators are deferred to future work.

3.7. Experimental Results and Evaluation

3.7.1. Test Methodology

The experimental evaluation was structured around three independent test campaigns, each targeting a distinct performance dimension of the platform. The first campaign measured the maximum single-hop radio range under near-ideal propagation conditions. The second validated the mesh relay capability in an urban environment, confirming that multi-hop forwarding compensates for obstructions that sever direct links. The third characterised the operational battery lifetime of a fully assembled node.

3.7.2. Range Evaluation

The range test was conducted along Avinguda de Madrid in Òdena (Igualada, Barcelona), a quasi-linear road in the Pla de les Gavarreres area that provides an extended, largely unobstructed corridor suited to single-hop range characterisation. To approach the theoretical ceiling of the SX1276 transceiver, the radio was configured with spreading factor 12 and bandwidth 125 kHz (SF12/BW125), the most link-budget-efficient combination available, achieving a receiver sensitivity of −138 dBm at the cost of reduced data rate.

Node A was stationed at a fixed reference point beside a parked lorry, as shown in Figure 3.21. Node B was then carried progressively further along the road while receiving periodic packets from Node A. Despite a moderate undulation along the corridor—visible in the elevation profile inset of Figure 3.21—a continuous line of sight was maintained throughout the test, making the experiment representative of open-terrain propagation conditions.

Packet delivery remained reliable at 100 % PDR up to approximately 1 000 m. Beyond that threshold, losses became noticeable: in the 1 000–1 200 m range, roughly two out of five packets were lost, corresponding to a PDR of approximately 60 %. The test was concluded at 1 200 m, at which point the link had not yet failed completely. This result should be understood as a lower bound on the maximum achievable range rather than its definitive limit: as is characteristic of radiofrequency propagation, a cleaner line of sight—obtainable from an elevated vantage point or over perfectly flat terrain—would be expected to extend the usable range further still.

3.7.3. Mesh Relay Demonstration

The mesh relay capability was validated through two urban deployment scenarios conducted in Vilanova del Camí. In each scenario, three nodes—referred to as Node A, Node B, and Node C—were placed such that a direct radio link between Node A and Node C was obstructed, while Node B maintained adequate signal margins to both endpoints.

Unobstructed-Street Scenario

In the first scenario, the three nodes were placed at intervals of approximately 100 m along a straight street. Figure 3.22 illustrates the placement.

In order to make the result more evident, the SF was reduced to its minimum value SF7 and the bandwidth was set to 250 kHz. Additionally, it was then verified through the application

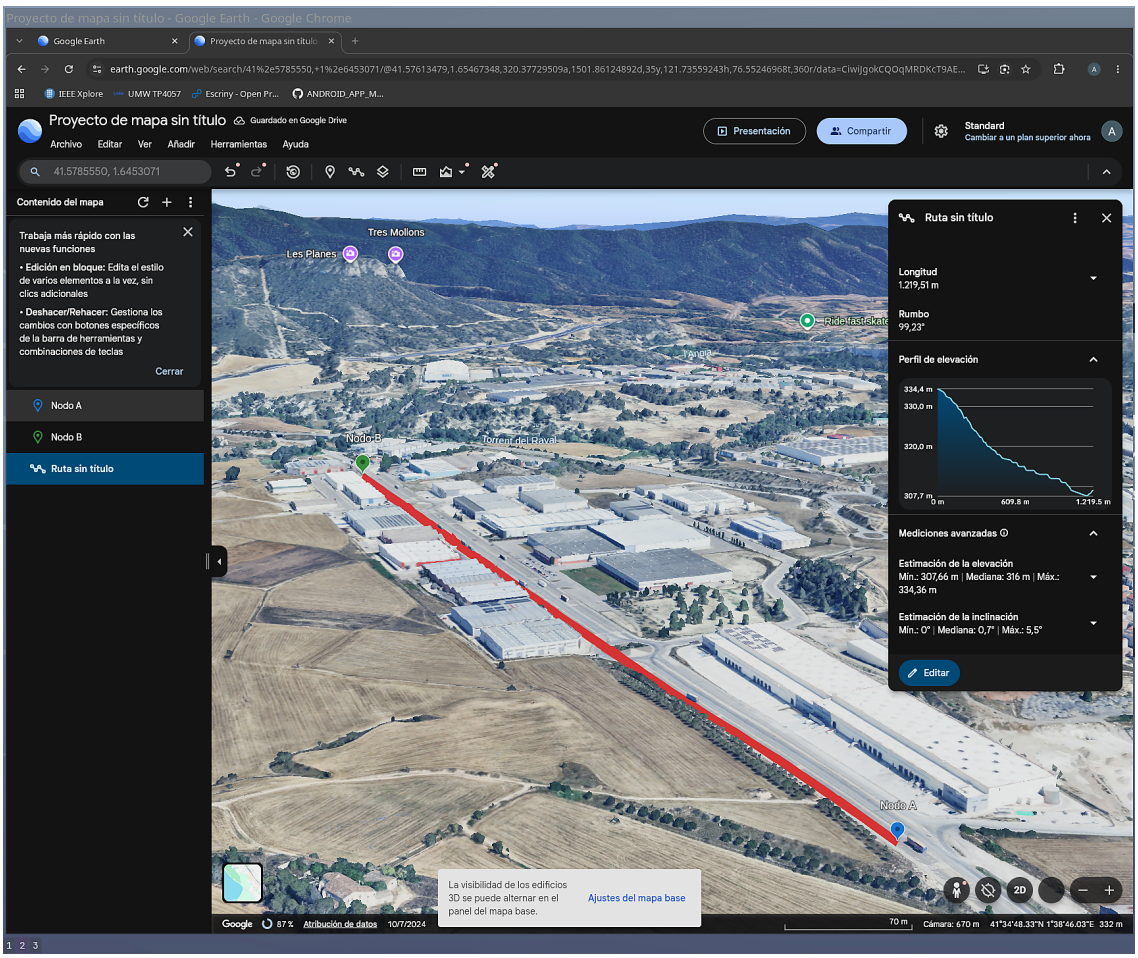


Figure 3.21.: Range test on Avinguda de Madrid, Òdena. Node A is positioned beside the parked lorry in the foreground; the elevation profile inset confirms a largely continuous line of sight along the test corridor despite moderate road undulation.

that the only neighboring node displayed for both Nodes A and C was Node B, confirming that communication between the two nodes was successfully established through the intermediate relay node.

Building-Obstruction Scenario

In the second scenario, each node was positioned on a different side of a different building block, such that no two nodes shared a direct line of sight, Figure 3.23 shows the arrangement.

Despite the more challenging propagation geometry, the mesh successfully relayed messages but with a worse link quality than in the unobstructed-street scenario as expected.



Figure 3.22.: Node placement in the unobstructed-street relay scenario. Node A and Node C are positioned on opposite sides of a street, with Node B acting as an intermediate relay.

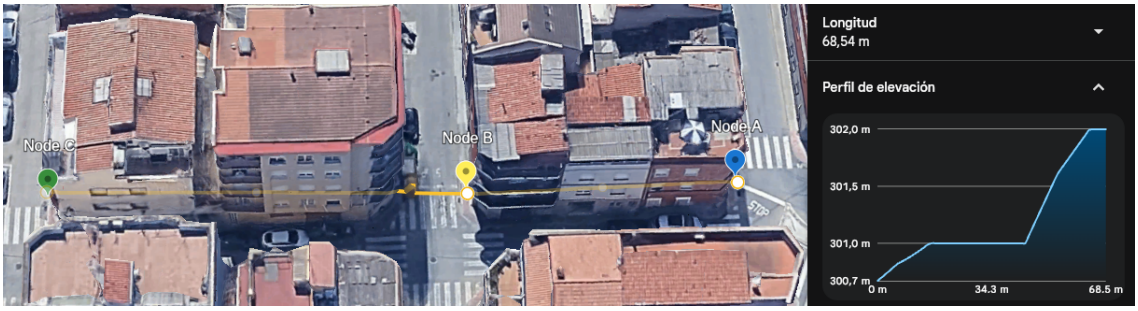


Figure 3.23.: Node placement in the building-obstruction relay scenario. Node A and Node C are positioned on opposite sides of different building blocks separated by 68, 54 m, with Node B acting as an intermediate relay.

3.7.4. Power Consumption Analysis

The node was fully charged to 4.20 V prior to the experiment and left to operate autonomously until the DW01A + FS8205A hardware protection circuit disconnected the cell at approximately 2.4 V. Battery state was sampled periodically via `AT+INFO?` to track the discharge curve reported by the firmware's look-up-table estimator. It is relevant to mention that the battery used for this experiment was not a new cell, but rather a recycled one, thus the final amount of hours of operation would increase with a new cell.

The node remained powered on with the BLE interface active and paired to the companion application, but no messages were sent and no user interaction took place. The node continued to transmit the neighbour-discovery beacon every 30 s as described in Chapter 3.5. The `PowerManager` task reduced the CPU clock from 240 MHz to 80 MHz after the configured 10-minute inactivity timeout, as confirmed by the absence of `notify_activity()` calls during the quiescent period. The node remained operational for *15 hours*.

4. Economic Study

This chapter presents the economic dimension of the project. It is divided in three sections: the labour cost associated with the engineering effort, the bill-of-materials cost of a single prototype unit, and the resulting total project cost. All figures are expressed in euros (EUR) and correspond to prices observed during 2025–2026 through the distributors listed in Appendix C. Volume production economics and non-recurring engineering amortisation are deliberately excluded, as the platform is presented as a research prototype rather than as a commercial product.

4.1. Labour Cost

The engineering work carried out over the duration of this thesis has been valued at the medium to high rate for a junior engineer in Spain, taken as 17 EUR per hour. This rate is representative of a recent graduate performing embedded, hardware, and mobile-software development, and is used here to establish an order of magnitude for the human effort required to reproduce the project.

Total effort was estimated from the Jira ticket record described in Section 3.2 and aggregated by subsystem. Table 4.1 summarises the distribution.

Table 4.1.: Engineering effort distribution and derived labour cost at 17 EUR/hour.

Activity	**Hours**	**Cost (EUR)**
Background study and state-of-the-art review	40	800
Network simulator and Smart Gossip algorithm	120	3 200
Hardware design (schematic, layout, manufacturing)	160	2 400
Firmware architecture and implementation	140	2 800
Android application development	60	1 200
Mechanical design and 3D modelling	20	400
Experimental testing and validation	20	400
Thesis writing and documentation	40	800
Total	**600**	**10 200**

The total engineering effort amounts to approximately 600 hours, consistent with the 24 ECTS workload of a bachelor thesis at 25 hours per credit. The resulting labour cost is 10 200 EUR.

4.2. Materials Cost per Unit

Table 4.2 details the material cost of a single assembled A3MESH node, following the bill of materials given in Appendix C. Unit prices correspond to small orders through LCSC for the

components sourced by JLCPCB, and to AliExpress or Amazon for the two items marked in Appendix C as separately procured.

Table 4.2.: Bill-of-materials cost of a single assembled node in hobbyist quantities. Lines marked † are sourced outside the JLCPCB order and priced as estimates.

Item	Qty	Unit (EUR)	Ext. (EUR)
ESP32-WROOM-32UE module	1	3.42	3.42
RFM95W-868S2 LoRa module†	1	5.00	5.00
CH340C USB-UART bridge	1	0.55	0.55
TP4056 Li-ion charger	1	0.16	0.16
TPS63001 buck-boost converter	1	1.28	1.28
DW01A battery protection IC	1	0.04	0.04
FS8205A dual N-MOSFET	1	0.06	0.06
DMP1045UQ-7 P-MOSFET	1	0.16	0.16
SS34 Schottky diode	1	0.03	0.03
Polyfuse 500 mA (1812)	1	0.09	0.09
Power inductor 2.2 µH	1	0.03	0.03
Status LEDs (red + green)	2	<0.01	0.02
SS8050 NPN LED driver	2	<0.01	0.01
Tactile pushbutton switch	2	0.06	0.13
USB-C receptacle (GCT USB4105)	1	0.06	0.06
JST-PH 2-pin battery connector†	1	0.10	0.10
Ceramic capacitors (0402/0603/1206)	14	<0.01	0.12
Chip resistors (0402/0603)	16	<0.01	0.02
Subtotal, active and passive components			11.28
PCB fabrication (2-layer, 74.5×89.8 mm)	1	2.00	2.00
SMD assembly service (JLCPCB)	1	4.00	4.00
Li-ion pouch cell (1 200 mAh, 3.7 V)	1	5.00	5.00
3D-printed TPU enclosure	1	2.50	2.50
Total per unit			**24.78**

The resulting per-unit cost is approximately 25 EUR. The dominant contributions are the RFM95W LoRa module, the JLCPCB assembly service, and the battery, which together account for over half of the total. Sourcing the LoRa module in volume, or replacing it with a bare SX1276 chip integrated on the same board, would reduce this figure meaningfully.

4.3. Total Project Cost

Combining the labour cost of Section 4.1 with the material cost of a small production run of five prototype units—the actual number of boards manufactured during this thesis for use in the experimental campaigns of Section 3.7.3—the total project cost is summarised in Table 4.3.

Engineering effort dominates the balance by two orders of magnitude, as is typical for research prototypes: the recurring cost of an additional unit is approximately 25 EUR, whereas the human cost of developing the platform in the first place is close to 10 200 EUR. From a pro-

Table 4.3.: Aggregate project cost, including engineering effort and prototype hardware.

Concept	**Cost (EUR)**
Engineering labour (600 h at 17 EUR/h)	10 200.00
Prototype units (5 nodes at 25.00 EUR each)	125.00
Total project cost	**10 325.00**

ductisation standpoint, this ratio implies that the platform becomes economically interesting only if the engineering investment is amortised over a production volume in the low thousands of units.

5. Conclusions

5.1. Achieved Objectives

Each of the objectives stated in Section 1.2 has been addressed. A custom low-power hardware platform was designed and manufactured on a single 74.5×89.8 mm two-layer board integrating an ESP32-WROOM-32UE microcontroller, an RFM95W LoRa transceiver, a cascaded charging and buck-boost power subsystem, and a compact PCB monopole antenna adapted from Texas Instruments Design Note DN024. On top of it, the layered firmware implements the LoRa and BLE communication pipelines, the Smart Gossip forwarding engine, neighbour discovery, AES-256-GCM encryption with ECDH key exchange, adaptive CPU frequency scaling, and battery monitoring.

The central research question—whether adaptive gossip can replace flooding on a LoRa mesh—was answered through a purpose-built discrete-event simulator, and the answer is affirmative with a clearly quantified trade-off. A dedicated Android application hides all networking complexity behind a conventional messaging interface, and a 3D-printed flexible enclosure bonded to the user's phone case satisfies the portability requirement without any manual attachment step. Finally, real-world experiments confirmed mesh relay operation in two urban obstruction scenarios and established a single-hop range exceeding 1.2 km under the SF12/BW125 configuration.

5.2. Main Contributions

The primary contribution of this thesis is the *Smart Gossip* adaptive forwarding algorithm, which computes, for each received packet, a relay probability from three locally observable signals—received signal strength, neighbour count, and duplicate copies overheard—bounded from below by a configurable floor. Its five parameters were tuned through Differential Evolution on the simulator described in Chapter 3.3, producing a configuration that, relative to managed flooding across networks of 15–50 nodes, reduces per-node transmission air time by 27–32 %, end-to-end delay by 58–86 %, and collision rate by 18–37 %, at the cost of a delivery-ratio penalty of 4.5–12.4 percentage points.

A secondary contribution is the simulator itself: the modified Meshtasticator environment, with a pluggable routing interface and a batch evaluation harness, constitutes a reusable platform for evaluating LoRa mesh forwarding policies under reproducible conditions. Finally, the hardware, firmware, and Android application together demonstrate that a user with no radio knowledge can exchange encrypted messages over a multi-hop LoRa network by pairing an ordinary smartphone with an inexpensive, pocket-sized node, and the AT-command interface between firmware and application defines a stable contract that allows either subsystem to evolve independently.

5.3. Lessons Learned

Several insights emerged from the development process that are worth recording for future work in this domain.

The simulation-first methodology proved indispensable. The five-parameter space of Smart Gossip cannot be explored manually: even a coarse ten-level grid yields 10^5 combinations, each requiring statistical averaging over multiple 30-minute simulations. Differential Evolution converged to a meaningful optimum in approximately 160 fitness evaluations, completing in under ten minutes of computation. The lesson is that adaptive mesh protocols in constrained-radio environments are inherently a joint simulation-and-hardware problem; neither alone is sufficient.

AI-assisted development demonstrated clear but bounded value. The Android application's structural scaffolding—ROOM schema, MVVM layer, Navigation graph, adapter infrastructure—was produced rapidly and correctly. Cryptographic code, however, required line-by-line human review: subtle errors in key size, nonce construction, or error propagation are syntactically valid yet semantically incorrect, and no tool reliably flags them. The lesson is that AI assistance is most effective as a force multiplier for well-understood engineering tasks, not as a substitute for domain expertise on security-critical code.

6. Future Work

Several directions for future work present themselves naturally from the limitations identified throughout this thesis.

The most operationally impactful extension would be the integration of battery-aware network behaviour. The firmware already reports battery state of charge via `AT+INFO?`; a natural next step is to act on it. When a node's remaining capacity falls below a configurable threshold, the firmware could raise the Smart Gossip probability floor toward 1—effectively reverting to full flooding for maximum relay participation—and simultaneously switch to a more robust radio configuration such as SF12/BW125. This dual response would ensure that a node nearing exhaustion contributes maximally to the mesh for its remaining operational lifetime rather than silently degrading, and could additionally generate an unsolicited SOS notification to paired smartphones with the node's identity and last-known battery level, a lightweight emergency signalling mechanism that requires no additional hardware.

A second line of improvement concerns the physical form factor. The current node thickness is dominated by the LiPo cell rather than the PCB itself; replacing the standard pouch cell with a thinner, purpose-sized flat cell of equivalent capacity would significantly reduce the overall profile of the assembled accessory. Alternatively, a revised PCB layout that places the tallest components—the module shields, the buck-boost inductor, and the USB-C connector—on opposite faces of the board would allow the cell to be recessed into a relief pocket in the enclosure, reducing the combined stack height without compromising capacity. Either approach would improve wearability and reduce the perceived intrusiveness of the device.

On the software side, the companion application currently targets Android exclusively. Extending it to iOS would substantially broaden the platform's reach, particularly in emergency response communities in regions where iOS market share is high. The firmware AT interface is already platform-agnostic; the principal effort would be the reimplementation of the BLE communication layer and the cryptographic subsystem using the iOS Core Bluetooth and CommonCrypto frameworks, which offer comparable capabilities to their Android counterparts. A shared cross-platform logic layer implemented in Kotlin Multiplatform or similar technology could reduce the duplication of business logic between the two applications.

Furthermore, the Smart Gossip parameters are currently fixed constants compiled into the firmware. A natural evolution would be an online adaptation mechanism that adjusts β and p_{floor} at runtime in response to observed network conditions—raising the density weight when the neighbour count grows and relaxing the floor when the duplicate rate indicates good coverage. Such closed-loop adaptation would remove the need for pre-optimised parameters altogether and make the algorithm self-tuning across the full range of network densities, including the small-network regime where the current static parameterisation underperforms managed flooding.

Finally, a further extension of considerable practical value would be the introduction of an internet gateway role for nodes that happen to have connectivity—through Wi-Fi, Ethernet, or mobile data—at a given moment. In the current design every node is equal; a gateway-capable node would act as a transparent bridge between the LoRa mesh and the wider internet, re-

laying outbound messages to an MQTT broker, a REST API, or a cloud service, and injecting inbound messages from those services back into the mesh. This bidirectional bridge opens several immediately practical scenarios: NTP-based time synchronisation distributed across the mesh without each node requiring its own internet link; distribution of firmware configuration updates or new group encryption keys without physical access to individual units; and outbound emergency notifications—triggered by an SOS event, a low-battery alert, or an attached sensor threshold—delivered as push notifications or e-mail to registered contacts even when the originating node has no direct connectivity. When nodes carry environmental sensors, the gateway could additionally forward periodic telemetry—temperature, humidity, air quality, or network statistics such as RSSI, SNR, and PDR—to platforms such as Grafana or Home Assistant, enabling remote monitoring and long-term performance analysis of geographically dispersed deployments. The gateway role could be implemented as a compile-time configuration option that activates an additional FreeRTOS task alongside the existing LoRa and BLE tasks, requiring no changes to the core firmware architecture and leaving unmodified nodes fully interoperable with gateway-enabled ones.

Bibliography

[23] *ESP-IDF Programming Guide, v5.x*. Accessed: June 2026. Espressif Systems. 2023. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html.

[24] *FreeRTOS Reference Manual: API Functions and Configuration Options*. Accessed: June 2026. Amazon Web Services, Inc. 2024. URL: https://www.freertos.org/Documentation/RTOS_book.html.

[Ade+17] Ferran Adelantado et al. "Understanding the Limits of LoRaWAN". In: *IEEE Communications Magazine* 55.9 (2017), pp. 34–40. DOI: 10.1109/MCOM.2017.1600613. URL: https://ieeexplore.ieee.org/document/8030482.

[Ála+22] José Álamos et al. "DSME-LoRa: Seamless Long Range Communication Between Arbitrary Nodes in the Constrained IoT". In: *arXiv preprint arXiv:2206.14077* (2022).

[Aug+16] Aloys Augustin et al. "A Study of LoRa: Long Range & Low Power Networks for the Internet of Things". In: *Sensors* 16.9 (2016), p. 1466. DOI: 10.3390/s16091466. URL: https://www.mdpi.com/1424-8220/16/9/1466.

[AWW05] Ian F. Akyildiz, Xudong Wang, and Weilin Wang. "Wireless Mesh Networks: A Survey". In: *Computer Networks* 47.4 (2005), pp. 445–487.

[Bar+18] Elaine Barker et al. *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*. Special Publication 800-56A Rev. 3. Accessed: June 2026. National Institute of Standards and Technology, 2018. DOI: 10.6028/NIST.SP.800-56Ar3. URL: https://doi.org/10.6028/NIST.SP.800-56Ar3.

[Blu23] Bluetooth Special Interest Group. *Bluetooth Core Specification, Version 5.4*. Tech. rep. Bluetooth SIG, 2023. URL: https://www.bluetooth.com/specifications/specs/core-specification-5-4/.

[Bor+16] Martin C. Bor et al. "Do LoRa Low-Power Wide-Area Networks Scale?" In: *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*. 2016, pp. 59–67. DOI: 10.1145/2988287.2989163. URL: https://dl.acm.org/doi/10.1145/2988287.2989163.

[CBR17] Marco Cattani, Carlo Alberto Boano, and Kay Römer. "An Experimental Evaluation of the Reliability of LoRa Long-Range Low-Power Wireless Communication". In: *Journal of Sensor and Actuator Networks* 6.2 (2017), p. 7. DOI: 10.3390/jsan6020007. URL: https://www.mdpi.com/2224-2708/6/2/7.

[Cro+18] Daniele Croce et al. “Impact of LoRa Imperfect Orthogonality: Analysis of Link-Level Performance”. In: *IEEE Communications Letters* 22.4 (2018), pp. 796–799. DOI: 10.1109/LCOMM.2018.2797057. URL: https://ieeexplore.ieee.org/document/8267219.

[CS14] Scott Chacon and Ben Straub. *Pro Git*. 2nd. Freely available online. Accessed: June 2026. Apress, 2014. DOI: 10.1007/978-1-4842-0076-6. URL: https://git-scm.com/book/en/v2.

[Dak+23] Arghavan Moradi Dakhel et al. “GitHub Copilot AI Pair Programmer: Asset or Liability?” In: *Journal of Systems and Software* 203 (2023), p. 111734. DOI: 10.1016/j.jss.2023.111734. URL: https://doi.org/10.1016/j.jss.2023.111734.

[DS11] Swagatam Das and Ponnuthurai Nagaratnam Suganthan. “Differential Evolution: A Survey of the State-of-the-Art”. In: *IEEE Transactions on Evolutionary Computation* 15.1 (2011), pp. 4–31. DOI: 10.1109/TEVC.2010.2059031. URL: https://ieeexplore.ieee.org/document/5601760.

[Dwo07] Morris Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. Tech. rep. SP 800-38D. National Institute of Standards and Technology, 2007. DOI: 10.6028/NIST.SP.800-38D.

[EH21] D. Eastlake and T. Hansen. *The FNV Non-Cryptographic Hash Algorithm*. Internet-Draft draft-eastlake-fnv-21. Informational. Accessed: June 2026. Internet Engineering Task Force, 2021. URL: https://datatracker.ietf.org/doc/draft-eastlake-fnv/.

[ES03] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Berlin, Heidelberg: Springer, 2003. DOI: 10.1007/978-3-662-05094-1.

[Esp23a] Espressif Systems. *ESP32 Technical Reference Manual, v5.3*. Technical Reference Manual. Accessed: June 2026. Espressif Systems Co., Ltd., 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.

[Esp23b] Espressif Systems. *ESP32-WROOM-32E & ESP32-WROOM-32UE Datasheet*. Datasheet. Accessed: June 2026. Espressif Systems Co., Ltd., 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf.

[Eur20] European Telecommunications Standards Institute. *ETSI EN 300 220-1 V3.1.1: Short Range Devices (SRD) Operating in the Frequency Range 25 MHz to 1000 MHz; Part 1: Technical Characteristics and Methods of Measurement*. European Standard EN 300 220-1 V3.1.1. Accessed: June 2026. ETSI, 2020. URL: https://www.etsi.org/deliver/etsi_en/300200_300299/30022001/03.01.01_60/en_30022001v030101p.pdf.

[Goo24a] Google LLC. *Guide to App Architecture*. Android Developers Documentation. Accessed: June 2026. 2024. URL: https://developer.android.com/topic/architecture.

[Goo24b] Google LLC. *Material Design 3*. Google Design Documentation. Accessed: June 2026. 2024. URL: https://m3.material.io.

[Goo24c] Google LLC. *Save Data in a Local Database Using Room*. Android Developers Documentation. Accessed: June 2026. 2024. URL: https://developer.android.com/training/data-storage/room.

[GOP12] Carles Gomez, Joaquim Oller, and Josep Paradells. "Overview and Evaluation of Bluetooth Low Energy: An Emerging Low-Power Wireless Technology". In: *Sensors* 12.9 (2012), pp. 11734–11753. DOI: 10.3390/s120911734. URL: https://www.mdpi.com/1424-8220/12/9/11734.

[HHL06] Zygmunt J. Haas, Joseph Y. Halpern, and Li Li. "Gossip-Based Ad Hoc Routing". In: *IEEE/ACM Transactions on Networking* 14.3 (2006), pp. 479–491. DOI: 10.1109/TNET.2006.876186. URL: https://ieeexplore.ieee.org/document/1644582.

[Hop18] Hope Microelectronics. *RFM95W LoRa Long Range Transceiver Module Datasheet*. Datasheet. Accessed: June 2026. Hope Microelectronics Co., Ltd., 2018. URL: https://cdn.sparkfun.com/assets/learn_tutorials/8/0/4/RFM95_96_97_98W.pdf.

[Int03] International Telecommunication Union. *ITU-T Recommendation V.250: Serial asynchronous automatic dialling and control*. Tech. rep. V.250. Accessed: June 2026. ITU-T, 2003. URL: https://www.itu.int/rec/T-REC-V.250/en.

[Jia23] Jiangsu Qin Heng. *CH340C USB Bus Convert Chip Datasheet*. Datasheet. Accessed: June 2026. Jiangsu Qin Heng Co., Ltd., 2023. URL: https://datasheet.lcsc.com/szlcsc/Jiangsu-Qin-Heng-CH340C_C84681.pdf.

[KCG06] Pradeep Kyasanur, Romit Roy Choudhury, and Indranil Gupta. "Smart Gossip: An Adaptive Gossip-based Broadcasting Service for Sensor Networks". In: *Proceedings of the IEEE International Conference on Mobile Ad Hoc and Sensor Systems (MASS)*. 2006, pp. 91–100. DOI: 10.1109/MOBHOC.2006.278671. URL: https://ieeexplore.ieee.org/document/4053909.

[KE10] Hugo Krawczyk and Pasi Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. Request for Comments 5869. Accessed: June 2026. Internet Engineering Task Force, May 2010. DOI: 10.17487/RFC5869. URL: https://www.rfc-editor.org/rfc/rfc5869.

[Loc26] LocalMesh. *LocalMesh: The Emergency Radio Network*. https://www.localmesh.nl/en/. Accessed: June 2026. 2026.

[LoR17] LoRa Alliance. *LoRaWAN 1.1 Specification*. Tech. rep. LoRa Alliance Technical Committee, 2017. URL: https://lora-alliance.org/resource_hub/lorawan-specification-v1-1/.

[McG08] D. McGrew. *An Interface and Algorithms for Authenticated Encryption*. RFC 5116. Accessed: June 2026. Internet Engineering Task Force, 2008. DOI: 10.17487/RFC5116. URL: https://www.rfc-editor.org/rfc/rfc5116.

[Mek+19] Kais Mekki et al. "A comparative study of LPWAN technologies for large-scale IoT deployment". In: *ICT Express* 5.1 (2019), pp. 1–7. DOI: 10.1016/j.icte.2017.12.005. URL: https://www.sciencedirect.com/science/article/pii/S2405959517302953.

[Mes24a] Meshtastic Project. *Meshtastic Documentation: Introduction*. https://meshtastic.org/docs/introduction/. Accessed: June 2026. 2024.

[Mes24b] Meshtastic Project. *Meshtastic Documentation: Mesh Broadcast Algorithm*. https://meshtastic.org/docs/overview/mesh-algo/. Accessed: June 2026. 2024.

[Mes24c] Meshtastic Project. *Meshtasticator: Discrete-Event and Interactive Simulator for Meshtastic*. https://github.com/meshtastic/meshtasticator. Accessed: June 2026. 2024.

[Mes25] MeshCore Project. *MeshCore: A Lightweight, Portable Mesh Routing Protocol for LoRa*. https://github.com/ripplebiz/MeshCore. Accessed: June 2026. 2025.

[Mes26] Meshtastic Project. *Meshtastic Documentation*. Accessed: 2026-06-06. 2026. URL: https://meshtastic.org/.

[MM04] C. Siva Ram Murthy and B. S. Manoj. *Ad Hoc Wireless Networks: Architectures and Protocols*. Prentice Hall, 2004.

[Nan23] Nanjing Extension Microelectronics. *TP4056-42 1 A Standalone Linear Li-ion Battery Charger with Thermal Regulation, ESOP-8 Datasheet*. Datasheet. Accessed: June 2026. Nanjing Extension Microelectronics Co., Ltd., 2023. URL: https://www.lcsc.com/datasheet/lcsc_datasheet_2410121619_TOPPOWER-Nanjing-Extension-Microelectronics-TP4056-42-ESOP8_C16581.pdf.

[Ni+99] Sze-Yao Ni et al. "The Broadcast Storm Problem in a Mobile Ad Hoc Network". In: *Proceedings of the 5th Annual ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom)*. 1999, pp. 151–162. DOI: 10.1145/313451.313525. URL: https://dl.acm.org/doi/10.1145/313451.313525.

[Nor23] Nordic Semiconductor ASA. *UART Service (NUS)*. Nordic Developer Zone Documentation. Accessed: June 2026. 2023. URL: https://developer.nordicsemi.com/nRF_Connect_SDK/doc/latest/nrf/libraries/bluetooth_services/services/nus.html.

[PSL05] Kenneth V. Price, Rainer M. Storn, and Jouni A. Lampinen. *Differential Evolution: A Practical Approach to Global Optimization*. Springer, 2005. DOI: 10.1007/3-540-31306-0. URL: https://link.springer.com/book/10.1007/3-540-31306-0.

[Pue+21] Roger Pueyo Centelles et al. "Beyond the Star of Stars: An Introduction to Multihop and Mesh for LoRa and LoRaWAN". In: *IEEE Pervasive Computing* 20.2 (2021), pp. 63–72. DOI: 10.1109/MPRV.2021.3063443. URL: https://ieeexplore.ieee.org/document/9395365.

[Pun24] Punch Through Design LLC. *BLE Starter for Android*. GitHub Repository. Accessed: June 2026. 2024. URL: https://github.com/PunchThrough/ble-starter-android.

[Reu26] Reuters. *Viasat and Cubic³ Demonstrate Satellite Voice Calls for Vehicles at MWC 2026*. Accessed: 2026-06-06. 2026. URL: https://www.investing.com/news/company-news/viasat-cubic-demo-satellite-voice-calls-for-vehicles-at-mwc-93CH-4525448.

[RF 24] RF Wireless World. *Chip vs Chirp in LoRaWAN*. https://www.rfwireless-world.com/terminology/chip-vs-chirp-lorawan. Accessed: June 2026. 2024.

[RKS17] Usman Raza, Parag Kulkarni, and Mahesh Sooriyabandara. "Low Power Wide Area Networks: An Overview". In: *IEEE Communications Surveys & Tutorials* 19.2 (2017), pp. 855–873. DOI: 10.1109/COMST.2017.2652320. URL: https://ieeexplore.ieee.org/document/7815384.

[Sem15] Semtech Corporation. *AN1200.22: LoRa Modulation Basics*. Application Note AN1200.22, Rev. 2. Semtech Corporation, 2015. URL: https://www.semtech.com/uploads/documents/an1200.22.pdf.

[Sem19] Semtech Corporation. *SX1276/77/78/79 Datasheet: 137 MHz to 1020 MHz Low Power Long Range Transceiver, Rev. 7*. Datasheet. Accessed: June 2026. Semtech Corporation, 2019. URL: https://semtech.my.salesforce.com/sfc/p/#E0000000JelG/a/2R000000Un7o/A_BEIBFLlTEHzDsNK9wXcPkI2GRBshLgm.WoEZOc3eA.

[Sie+12] Matti Siekkinen et al. "How Low Energy is Bluetooth Low Energy? Comparative Measurements with ZigBee/802.15.4". In: *Proceedings of the IEEE Wireless Communications and Networking Conference Workshops (WCNCW)*. 2012, pp. 232–237. DOI: 10.1109/WCNCW.2012.6215496. URL: https://ieeexplore.ieee.org/document/6215496.

[Sil24] Silicon Laboratories. *Bluetooth Connection Flowcharts — Bluetooth LE Fundamentals*. https://docs.silabs.com/bluetooth/latest/bluetooth-fundamentals-connections/connection-flowcharts. Accessed: 2025. 2024.

[SP97] Rainer Storn and Kenneth Price. "Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces". In: *Journal of Global Optimization* 11.4 (1997), pp. 341–359. DOI: 10.1023/A:1008202821328. URL: https://link.springer.com/article/10.1023/A:1008202821328.

[SS20] Ken Schwaber and Jeff Sutherland. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org. Accessed: June 2026. Nov. 2020. URL: https://scrumguides.org/scrum-guide.html.

[SYA11] Fatih Senel, Mohamed F. Younis, and Kemal Akkaya. "Bio-Inspired Relay Node Placement Heuristics for Repairing Damaged Wireless Sensor Networks". In: *IEEE Transactions on Vehicular Technology* 60.4 (2011), pp. 1835–1848. DOI: 10.1109/TVT.2011.2118776. URL: https://ieeexplore.ieee.org/document/5715616.

[TB15] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. 4th ed. Pearson, 2015. ISBN: 978-0133591620.

[Tea24] Team SimPy. *SimPy: Discrete-Event Simulation for Python*. https://simpy.readthedocs.io/. Accessed: June 2026. 2024.

[Tex08] Texas Instruments. *DN024 – Antenna Quick Guide*. Texas Instruments Design Note. Accessed: 2026-06-06. 2008. URL: https://www.ti.com/lit/an/swra161b/swra161b.pdf.

[Tex12] Texas Instruments. *TPS6300x High-Efficient Single Inductor Buck-Boost Converter with 1.8-A Switches Datasheet (SLVS640F)*. Datasheet SLVS640F. Accessed: June 2026. Texas Instruments Incorporated, 2012. URL: https://www.ti.com/lit/ds/symlink/tps63001.pdf.

[Tos+17] Jacopo Tosi et al. "Performance Evaluation of Bluetooth Low Energy: A Systematic Review". In: *Sensors* 17.12 (2017), p. 2898. DOI: 10.3390/s17122898. URL: https://www.mdpi.com/1424-8220/17/12/2898.

[Tse+02] Yu-Chee Tseng et al. "The Broadcast Storm Problem in a Mobile Ad Hoc Network". In: *Wireless Networks* 8.2–3 (2002), pp. 153–167. DOI: 10.1023/A:1013763825347. URL: https://link.springer.com/article/10.1023/A:1013763825347.

[Van17] Lorenzo Vangelista. "Frequency Shift Chirp Modulation: The LoRa Modulation". In: *IEEE Signal Processing Letters* 24.12 (2017), pp. 1818–1821. DOI: 10.1109/LSP.2017.2762960. URL: https://ieeexplore.ieee.org/document/8067462.

[Vir+20] Pauli Virtanen et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2. URL: https://doi.org/10.1038/s41592-019-0686-2.

[WC02] Brad Williams and Tracy Camp. "Comparison of Broadcasting Techniques for Mobile Ad Hoc Networks". In: *Proceedings of the 3rd ACM International Symposium on Mobile Ad Hoc Networking and Computing (MobiHoc)*. 2002, pp. 194–205. DOI: 10.1145/513800.513825. URL: https://dl.acm.org/doi/10.1145/513800.513825.

[Wir24] WirelessPi. *Understanding LoRa PHY — Long Range Physical Layer*. https://wirelesspi.com/understanding-lora-phy-long-range-physical-layer/. Accessed: June 2026. 2024.

[Won+24] Adrian Wing-Lam Wong et al. "Multi-Hop and Mesh for LoRa Networks: Recent Advancements, Issues, and Recommended Applications". In: *ACM Computing Surveys* 56.6 (2024), pp. 1–43. DOI: 10.1145/3638241. URL: https://dl.acm.org/doi/10.1145/3638241.

[ZXi24] ZXing Authors. *ZXing ("Zebra Crossing") Barcode Scanning Library for Java and Android*. GitHub Repository. Accessed: June 2026. 2024. URL: https://github.com/zxing/zxing.

Part II.

Appendices

A. AT Command Reference

This appendix provides the complete reference for the A3MESH AT-command interface implemented in the node firmware (version `1.0.0`). Commands are issued by the companion mobile application over BLE using the Nordic UART Service (NUS) write characteristic (`UUID 6E400002`).

Important: every command string must be terminated with a carriage-return followed by a line-feed (`\r\n`, i.e. the byte sequence `0x0D 0x0A`). The firmware parser discards any input that does not carry this terminator.

The node always ends its response with one of the following terminal status lines:

- `+OK\r\n` — the command completed successfully.
- `+ERR=`n`\r\n` — the command failed; n is the error code defined in Table A.1.

Notation. In the syntax columns below, tokens enclosed in angle brackets (⟨name⟩) represent mandatory arguments to be substituted by the caller. Node identifiers and group identifiers are 32-bit unsigned integers encoded as exactly eight lower-case hexadecimal characters (e.g. `0ab1c2d3`). Cryptographic keys are 32-byte (256-bit) values encoded as exactly 64 hexadecimal characters.

Table A.1.: AT-command error codes.

Code	Meaning
`+ERR=1`	Input does not begin with the prefix `AT`.
`+ERR=2`	Command not recognised by the parser.
`+ERR=3`	Argument parse error or value outside the permitted range.
`+ERR=4`	LoRa transmit queue full; the radio is busy.
`+ERR=5`	Cryptographic error or required key not provisioned.
`+ERR=6`	Key storage table full; no slot available for the new key.

A.1. Basic Command

A.1.1. `AT` — Connectivity check

No-operation command used to verify that the BLE link and the firmware command parser are operational. Produces no side effects.

Syntax	Response
`AT\r\n`	`+OK\r\n`

A.2. Messaging Commands

A.2.1. AT+SEND — Transmit plain (unencrypted) message

Constructs an unencrypted k_data_plain packet addressed to ⟨dst⟩ and queues it for LoRa transmission. Set ⟨dst⟩ to ffffffff to broadcast to all reachable nodes. The payload field may contain any sequence of printable ASCII characters up to 233 bytes in length.

Syntax	Response
AT+SEND=⟨dst⟩,⟨payload⟩\r\n	+OK\r\n (success) +ERR=3\r\n (invalid arguments) +ERR=4\r\n (TX queue full)

A.2.2. AT+SENDE2E — Transmit end-to-end encrypted message

Encrypts ⟨payload⟩ using the pre-provisioned AES-256-GCM node-to-node key for peer ⟨dst⟩ and transmits the resulting ciphertext as a k_enc_node2node packet. The peer key for ⟨dst⟩ must have been loaded previously with AT+SETPEERKEY.

Syntax	Response
AT+SENDE2E=⟨dst⟩,⟨payload⟩\r\n	+OK\r\n (success) +ERR=3\r\n (invalid arguments) +ERR=5\r\n (key not provisioned) +ERR=4\r\n (TX queue full)

A.2.3. AT+SENDGRP — Transmit group-encrypted broadcast

Encrypts ⟨payload⟩ with the pre-provisioned AES-256-GCM group key for ⟨group_id⟩ and transmits the ciphertext as a broadcast (k_enc_group packet addressed to all nodes). All nodes holding the same group key will be able to decrypt the message. The group key must have been loaded in advance with AT+SETGRPKEY.

Syntax	Response
AT+SENDGRP=⟨group_id⟩,⟨payload⟩\r\n	+OK\r\n (success) +ERR=3\r\n (invalid arguments) +ERR=5\r\n (key not provisioned) +ERR=4\r\n (TX queue full)

A.2.4. AT+SENDNET — Transmit network-layer encrypted message

Encrypts ⟨payload⟩ with the shared network-wide AES-256-GCM key and transmits the ciphertext as a k_enc_network packet addressed to ⟨dst⟩. The network key must have been loaded with AT+SETNETKEY. At the destination, the firmware decrypts the payload transparently and delivers it through the standard +RCV notification, so the receiving application requires no special handling for this encryption scope.

Syntax	Response
AT+SENDNET=⟨dst⟩,⟨payload⟩\r\n	+OK\r\n (success) +ERR=3\r\n (invalid arguments) +ERR=5\r\n (key not provisioned) +ERR=4\r\n (TX queue full)

A.2.5. AT+SENDKX — Transmit key-exchange packet

Transmits a k_key_exchange packet to ⟨dst⟩ carrying an arbitrary ⟨payload⟩ (typically a cryptographic public value or shared secret in hexadecimal encoding). The time-to-live counter is set to the minimum value (0x01), so the packet is never relayed beyond direct radio range.

Syntax	Response
AT+SENDKX=⟨dst⟩,⟨payload⟩\r\n	+OK\r\n (success) +ERR=3\r\n (invalid arguments) +ERR=4\r\n (TX queue full)

A.3. Query Commands

A.3.1. AT+NODEID? — Query local node identifier

Returns the unique 32-bit identifier of this node. The identifier is derived at firmware startup from the lower four bytes of the ESP32 BLE MAC address.

Syntax	Response
AT+NODEID?\r\n	+NODEID=⟨node_id_hex8⟩\r\n

A.3.2. AT+NEIGHBORS? — Query neighbor table

Returns the current contents of the neighbor table (up to ten entries). Each entry reports the neighbour's node identifier, the received signal strength (RSSI in dBm) from the most recent discovery beacon, and the system uptime timestamp (in ms) at which that beacon was received.

Syntax	Response
AT+NEIGHBORS?\r\n	*No neighbours:* +NEIGHBORS=0\r\n +OK\r\n *With n neighbours:* +NEIGHBORS=*n*\r\n +N:⟨id⟩,RSSI:⟨dBm⟩,T:⟨ms⟩;\r\n *(one line per neighbour)* +OK\r\n

A.3.3. AT+INFO? — Query system diagnostics

Returns a multi-field diagnostic report covering the node identity, firmware version, battery charge level, device uptime, and current LoRa radio configuration.

Syntax	Response
AT+INFO?\r\n	+NODEID=⟨hex8⟩\r\n +FWVER=⟨version⟩\r\n +BATT=⟨pct⟩\r\n +UPTIME=⟨hh:mm:ss⟩\r\n +FREQ=⟨MHz⟩\r\n +SF=SF⟨n⟩\r\n +BW=⟨n⟩ kHz\r\n +CR=4/⟨n⟩\r\n +NEIGHBORS=⟨count⟩\r\n +OK\r\n

A.3.4. AT+LORA? — Query LoRa radio configuration

Returns the current spreading factor, signal bandwidth, and coding rate of the LoRa radio without modifying any settings.

Syntax	Response
AT+LORA?\r\n	+SF=SF⟨n⟩\r\n +BW=⟨n⟩ kHz\r\n +CR=4/⟨n⟩\r\n +OK\r\n

A.4. Radio Configuration Commands

A.4.1. AT+LORA — Set LoRa radio parameters

Reconfigures the LoRa transceiver immediately. The permitted parameter ranges are: spreading factor $\langle \mathrm{SF} \rangle \in \{7, \ldots, 12\}$; bandwidth $\langle \mathrm{BW} \rangle \in \{125, 250, 500\}$ kHz; coding-rate denominator $\langle \mathrm{CR} \rangle \in \{5, \ldots, 8\}$ (corresponding to rates 4/5 through 4/8). All three parameters must be supplied simultaneously; partial updates are not supported. Any out-of-range value triggers +ERR=3 and leaves the radio configuration unchanged.

Syntax	Response
AT+LORA=⟨SF⟩,⟨BW⟩,⟨CR⟩\r\n	+OK\r\n (success) +ERR=3\r\n (invalid parameters)

A.5. Key Management Commands

A.5.1. AT+SETNETKEY — Provision shared network key

Loads the 32-byte (256-bit) network-wide AES-256-GCM key into volatile memory. ⟨key_64hex⟩ must be exactly 64 hexadecimal characters. This key is subsequently used by AT+SENDNET for transparent network-layer encryption and decryption.

Syntax	Response
AT+SETNETKEY=⟨key_64hex⟩\r\n	+OK\r\n (success) +ERR=3\r\n (invalid key format)

A.5.2. AT+SETPEERKEY — Provision node-to-node peer key

Loads the 32-byte AES-256-GCM symmetric key for peer ⟨peer_id⟩ into the firmware key table. ⟨key_64hex⟩ must be exactly 64 hexadecimal characters. Once provisioned, this key enables end-to-end encrypted communication with that specific peer via AT+SENDE2E.

Syntax	Response
AT+SETPEERKEY=⟨peer_id⟩,⟨key_64hex⟩\r\n	+OK\r\n (success) +ERR=3\r\n (parse error) +ERR=6\r\n (key table full)

A.5.3. AT+SETGRPKEY — Provision group key

Loads the 32-byte AES-256-GCM group key for group identifier ⟨group_id⟩ into the firmware key table. ⟨key_64hex⟩ must be exactly 64 hexadecimal characters. Once provisioned, this key enables group-encrypted broadcasts via AT+SENDGRP.

Syntax	Response
AT+SETGRPKEY=⟨group_id⟩,⟨key_64hex⟩\r\n	+OK\r\n (success) +ERR=3\r\n (parse error) +ERR=6\r\n (key table full)

A.5.4. AT+CLRKEYS — Clear all provisioned keys

Erases all stored keys (network, peer, and group) from volatile memory and resets the replay-protection counters. This operation is irreversible within the current power cycle; all keys must be re-provisioned if secure messaging is to continue.

Syntax	Response
AT+CLRKEYS\r\n	+OK\r\n

A.6. Unsolicited Notifications

The firmware pushes unsolicited notification strings to the mobile application via BLE GATT notifications on the NUS RX characteristic (`UUID 6E400003`) independently of any command exchange. In all formats, ⟨src⟩ is the originating node identifier, ⟨fwd⟩ is the node that last forwarded the packet (its `node_id` field), and ⟨dst⟩ is the destination address as carried in the packet header. All notifications are terminated with `\r\n`.

Table A.2.: Unsolicited notifications issued by the firmware.

Notification format	**Triggering event**
`+RCV=⟨src⟩,⟨fwd⟩,⟨dst⟩,⟨payload⟩`	Plain or network-decrypted message delivered locally.
`+RCVE2E=⟨src⟩,⟨fwd⟩,⟨payload⟩`	Node-to-node AES-256-GCM message addressed to this node.
`+RCVGRP=⟨group_id⟩,⟨src⟩,⟨fwd⟩,⟨payload⟩`	Group-encrypted broadcast decrypted with the matching group key.
`+KX=⟨src⟩,⟨fwd⟩,⟨payload⟩`	Key-exchange packet received; payload carries the cryptographic value.
`+DISC=⟨node_id⟩`	New neighbour discovered via a received discovery beacon.

B. PCB Schematics

This appendix contains the full electrical schematic (Figure B.1), followed by the front copper layer (Figure B.2) and the back copper layer (Figure B.3) of the printed circuit board, as exported from KiCad.

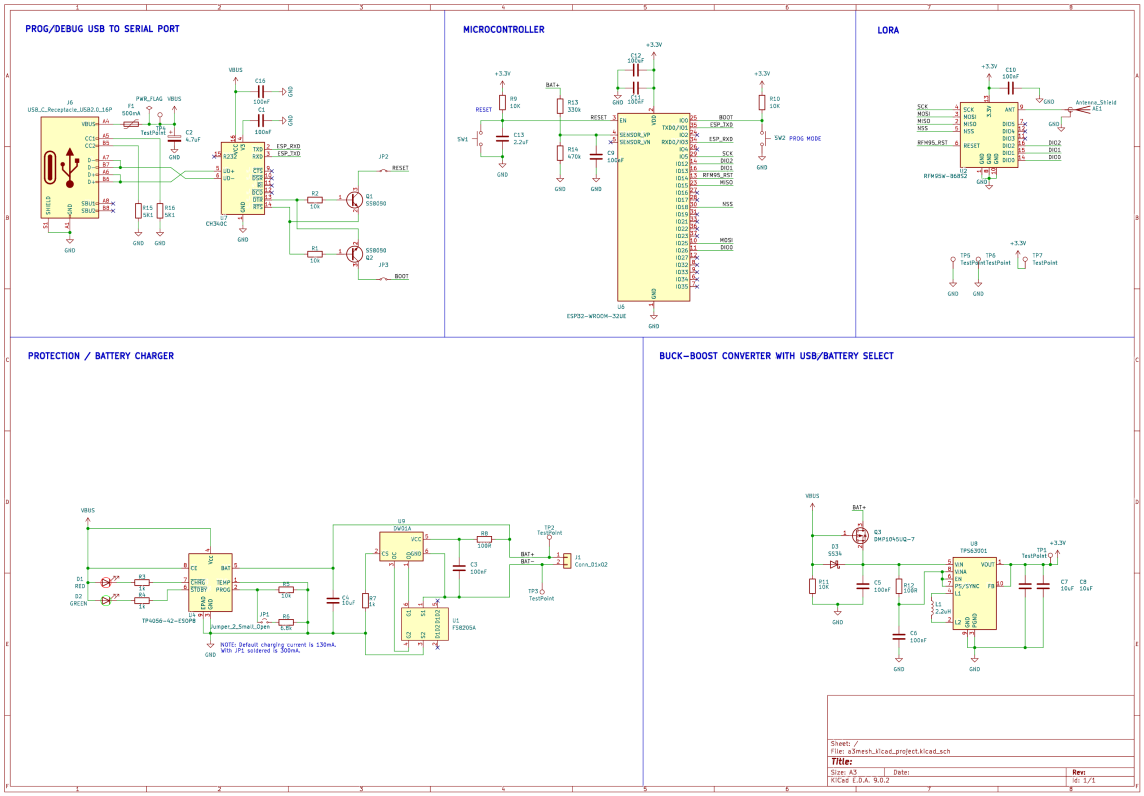


Figure B.1.: Electrical schematic.

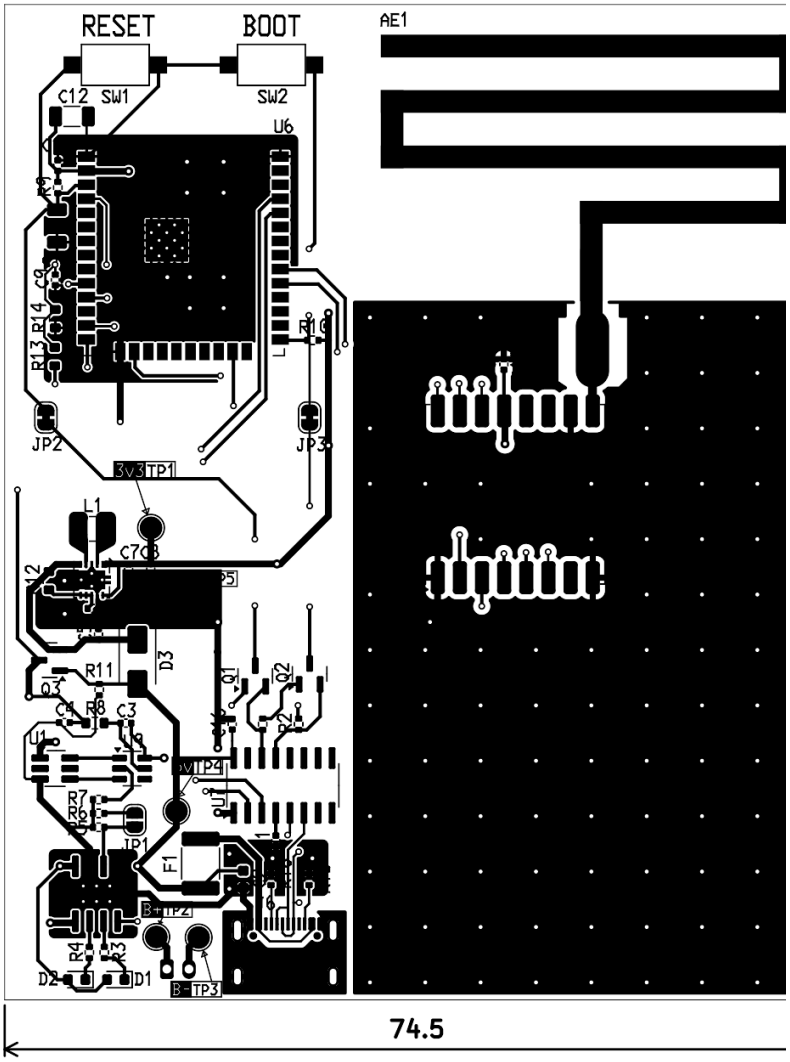


Figure B.2.: Front copper layer (F.Cu) with silkscreen.

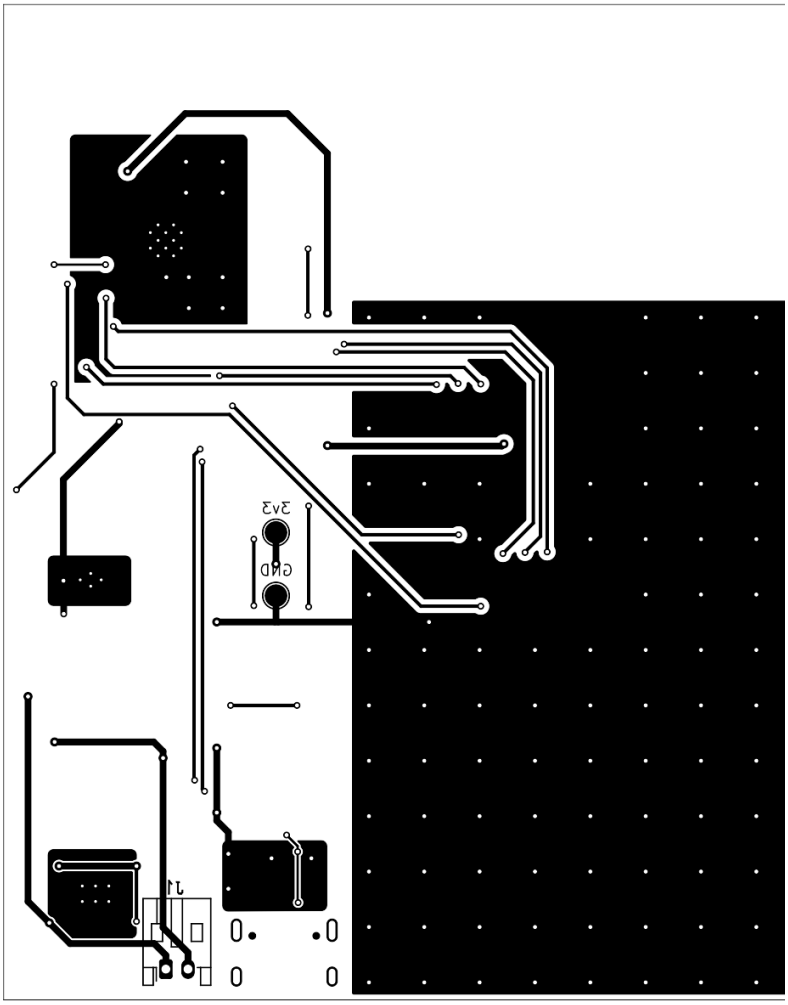


Figure B.3.: Back copper layer (B.Cu) with silkscreen.

C. Bill of Materials

Table C.1 lists all components assembled on the A3MESH PCB, exported from the KiCad schematic. Components sourced through JLCPCB carry an LCSC part number in the rightmost column; parts obtained separately are indicated with a dash. Test points, PCB logos, and bridged solder jumpers are excluded from the table as they require no physical procurement.

Table C.1.: A3MESH bill of materials.

Designator(s)	Description	Value	Package	Qty	LCSC #
Microcontroller and Wireless Modules					
U6	ESP32 MCU module (Wi-Fi + BLE)	ESP32-WROOM-32UE	LGA-38 module	1	C701344
U2	LoRa transceiver module (SX1276, 868 MHz)	RFM95W-868S2	SMD module	1	—[1]
U7	USB-to-UART bridge	CH340C	SOIC-16	1	C7464026
Power Management					
U4	Li-ion single-cell battery charger	TP4056	ESOP-8	1	C16581
U8	Buck-boost DC-DC converter (3.3 V)	TPS63001	WSON-10	1	C28060
U9	Li-ion battery protection IC	DW01A	SOT-23-6	1	C351410
U1	Dual N-channel MOSFET (battery cutoff)	FS8205A	SOT-23-6	1	C908265
Q3	P-channel MOSFET (power path switch)	DMP1045UQ-7	SOT-23	1	C706746
D3	Schottky diode (reverse-polarity protection)	SS34	SMA	1	C8678
F1	Polyfuse (overcurrent protection)	500 mA	1812	1	C12559
L1	Power inductor (DC-DC converter)	2.2 µH	1008	1	C5832372
Status Indicators and User Interface					
D1	LED status indicator (red)	Red LED	0603	1	C2286
D2	LED status indicator (green)	Green LED	0603	1	C2289
Q1, Q2	NPN transistor (LED driver)	SS8050	SOT-23	2	C916392
SW1, SW2	Tactile pushbutton switch	TS-1088-AR02016	SMD	2	C118141
Connectors					
J6	USB-C receptacle (USB 2.0, 16-pin)	GCT USB4105	SMD top-mount	1	C2765186
J1	JST-PH 2-pin battery connector	JST-PH-2	SMD	1	—[2]
Capacitors					
C1, C3, C5, C6, C9, C10, C11, C16	Decoupling capacitor	100 nF	0402	8	C14663

(continued on next page)

(continued from previous page)

Designator(s)	Description	Value	Package	Qty	LCSC #
C4, C7, C8	Bulk decoupling capacitor	10 µF	0402	3	C15525
C2	Filter capacitor	4.7 µF	0603	1	C19666
C13	Filter capacitor	2.2 µF	1206	1	C12530
C12	Bulk filter capacitor	100 µF	1206	1	C15008
Resistors					
R1, R2, R5, R9, R10, R11	Pull-up / pull-down resistor	10 kΩ	0402	6	C25744
R3, R4, R7	Current-limiting resistor	1 kΩ	0402	3	C11702
R8, R12	LED current-limiting resistor	100 Ω	0603	2	C22775
R15, R16	USB-C CC pull-down resistor	5.1 kΩ	0402	2	C25905
R13	Battery voltage divider (high-side)	330 kΩ	0603	1	C23137
R14	Battery voltage divider (low-side)	470 kΩ	0603	1	C23178
R6	Charge-current programming resistor	6.8 kΩ	0402	1	C2906876

[1] The RFM95W-868S2 module was sourced directly from AliExpress and hand-soldered; it is therefore absent from the JLCPCB LCSC BOM.
[2] The JST-PH 2-pin battery connector was sourced from Amazon and hand-soldered; it is therefore absent from the JLCPCB LCSC BOM.

D. Source Code Repositories

All source code produced for this project is hosted on GitHub: `https://github.com/alcaaron/tfg_manuscript.git`. The LaTeX manuscript itself is the root repository and contains the remaining four artefacts as git submodules, so that a single `git clone --recurse-submodules` retrieves the complete project tree. The top-level layout is shown below.

```
tfg_manuscript/
├── tfe.tex ..... main LaTeX source
├── tfe.bib ..... BibTeX bibliography
├── tfgitic.cls ..... UPC/ITIS thesis document class
├── custom_cmd.sty ..... custom LaTeX macros
├── images/ ..... figures embedded in the document
├── tfg_firmware/ ..... [submodule] ESP32 node firmware
├── tfg_hardware/ ..... [submodule] KiCad PCB design
├── tfg_software/ ..... [submodule] Android companion app
├── MeshtasticatorMeshAlgoComp/ ..... [submodule] network simulator
└── tfg_3d/ ..... [submodule] mechanical enclosure
```

D.1. Node Firmware

Repository: `https://github.com/alcaaron/tfg_firmware`

The firmware is a PlatformIO project targeting the ESP32 (Heltec WiFi LoRa 32 V3 board definition). All application logic is written in C++17 on top of the Arduino framework and FreeRTOS. The build entry point is `platformio.ini`; flashing requires only `pio run --target upload`.

```
tfg_firmware/firmware/
├── platformio.ini ..... board, framework and library dependencies
├── include/
│   ├── config.h ..... all compile-time constants (SF, BW, TTL, keys...)
│   └── PassiveACK.h ..... passive-ACK forward-cancel interface
├── src/
│   └── main.cpp ..... FreeRTOS task wiring + AT command parser
└── lib/
    ├── ble/ ..... NUS GATT server; BLE TX/RX tasks
    ├── crypto/ ... AES-256-GCM layer (AESGCM, CryptoManager, CryptoKeys, NonceManager, ReplayProtection)
    ├── lora/ ..... LoRa radio driver (RadioLib wrapper); TX/RX tasks
    ├── mesh/ . A3MESHManager (neighbor table, Smart Gossip, duplicate cache) + neighbor-check task
    └── packet/ ..... A3MESH packet serialization / deserialization
```

```
└── power/ ..... Battery LUT + adaptive CPU-frequency manager
```

The only file a developer needs to touch for most changes is `include/config.h` (radio parameters, hop limits, gossip tuning constants) and the `lib/mesh/` module (routing logic). The AT command set is implemented exclusively in `src/main.cpp` in the `BLE2LoRaTask` function.

D.2. PCB Hardware Design

Repository: `https://github.com/alcaaron/tfg_hardware`

The hardware design is a two-layer KiCad 8 project. Opening `a3mesh_kicad_project.kicad_pro` in KiCad loads both the schematic and the PCB layout. Custom symbols and footprints that are not available in the standard KiCad libraries are stored alongside the project.

```
tfg_hardware/a3mesh_kicad_project/
├── a3mesh_kicad_project.kicad_pro ..... KiCad project file
├── a3mesh_kicad_project.kicad_sch ..... Electrical schematic
├── a3mesh_kicad_project.kicad_pcb ..... PCB layout
├── a3mesh_kicad_project.csv ..... Exported BOM (KiCad format)
├── New_Library.kicad_sym ..... Custom schematic symbol library
├── symbol_lib/
│   ├── dw01.kicad_sym ..... DW01A battery protection IC
│   ├── fs8205.kicad_sym ..... FS8205A dual N-MOSFET
│   └── tp4056.kicad_sym ..... TP4056 Li-ion charger IC
├── RF_Library.pretty/
│   └── ANTENNA_868_915_955_MONOPOLE_PCB.kicad_mod ..... PCB monopole antenna footprint
├── production/
│   ├── bom.csv ..... LCSC-referenced BOM
│   ├── positions.csv ..... Pick-and-place centroids
│   └── netlist.ipc ..... IPC-D-356 netlist
└── KICAD_OUTPUT_FILES/GBR/ ..... Gerber and drill files for manufacture
```

D.3. Android Companion Application

Repository: `https://github.com/alcaaron/tfg_software`

The companion app is written in Kotlin and built with Gradle. It requires Android Studio (Hedgehog or later) and a device running Android 10 or above. The project follows the single-activity, multi-fragment navigation pattern and persists contacts and messages with the Room database.

```
tfg_software/
├── build.gradle ..... Root Gradle configuration
├── settings.gradle ..... Project module declarations
└── app/src/main/
    ├── AndroidManifest.xml ..... Android application manifest
    └── java/com/punchthrough/blestarterappandroid/
        ├── MainActivity.kt ..... Application entry point and BLE device scanning interface
        ├── BleConnectionService.kt .. Foreground service managing the BLE connection lifecycle
        ├── BleViewModel.kt ..... AT command dispatcher and shared UI/application state holder
        ├── ble/ ..... Low-level BLE communication layer
        │   ├── ConnectionManager.kt ..... GATT operation queue and callback handling
        │   └── BleExtensions.kt ..... Helper extensions for BLE/GATT operations
        ├── crypto/ ..... Cryptographic key exchange, derivation, and storage
        │   ├── EcdhHelper.kt ..... ECDH-P256 key agreement implementation
        │   ├── HkdfSha256.kt ..... HKDF-SHA256 key derivation functions
        │   ├── A3MeshKeyStore.kt ..... Peer and group key storage
        │   └── NetworkKeyManager.kt ..... Network-wide key management
        ├── data/ ..... Room persistence layer
        │   ├── model/ ..... Entity definitions such as Contact.kt and Message.kt
        │   ├── dao/ ..... Data access objects such as ContactDao.kt and MessageDao.kt
        │   └── database/ ..... Room database configuration and AppDatabase.kt
        └── ui/ ..... User interface fragments and adapters
            ├── ChatsFragment.kt ..... Conversation list screen
            ├── ChatActivity.kt ..... Message thread view
            ├── ContactsFragment.kt ..... Contact management interface
            ├── DeviceFragment.kt ..... Node diagnostics and device information view
            └── SettingsFragment.kt ..... Language settings and key provisioning interface
    └── res/
        ├── layout/ ..... XML layout definitions for Android views
        ├── navigation/ ..... Navigation Component graph
        └── values[-ca|-es|-night]/ ..... Localized strings (EN/CA/ES) and theme resources
```

D.4. Network Simulator

Repository: `https://github.com/alcaaron/MeshtasticatorMeshAlgoComp`

The simulator is a Python 3 discrete-event framework built on SimPy, derived from Meshtasticator and extended with a pluggable routing-algorithm interface. Install dependencies with `pip install -r requirements.txt` and run `python batchSim.py` to reproduce the thesis results.

```
MeshtasticatorMeshAlgoComp/
├── batchSim.py ..... Multi-seed parallel batch simulation runner
├── loraMesh.py ..... Main entry point for single-run simulations
├── tuneSmartGossip.py ..... Differential-evolution tuner for Smart Gossip parameters
├── interactiveSim.py ..... Interactive GUI-based simulation runner
├── plotExample.py ..... Result plotting and visualization helpers
├── requirements.txt ..... Python dependencies (e.g., SimPy, NumPy, SciPy)
├── lib/
│   ├── config.py ..... Simulation configuration parameters such as SF, bandwidth, and scenario settings
│   ├── node.py Node model and routing algorithm implementations, including Managed Flood and Adaptive Gossip
│   ├── phy.py ..... Physical-layer channel model based on the 3GPP Suburban Macro scenario
│   ├── mac.py ..... LoRa airtime computation and duty-cycle enforcement logic
│   ├── packet.py ..... Packet representation and duplicate-detection mechanisms
│   ├── discrete_event.py ..... Integration with the SimPy discrete-event simulation engine
│   ├── common.py ..... Metric collection, statistics, and result aggregation utilities
│   ├── point.py ..... Node placement, geometry, and topology utility functions
│   └── gui.py ..... Interactive visualization layer based on Pygame and Matplotlib
└── tests/
    ├── test_phy.py ..... Unit tests for the physical-layer model
    ├── test_point.py ..... Unit tests for topology and point utilities
    ├── test_common.py ..... Unit tests for shared utility and metric modules
    └── test_full_discrete_sim.py .... End-to-end test for the complete discrete-event simulator
```

To implement a new routing algorithm, add a handler in `lib/node.py` following the `ADAPTATIVE_GOSSIP` pattern and register it in `lib/config.py`.

D.5. Mechanical Enclosure

Repository: `https://github.com/alcaaron/tfg_3d`

The enclosure was designed in Autodesk Fusion 360. The repository contains a single `.f3z` archive, which is a self-contained Fusion 360 project file embedding all bodies, components, and manufacturing history.

```
tfg_3d/
└── a3mesh_fusion_project.f3z... Fusion 360 project containing the mechanical enclosure design
```

The `.f3z` file can be imported directly into Fusion 360.